

Лабораторна робота №4. Створення вебзастосунка використовуючи фреймворк Django: розгортання серверу Django, організація маршрутизації та розроблення представлення з використанням шаблонів.

Завдання. Ознайомитися з можливостями фреймворку Django. Виконати встановлення та налаштування. Для заданої предметної галузі та досвіду виконання попередніх лабораторних робіт організувати маршрутизацію запитів та розробити представлення вебзастосунку. Оскільки підключення до бази даних не передбачено, то слід використовувати набір фіксованих даних.

ЗМІСТ

1 ВСТУП ДО DJANGO	5
1.1 Що таке Django?	5
1.2 Встановлення та налаштування Django	7
1.2.1 Пакетний менеджер <code>pip</code>	7
1.2.2 Встановлення віртуального середовища	9
1.2.3 Активація віртуального середовища	11
1.2.4 Установка Django	13
1.2.5 Перевірка установки	14
1.2.6 Деактивація віртуального середовища	15
1.3 Створення першого проекту	15
1.4 Створення першого додатку	19
2 ПРЕДСТАВЛЕННЯ ТА МАРШРУТИЗАЦІЯ	27
2.1 Обробка запиту	27
2.2 Визначення маршрутів та функції <code>path</code> та <code>re_path</code>	32
2.2.1 <code>re_path()</code>	33
2.2.2 Черговість маршрутів	35
2.2.3 Основні елементи синтаксису регулярних виразів	35
2.2.4 Передача значень у функцію	36
2.3 Отримання даних запиту. <code>HttpRequest</code>	38
2.4 <code>HttpResponse</code> і надсилання відповіді	40
2.5 Параметри подань	44
2.5.1 Визначення параметрів через функцію <code>path()</code>	44
2.5.2 Значення для параметрів за замовчуванням	47
2.5.3 Визначення параметрів через функцію <code>re_path()</code>	49
2.6 Вкладені маршрути	51
2.6.1 Функція <code>include()</code>	51
2.6.2 Отримання параметрів	53
2.7 Параметри рядка запиту	55
2.8 Переадресація та надсилання статусних кодів	58
2.8.1 Переадресація	58
2.8.2 Надсилання статусних кодів	60
2.9 Надсилання <code>json</code>	63
2.9.1 Клас <code>JsonResponse</code>	64

2.9.2 Серіалізація довільних об'єктів	65
2.10 Надсилення та отримання кук	67
2.10.1 Встановлення куки	67
2.10.2 Отримання куки	69
3 ШАБЛОНИ.....	71
3.1 Створення та використання шаблонів	71
3.1.1 Функція render()	74
3.1.2 Клас TemplateResponse.....	75
3.2 Передання даних у шаблони	76
3.2.1 Передавання простих даних	76
3.2.2 Передання складних даних	77
3.2.3 Клас TemplateResponse.....	79
3.2.4 Передача об'єктів класів.....	79
3.3 Вбудовані теги шаблонів.....	80
3.3.1 autoescape.....	80
3.3.2 Коментарі.....	82
3.3.3 if..else.....	82
3.3.4 Цикли	84
3.3.5 Визначення змінних	87
3.3.6 Дати	87
3.4 Фільтри шаблонів.....	88
3.4.1 add.....	88
3.4.2 capfirst	89
3.4.3 cut	90
3.4.4 Перевірка значення і значення за замовчуванням.....	90
3.4.5 floatformat	91
3.4.6 Форматування дат.....	93
3.4.7 Операції зі списками	96
3.5 Статичні файли.....	97
3.5.1 Розміщення за замовченням	98
3.5.2 Налаштування шляхів до файлів.....	100
3.6 Клас TemplateView	101
3.6.1 Визначення шаблонів	101
3.6.2 Передання даних у шаблони.....	102
3.7 Конфігурація шаблонів.....	103
3.7.1 Налаштування за замовченням.....	104

3.7.2 Шляхи до шаблонів	104
3.8 Розширення шаблонів і фільтр extends	107
3.9 Вкладені шаблони	111
3.9.1 Фільтр include.....	111
3.9.2 Передання даних.....	112
3.9.2 Передача даних із батьківського шаблону	113

1 ВСТУП ДО DJANGO

1.1 Що таке Django?

Django - це фреймворк для створення веб-застосунків за допомогою мови програмування Python.

Django був створений у 2005 році, коли веб-розробники з газети Lawrence Journal-World почали використовувати Python як мову для створення веб-сайтів. А у 2008 році вийшов публічний перший реліз фреймворку. На сьогоднішній день він продовжує розвиватися. Так, поточною версією фреймворку на момент написання цієї статті є версія 4.2, яка вийшла у квітні 2023 року. Кожен новий реліз фреймворку виходить у середньому кожні 8 місяців. Крім того, постійно виходять оновлення та виправлення у безпеці.

Django є досить популярним. Він використовується на багатьох сайтах, у тому числі таких, як Pinterest, PBS, Instagram, BitBucket, Washington Times, Mozilla та багатьох інших. Фреймворк є безкоштовним. Він розвивається як Open Source, його вихідний код відкритий, його можна знайти репозиторії на <https://github.com/django/django>.

На Django можна створювати широкий діапазон веб-застосунків: від невеликих персональних сайтів до високонавантажених складних веб-сервісів. Django за замовчуванням пропонує готову функціональність для ряду поширених завдань, наприклад, систему аутентифікації, генерацію карт сайту і т.д., завдяки чому нам можна не винаходити велосипед і взяти вже готові компоненти.

У Django багато уваги приділяється безпеці, завдяки чому фреймворк допомагає розробникам уникнути багатьох поширених проблем у системі безпеки, наприклад, sql-ін'єкцій. Фреймворк Django реалізує архітектурний патерн Model-View-Template або скорочено MVT, який є модифікацією поширеного у веб-програмуванні патерну MVC (Model-View-Controller).

Схематично ми можемо представити архітектуру MVT Django наступним чином:

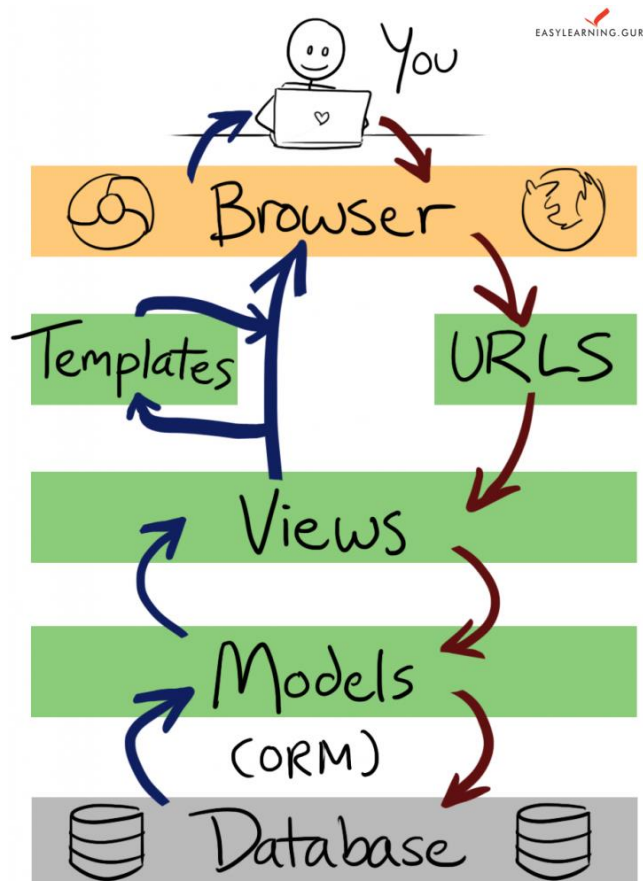


Рисунок 1.1 – Архітектура MVT Django

Основні елементи патерну:

URL dispatcher: при отриманні запиту на підставі запитаної URL-адреси визначає, який ресурс повинен обробляти даний запит.

View: отримує запит, обробляє його та надсилає у відповідь користувачеві деяку відповідь. Якщо для обробки запиту необхідне звернення до моделі та бази даних, *View* взаємодіє з ними. Для створення відповіді можна використовувати *Template* або шаблони. В архітектурі *MVC* цьому компоненту відповідають контролери (але не уявлення).

Model: описує дані, що використовуються у додатку. Окремі класи, як правило, відповідають таблицям у базі даних.

Template: представляє логіку подання у вигляді згенерованої розмітки HTML. У *MVC* цьому компоненту відповідає *View*, тобто уявлення.

Коли до програми надходить запит, *URL dispatcher* визначає, з яким ресурсом зіставляється даний запит і передає цей запит обраному ресурсу. Ресурс фактично

Барбарук Віктор Миколайович. Сучасні технології розробки WEB-застосунків з використанням мови Python

представляє функцію або *View*, який отримує запит та певним чином обробляє його. У процесі обробки *View* може звертатися до моделей і бази даних, отримувати з неї дані, або, навпаки, зберігати дані. Результат обробки запиту відправляється назад, і цей результат бачить користувач у своєму браузері. Як правило, результат обробки запиту представляє згенерований HTML-код, для генерації якого застосовуються шаблони (*Template*).

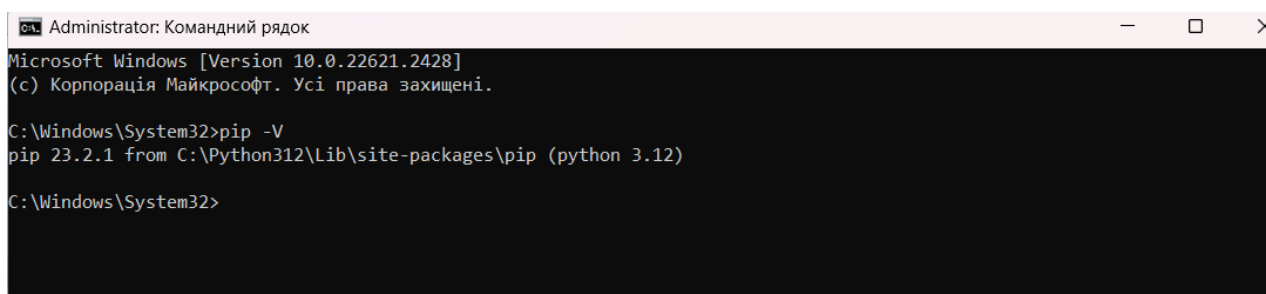
1.2 Встановлення та налаштування Django

Перед початком роботи з *Django* нам природно необхідно встановити інтерпретатор *Python*. Детальніше про це можна почитати у [1].

Існують різні способи встановлення *Django*. Розглянемо спосіб, що рекомендується.

1.2.1 Пакетний менеджер *pip*

Пакети *Django* розміщуються у центральній репозиторії більшості пакетів *Python* - *Package Index (PyPI)*. І для встановлення цього репозиторію нам знадобиться пакетний менеджер *pip*. Менеджер *pip* дозволяє завантажувати пакети та керувати ними. Зазвичай при установці *python* також встановлюється менеджер *pip*. У цьому випадку ми можемо перевірити версію менеджера, виконавши в командному рядку/терміналі команду *pip -V* (*V* - з великої літери) з правами адміністратора:



```
Administrator: Командний рядок
Microsoft Windows [Version 10.0.22621.2428]
(c) Корпорація Майкрософт. Усі права захищені.

C:\Windows\System32>pip -V
pip 23.2.1 from C:\Python312\Lib\site-packages\pip (python 3.12)

C:\Windows\System32>
```

Рисунок 1.2 – Перевірка встановлення пакетного менеджера *pip*

Якщо *pip* не встановлено, ми побачимо помилку типу

"pip" не є внутрішньою чи зовнішньою командою, що виконується програмою або пакетним файлом

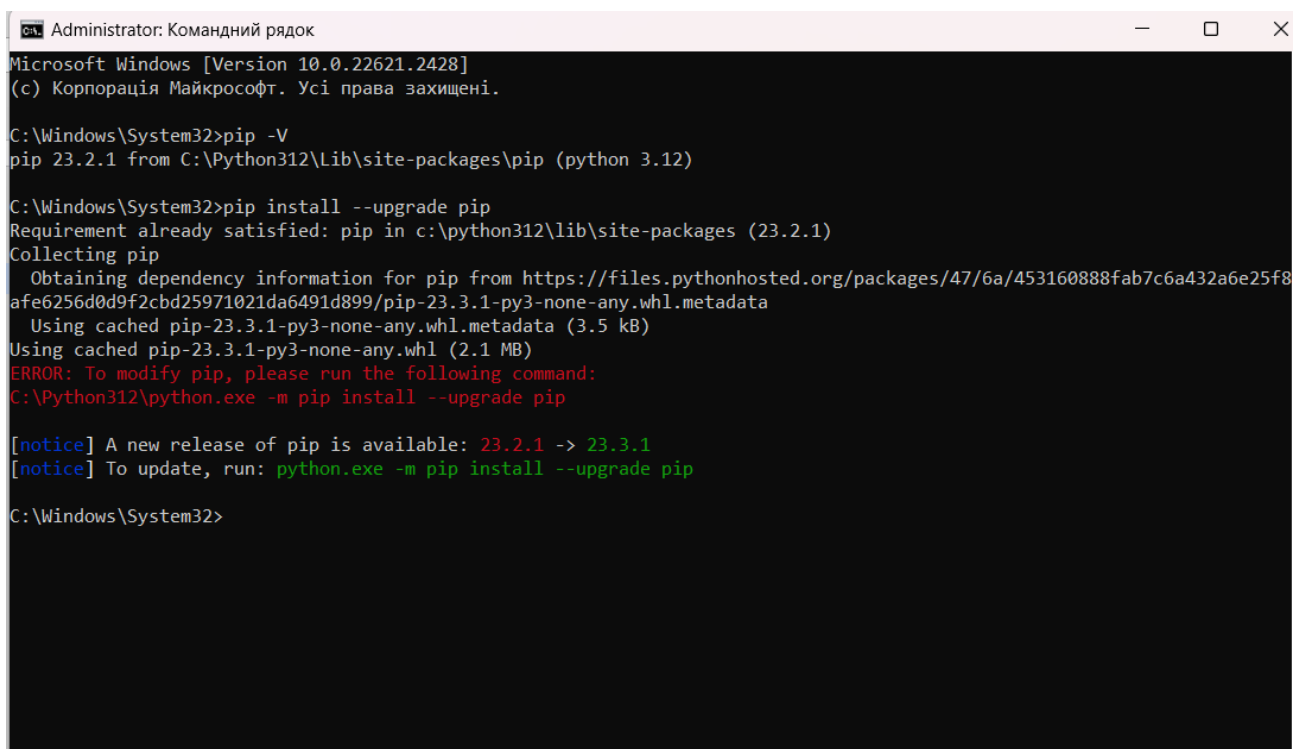
В цьому випадку нам потрібно встановити *pip*. Для цього можна виконати в командному рядку/консолі таку команду:

```
python -m ensurepip --upgrade
```

Якщо *pip* раніше вже було встановлено, можна його оновити за допомогою команди

```
pip install --upgrade pip
```

Якщо оновлення закінчилося exception



```
Administrator: Командний рядок
Microsoft Windows [Version 10.0.22621.2428]
(c) Корпорація Майкрософт. Усі права захищені.

C:\Windows\System32>pip -V
pip 23.2.1 from C:\Python312\Lib\site-packages\pip (python 3.12)

C:\Windows\System32>pip install --upgrade pip
Requirement already satisfied: pip in c:\python312\lib\site-packages (23.2.1)
Collecting pip
  Obtaining dependency information for pip from https://files.pythonhosted.org/packages/47/6a/453160888fab7c6a432a6e25f8afe6256d0d9f2cbd25971021da6491d899/pip-23.3.1-py3-none-any.whl.metadata
  Using cached pip-23.3.1-py3-none-any.whl.metadata (3.5 kB)
  Using cached pip-23.3.1-py3-none-any.whl (2.1 MB)
ERROR: To modify pip, please run the following command:
C:\Python312\python.exe -m pip install --upgrade pip

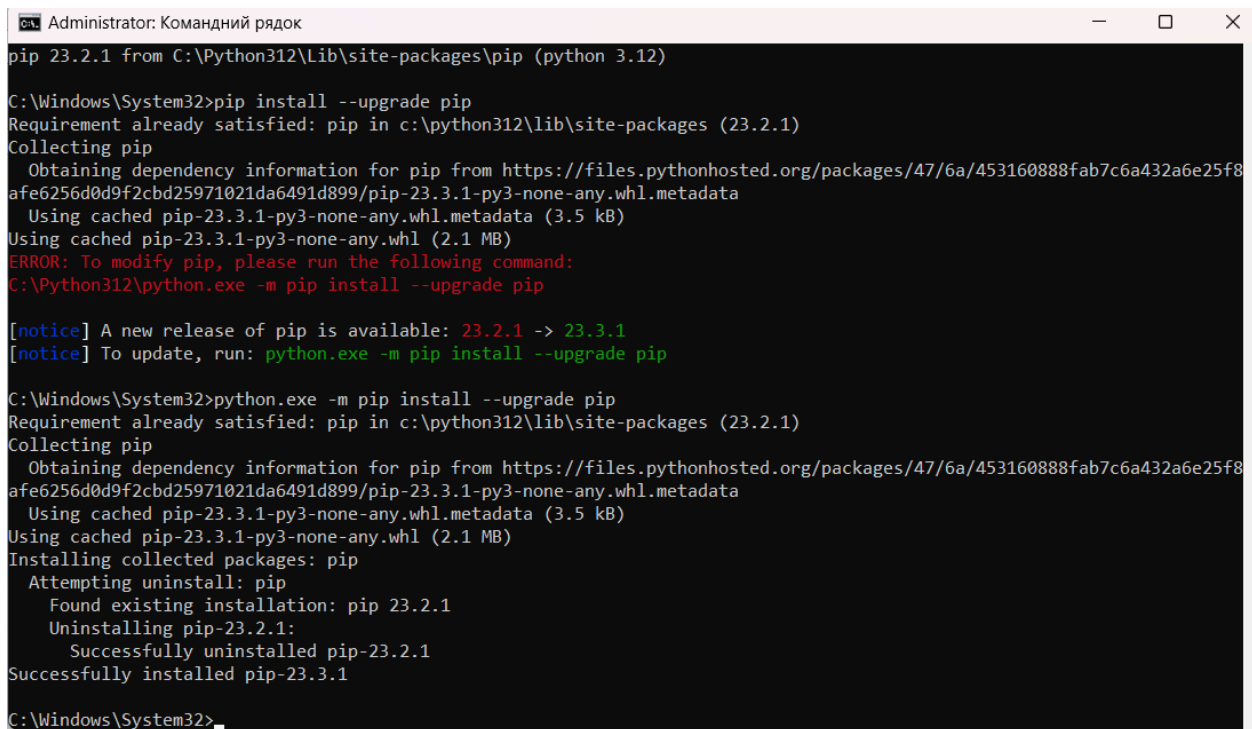
[notice] A new release of pip is available: 23.2.1 -> 23.3.1
[notice] To update, run: python.exe -m pip install --upgrade pip

C:\Windows\System32>
```

то слід скористатися рекомендованою командою:

```
python.exe -m pip install --upgrade pip
```

В результаті отримаємо оновлення *pip*:



```
Administrator: Командний рядок
pip 23.2.1 from C:\Python312\Lib\site-packages\pip (python 3.12)

C:\Windows\System32>pip install --upgrade pip
Requirement already satisfied: pip in c:\python312\lib\site-packages (23.2.1)
Collecting pip
  Obtaining dependency information for pip from https://files.pythonhosted.org/packages/47/6a/453160888fab7c6a432a6e25f8afe6256d0d9f2cbd25971021da6491d899/pip-23.3.1-py3-none-any.whl.metadata
  Using cached pip-23.3.1-py3-none-any.whl.metadata (3.5 kB)
  Using cached pip-23.3.1-py3-none-any.whl (2.1 MB)
ERROR: To modify pip, please run the following command:
C:\Python312\python.exe -m pip install --upgrade pip

[notice] A new release of pip is available: 23.2.1 -> 23.3.1
[notice] To update, run: python.exe -m pip install --upgrade pip

C:\Windows\System32>python.exe -m pip install --upgrade pip
Requirement already satisfied: pip in c:\python312\lib\site-packages (23.2.1)
Collecting pip
  Obtaining dependency information for pip from https://files.pythonhosted.org/packages/47/6a/453160888fab7c6a432a6e25f8afe6256d0d9f2cbd25971021da6491d899/pip-23.3.1-py3-none-any.whl.metadata
  Using cached pip-23.3.1-py3-none-any.whl.metadata (3.5 kB)
  Using cached pip-23.3.1-py3-none-any.whl (2.1 MB)
Installing collected packages: pip
  Attempting uninstall: pip
    Found existing installation: pip 23.2.1
    Uninstalling pip-23.2.1:
      Successfully uninstalled pip-23.2.1
Successfully installed pip-23.3.1

C:\Windows\System32>
```

Рисунок 1.3 – Результат оновлення *pip*

1.2.2 Встановлення віртуального середовища

Віртуальне середовище або *venv* не є невід'ємною частиною розробки *Django*. Однак її рекомендується використовувати, оскільки вона дозволяє створити безліч віртуальних середовищ *Python* на одній операційній системі. Завдяки віртуальному середовищу програма може запускатися незалежно від інших програм на *Python*.

В принципі, можна запускати програми на *Django* і без віртуального середовища. У цьому випадку всі пакети *Django* встановлюються глобально. Однак якщо після створення першої програми вийде нова версія *Django*? Якщо ми захочемо використовувати для другого проекту нову версію *Django*, то через глобальну інсталяцію пакетів доведеться оновлювати перший проект, який використовує стару версію. Це потребує деякої додаткової роботи з оновлення, оскільки не завжди дотримується зворотна сумісність між пакетами. Якщо ми вирішимо використовувати для другого проекту стару версію, то ми втратимо потенційні

переваги нової версії. І використання віртуального середовища якраз дозволяє розмежувати пакети для кожного проекту.

Для роботи з віртуальним середовищем у *python* застосовується вбудований модуль *venv*.

Отже, створимо віртуальне середовище. Спочатку визначимо каталог проєктів *django*. Наприклад, нехай це буде каталог *C:\django*. Насамперед перейдемо в терміналі/командному рядку до цього каталогу за допомогою команди *cd*.

```
cd C:\django
```

Потім для створення віртуального середовища виконаємо таку команду:

```
python -m venv .venv
```

Модулю *venv* передається назва середовища, яке в даному випадку буде називатися *".venv"*. Для найменування віртуальних середовищ немає певних умовностей. Приклад консольного виведення:

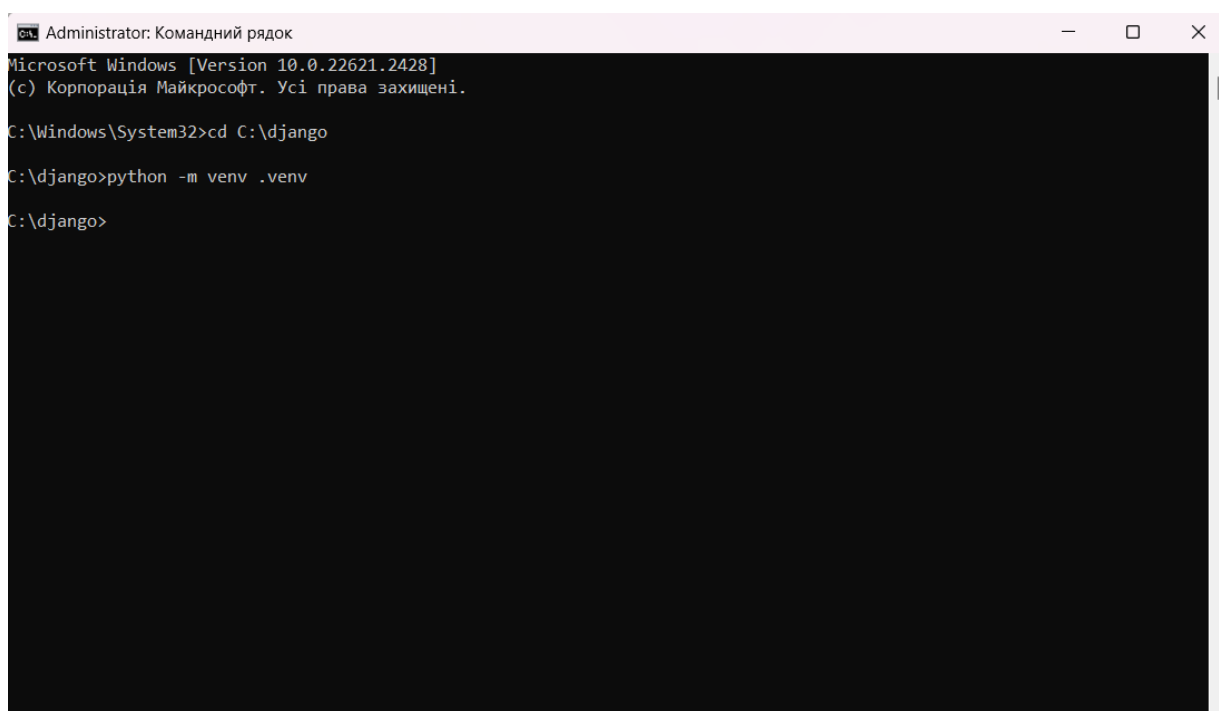


Рисунок 1.4 – Команди створення віртуального середовища

Після цього в поточній папці (*C:\django*) буде створено підкаталог ".venv".

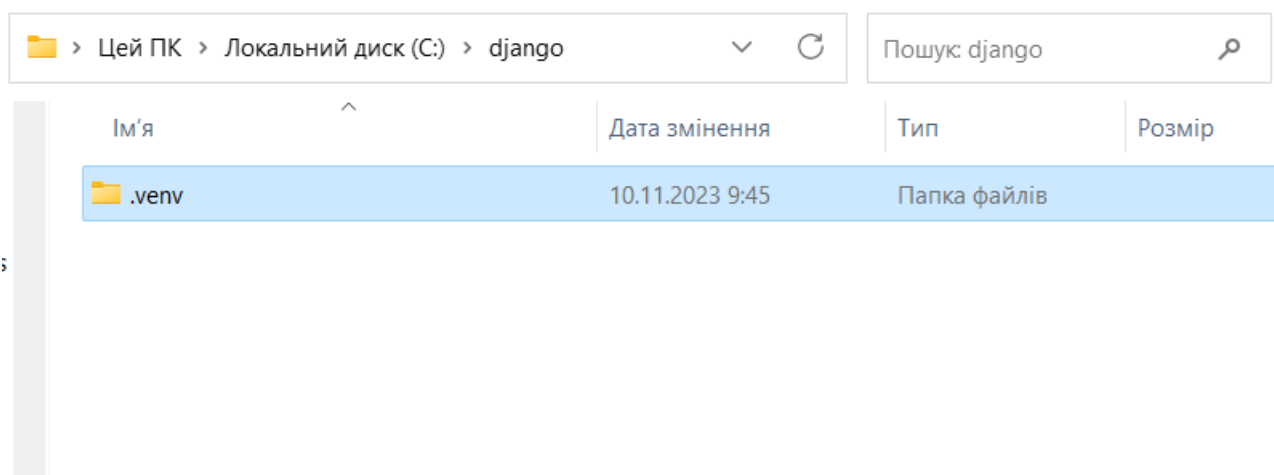


Рисунок 1.5 – Результат створення віртуального середовища

1.2.3 Активація віртуального середовища

Для використання віртуальне середовище необхідно активувати. І щоразу, коли ми працюватимемо з проектом *Django*, пов'язане з ним віртуальне середовище треба активувати. Наприклад, активуємо вище створене середовище, яке знаходиться в поточному каталозі в папці *.venv*. Процес активації трохи відрізняється залежно від операційної системи та від того, які інструменти застосовуються. Так, у Windows можна використовувати командний рядок та *PowerShell*, але між ними є відмінності.

Активація в Windows у командному рядку

Якщо наша ОС – Windows, то у папці *.venv/Scripts/* ми можемо знайти файл *activate.bat*, який активує віртуальне середовище. Так, у Windows активація віртуального середовища в командному рядку буде виглядати так:

```
.venv\Scripts\activate.bat
```

Активація у Windows у PowerShell

Також при роботі на Windows у папці `.venv/Scripts/` ми можемо знайти файл `activate.ps1`, який також активує віртуальне середовище, але застосовується лише у *PowerShell*. Але при роботі з *PowerShell* слід враховувати, що за умовчанням у цій оболонці заборонено використовувати скрипти. Тому перед активацією середовища необхідно встановити дозволи для користувача. Тому для активації віртуального середовища *PowerShell* необхідно виконати дві наступні команди:

```
Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope  
CurrentUser  
.venv\Scripts\Activate.ps1
```

Активация в Linux та MacOS

Для Linux та MacOS активация буде здійснюватися за допомогою наступної команди:

```
source .venv/bin/activate
```

Далі я наводитиму приклади на основі командного рядка Windows, проте всі інші приклади не залежатимуть від того, що використовується - PowerShell або командний рядок, Windows, Linux або MacOS. У будь-якому випадку після успішної активації зліва від поточного каталогу ми побачимо у дужках назву віртуального середовища:

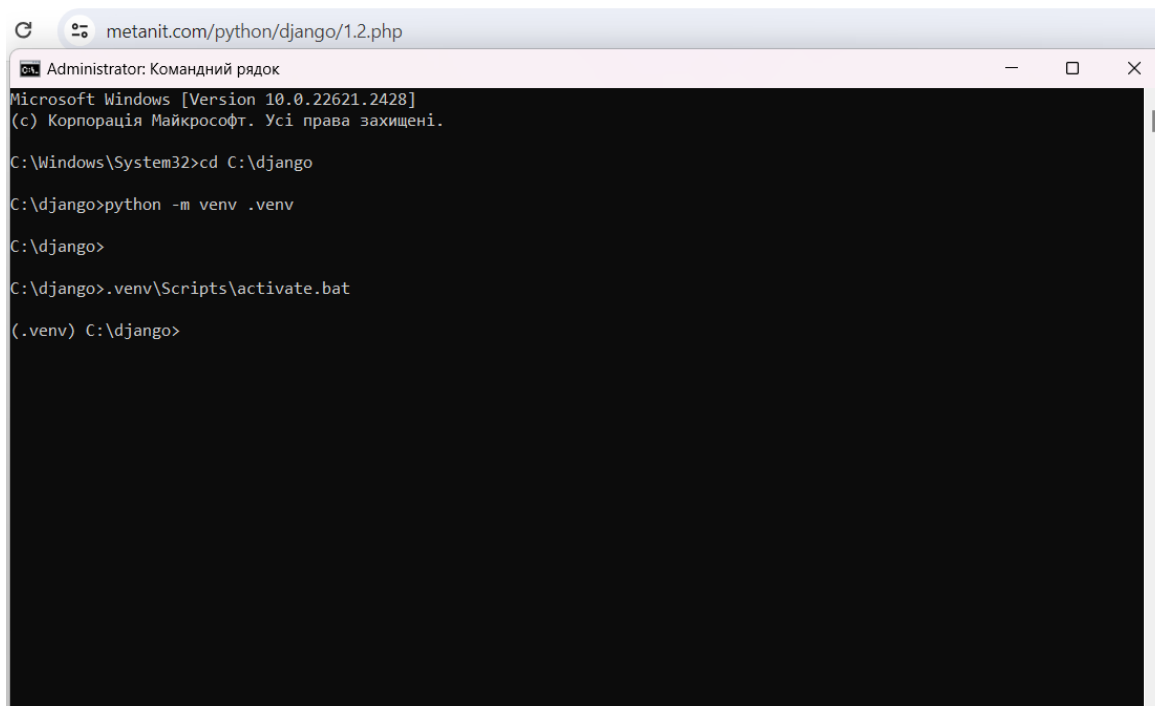


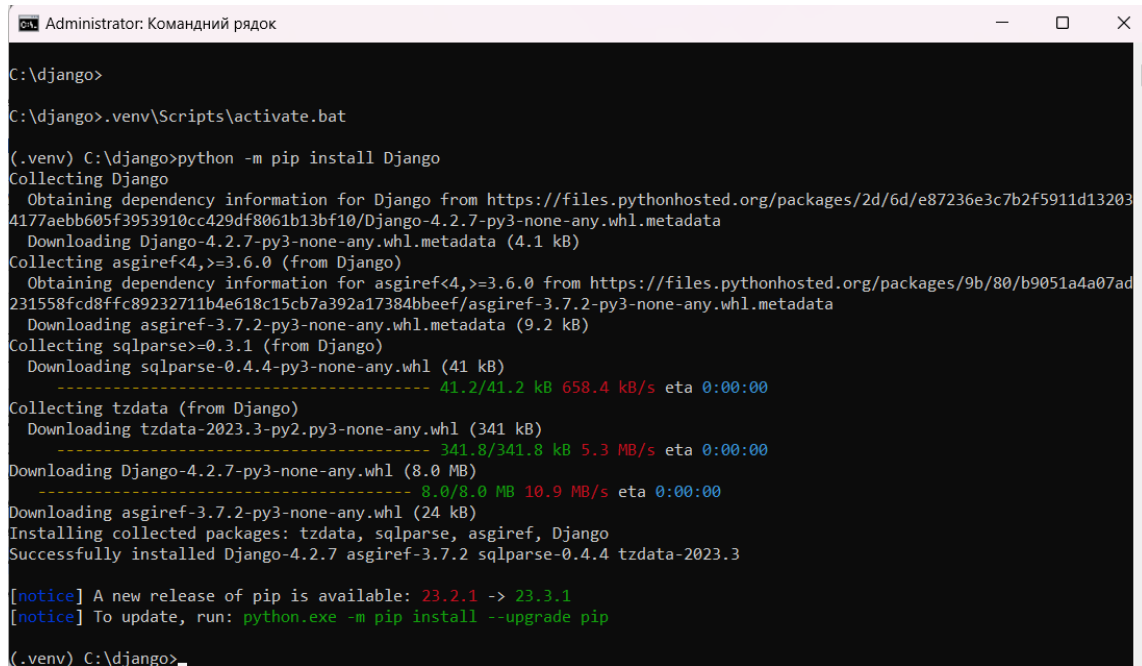
Рисунок 1.6 – Активація віртуального середовища

1.2.4 Установка Django

Після активації віртуального середовища для встановлення Django виконаємо в консолі наступну команду

```
python -m pip install Django
```

Вона встановлює останню версію Django.



```
Administrator: Командний рядок

C:\django>

C:\django>.venv\Scripts\activate.bat

(.venv) C:\django>python -m pip install Django
Collecting Django
  Obtaining dependency information for Django from https://files.pythonhosted.org/packages/2d/6d/e87236e3c7b2f5911d132034177aebb605f3953910cc429df8061b13bf10/Django-4.2.7-py3-none-any.whl.metadata
  Downloading Django-4.2.7-py3-none-any.whl.metadata (4.1 kB)
Collecting asgiref<4,>=3.6.0 (from Django)
  Obtaining dependency information for asgiref<4,>=3.6.0 from https://files.pythonhosted.org/packages/9b/80/b9051a4a07ad231558fcd8fffc89232711b4e618c15cb7a392a17384bbeef/asgiref-3.7.2-py3-none-any.whl.metadata
  Downloading asgiref-3.7.2-py3-none-any.whl.metadata (9.2 kB)
Collecting sqlparse>=0.3.1 (from Django)
  Downloading sqlparse-0.4.4-py3-none-any.whl (41 kB)
----- 41.2/41.2 kB 658.4 kB/s eta 0:00:00
Collecting tzdata (from Django)
  Downloading tzdata-2023.3-py2.py3-none-any.whl (341 kB)
----- 341.8/341.8 kB 5.3 MB/s eta 0:00:00
Downloading Django-4.2.7-py3-none-any.whl (8.0 MB)
----- 8.0/8.0 MB 10.9 MB/s eta 0:00:00
Downloading asgiref-3.7.2-py3-none-any.whl (24 kB)
Installing collected packages: tzdata, sqlparse, asgiref, Django
Successfully installed Django-4.2.7 asgiref-3.7.2 sqlparse-0.4.4 tzdata-2023.3

[notice] A new release of pip is available: 23.2.1 -> 23.3.1
[notice] To update, run: python.exe -m pip install --upgrade pip

(.venv) C:\django>
```

Рисунок 1.7 – Встановлення Django

1.2.5 Перевірка установки

Щоб переконатися, що все встановлено правильно, ми можемо перейти до python інтерпретатора. Для цього введемо в терміналі команду

```
python
```

І потім виконаємо послідовно дві інструкції та отримаємо номер версії Django:

```
Administrator: Командний рядок - python
Collecting Django
  Obtaining dependency information for Django from https://files.pythonhosted.org/packages/2d/6d/e87236e3c7b2f5911d132034177aebb605f3953910cc429df8061b13bf10/Django-4.2.7-py3-none-any.whl.metadata
  Downloading Django-4.2.7-py3-none-any.whl.metadata (4.1 kB)
Collecting asgiref<4,>=3.6.0 (from Django)
  Obtaining dependency information for asgiref<4,>=3.6.0 from https://files.pythonhosted.org/packages/9b/80/b9051a4a07ad231558fcd8ffc89232711b4e618c15cb7a392a17384bbeef/asgiref-3.7.2-py3-none-any.whl.metadata
  Downloading asgiref-3.7.2-py3-none-any.whl.metadata (9.2 kB)
Collecting sqlparse>=0.3.1 (from Django)
  Downloading sqlparse-0.4.4-py3-none-any.whl (41 kB)
----- 41.2/41.2 kB 658.4 kB/s eta 0:00:00
Collecting tzdata (from Django)
  Downloading tzdata-2023.3-py2.py3-none-any.whl (341 kB)
----- 341.8/341.8 kB 5.3 MB/s eta 0:00:00
Downloading Django-4.2.7-py3-none-any.whl (8.0 MB)
----- 8.0/8.0 MB 10.9 MB/s eta 0:00:00
Downloading asgiref-3.7.2-py3-none-any.whl (24 kB)
Installing collected packages: tzdata, sqlparse, asgiref, Django
Successfully installed Django-4.2.7 asgiref-3.7.2 sqlparse-0.4.4 tzdata-2023.3

[notice] A new release of pip is available: 23.2.1 -> 23.3.1
[notice] To update, run: python.exe -m pip install --upgrade pip

(.venv) C:\django>python
Python 3.12.0 (tags/v3.12.0:0fb18b0, Oct  2 2023, 13:03:39) [MSC v.1935 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license" for more information.
>>> import django
>>> print(django.get_version())
4.2.7
>>>
```

Рисунок 1.8 – Результат перевірки встановлення Django

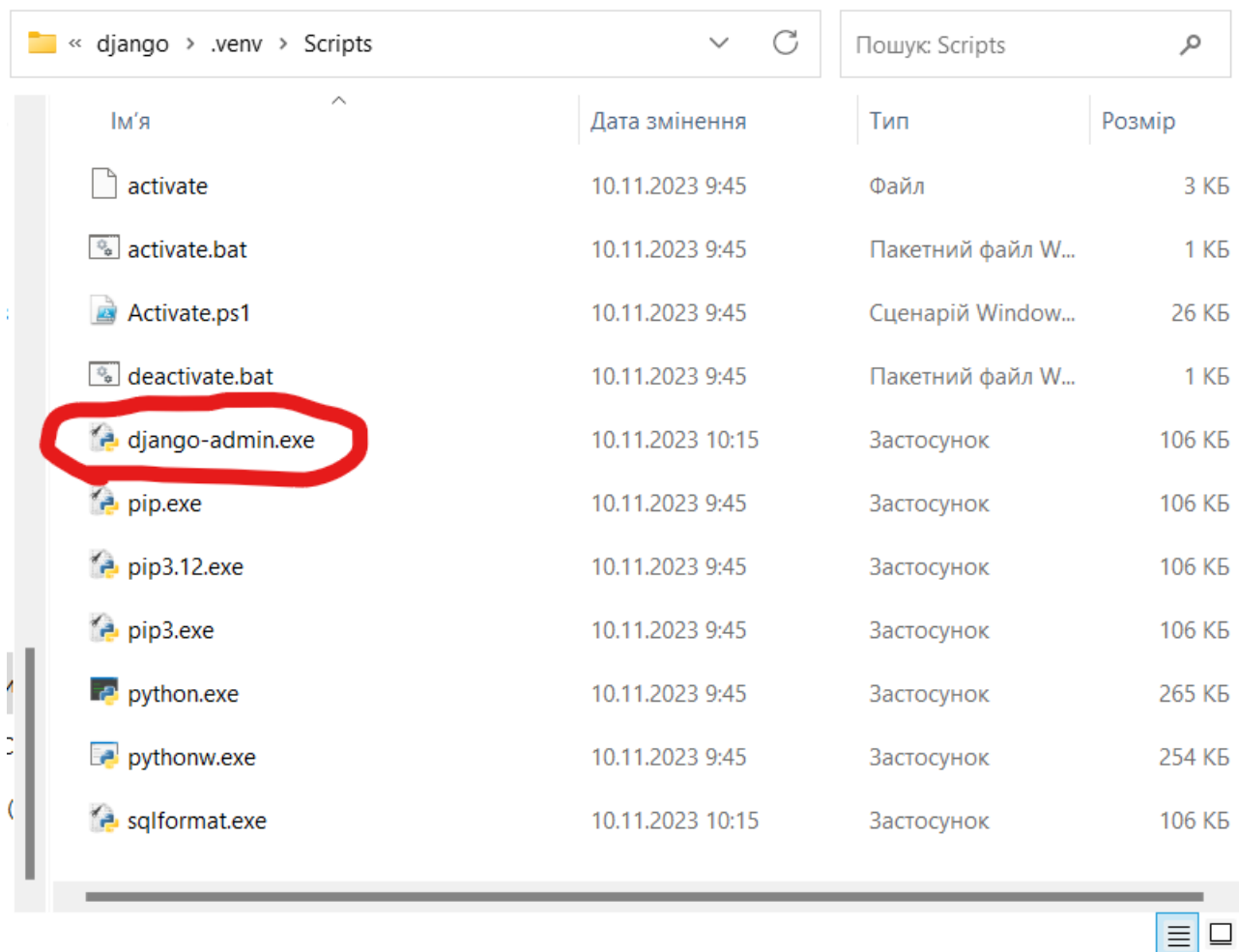
1.2.6 Деактивація віртуального середовища

Після закінчення роботи з віртуальним середовищем ми можемо його деактивувати за допомогою команди:

```
deactivate
```

1.3 Створення першого проекту

Під час встановлення *Django* в папці віртуального середовища встановлюється утиліта *django-admin*. А на Windows також виконується файл *django-admin.exe*. Їх можна знайти в папці віртуального середовища, в яке проводилася установка *Django*: на Windows - у підкаталозі *Scripts*, а на *Linux/MacOS* - у каталозі *bin*.



« django > .venv > Scripts

Пошук: Scripts

Ім'я	Дата змінення	Тип	Розмір
activate	10.11.2023 9:45	Файл	3 КБ
activate.bat	10.11.2023 9:45	Пакетний файл W...	1 КБ
Activate.ps1	10.11.2023 9:45	Сценарій Window...	26 КБ
deactivate.bat	10.11.2023 9:45	Пакетний файл W...	1 КБ
django-admin.exe	10.11.2023 10:15	Застосунок	106 КБ
pip.exe	10.11.2023 9:45	Застосунок	106 КБ
pip3.12.exe	10.11.2023 9:45	Застосунок	106 КБ
pip3.exe	10.11.2023 9:45	Застосунок	106 КБ
python.exe	10.11.2023 9:45	Застосунок	265 КБ
pythonw.exe	10.11.2023 9:45	Застосунок	254 КБ
sqlformat.exe	10.11.2023 10:15	Застосунок	106 КБ

Рисунок 1.9 – Зміст теки *Scripts*

django-admin надає низку команд для керування проектом *Django*. Зокрема, для створення проекту застосовується команда *StartProject*. Цій команді як аргумент передається назва проекту.

Отже, створимо перший *Django*. Нехай він розташовуватиметься в тій же папці, де розташовується каталог віртуального середовища. І для цього спочатку активуємо раніше створене віртуальне середовище

```
cd C:\django
.venv\Scripts\activate.bat
```

(наприклад, середовище *.venv*, яке було створено в минулій темі, якщо воно раніше не було активовано).

І після активації віртуального середовища виконаємо наступну команду

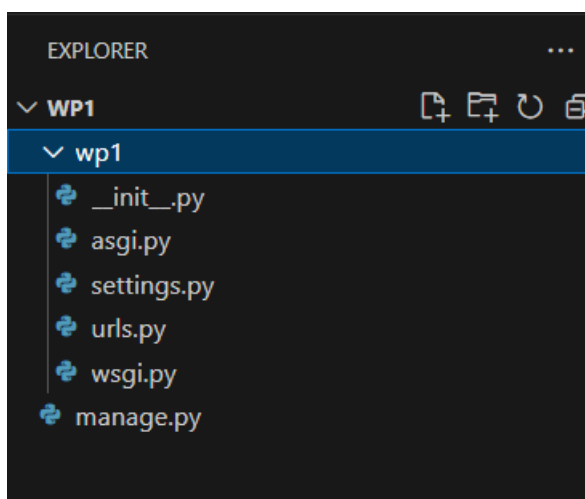
```
c:\django>django-admin startproject wp1
```

В даному випадку ми створюємо проект з ім'ям "wp1". І після виконання цієї команди у поточній папці (c:\django) буде створено каталог wp1.

```
(.venv) C:\django>django-admin startproject wp1
(.venv) C:\django>
```

Рисунок 1.10 – Створення нового проекту

Створений каталог складатиметься з наступних елементів:



manage.py: виконує різні команди проекту, наприклад, створює та запускає програму

wp1 - власне папка проекту *wp1*, яка містить такі файли:

__init__.py: цей файл вказує на те, що папка, в якій він знаходиться, буде розглядатися як модуль. Це стандартний файл для програми *Python*.

settings.py: містить налаштування конфігурації проекту

urls.py: містить шаблони *URL*-адрес, по суті визначає систему маршрутизації проекту

wsgi.py: містить властивості конфігурації *Web Server Gateway Interface (WSGI)*. Він використовується для розгортання проекту.

asgi.py: назва файлу представляє скорочення від *Asynchronous Server Gateway Interface* та розширює можливості *WSGI*, додаючи підтримку для взаємодії між асинхронними веб-серверами та додатками.

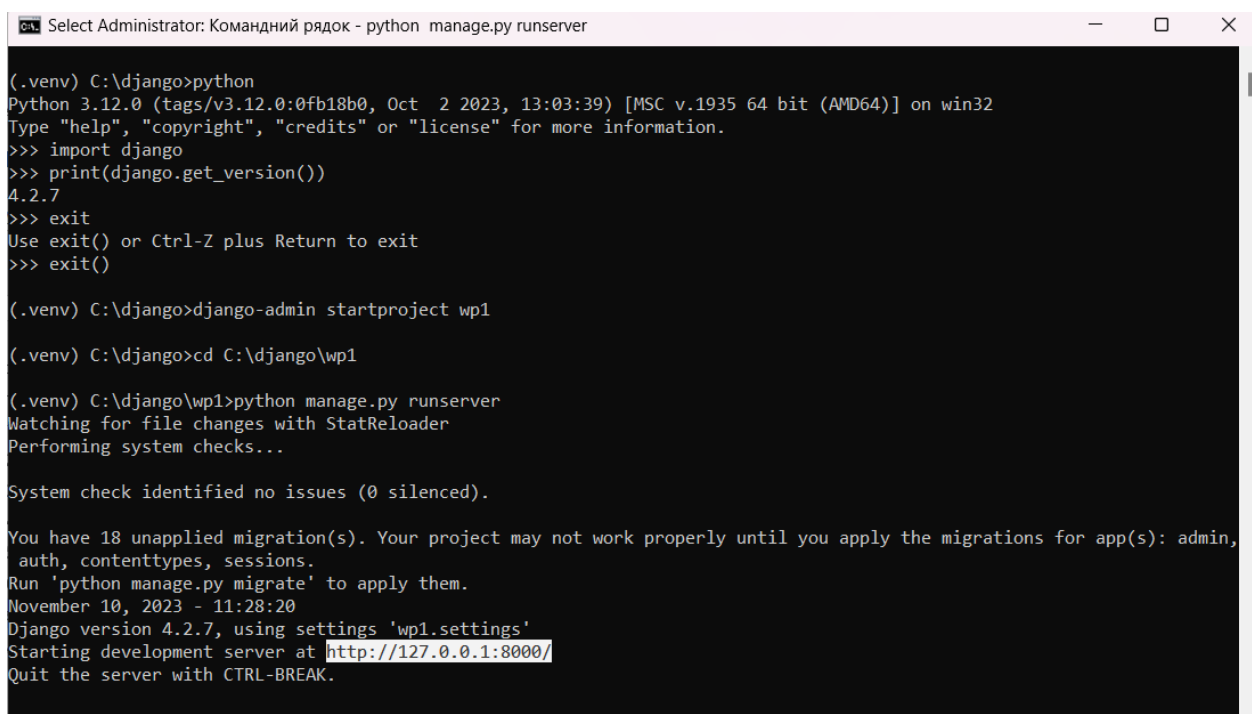
Запустимо проект виконання. Для цього за допомогою `cd` команди перейдемо в консолі до папки проекту.

```
cd C:\django\wp1
```

І потім для запуску проекту виконаємо таку команду:

```
python manage.py runserver
```

Після запуску проекту в консолі ми побачимо адресу, за якою запущено проект.



```
Select Administrator: Командний рядок - python manage.py runserver

(.env) C:\django>python
Python 3.12.0 (tags/v3.12.0:0fb18b0, Oct 2 2023, 13:03:39) [MSC v.1935 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license" for more information.
>>> import django
>>> print(django.get_version())
4.2.7
>>> exit
Use exit() or Ctrl-Z plus Return to exit
>>> exit()

(.env) C:\django>django-admin startproject wp1

(.env) C:\django>cd C:\django\wp1

(.env) C:\django\wp1>python manage.py runserver
Watching for file changes with StatReloader
Performing system checks...

System check identified no issues (0 silenced).

You have 18 unapplied migration(s). Your project may not work properly until you apply the migrations for app(s): admin,
auth, contenttypes, sessions.
Run 'python manage.py migrate' to apply them.
November 10, 2023 - 11:28:20
Django version 4.2.7, using settings 'wp1.settings'
Starting development server at http://127.0.0.1:8000/
Quit the server with CTRL-BREAK.
```

Рисунок 1.11 – Результат запуску Django

Як правило, це адреса `http://127.0.0.1:8000/`. Відкриємо будь-який веб-браузер та введемо цю адресу в адресний рядок браузера. І нам відкриється вміст за замовчанням.

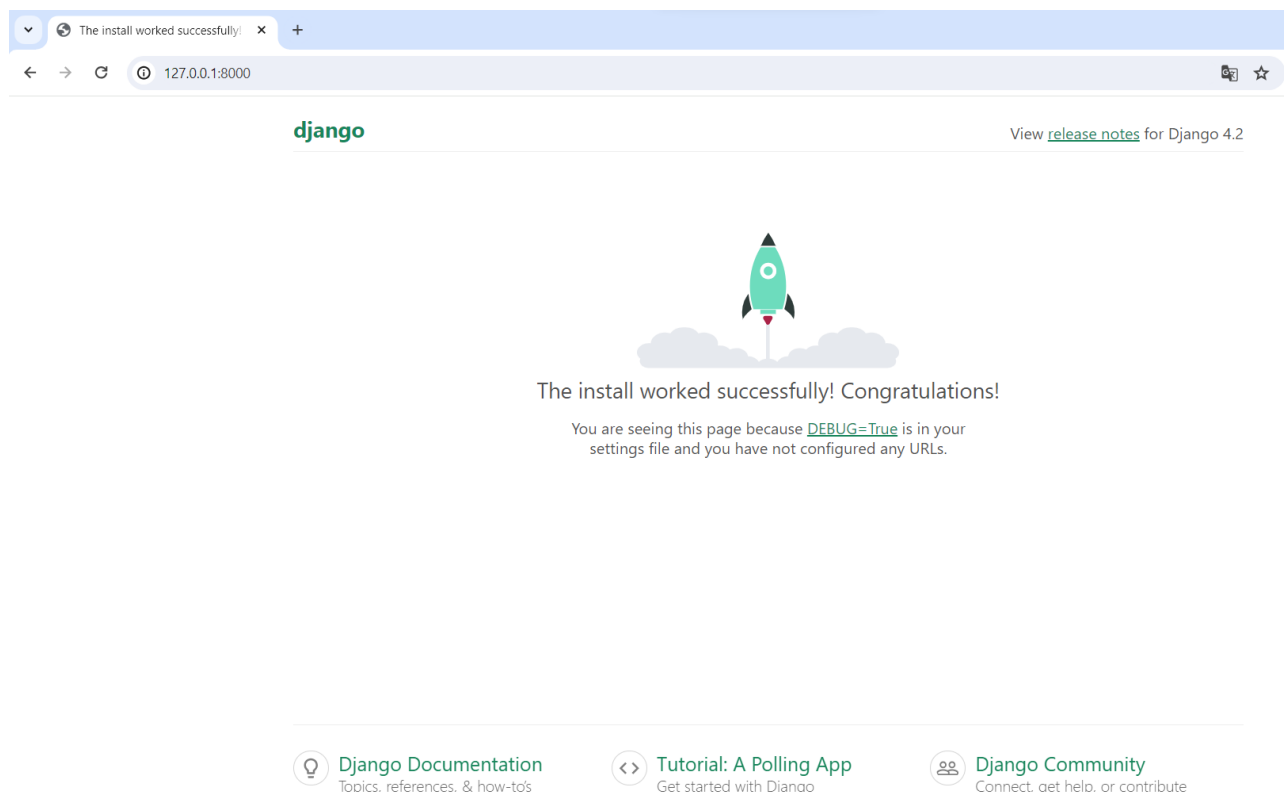


Рисунок 1.12 – Результат запуску серверу

1.4 Створення першого додатку

Веб-додаток або проект Django складається з окремих програм. Разом вони утворюють повноцінний веб-додаток. Кожна програма представляє якусь певну функціональність або групу функціональностей. Один проект може включати багато програм. Це дозволяє виділити групу завдань в окремий модуль та розробляти їх відносно незалежно від інших. Крім того, ми можемо переносити програму з одного проекту до іншого незалежно від іншої функціональності проекту.

При створенні проекту він містить кілька програм за замовчуванням:

django.contrib.admin

django.contrib.auth

django.contrib.contenttypes

django.contrib.sessions

django.contrib.messages

django.contrib.staticfiles

Список усіх програм можна знайти в проєкті у файлі *settings.py* у змінній *INSTALLED_APPS*:

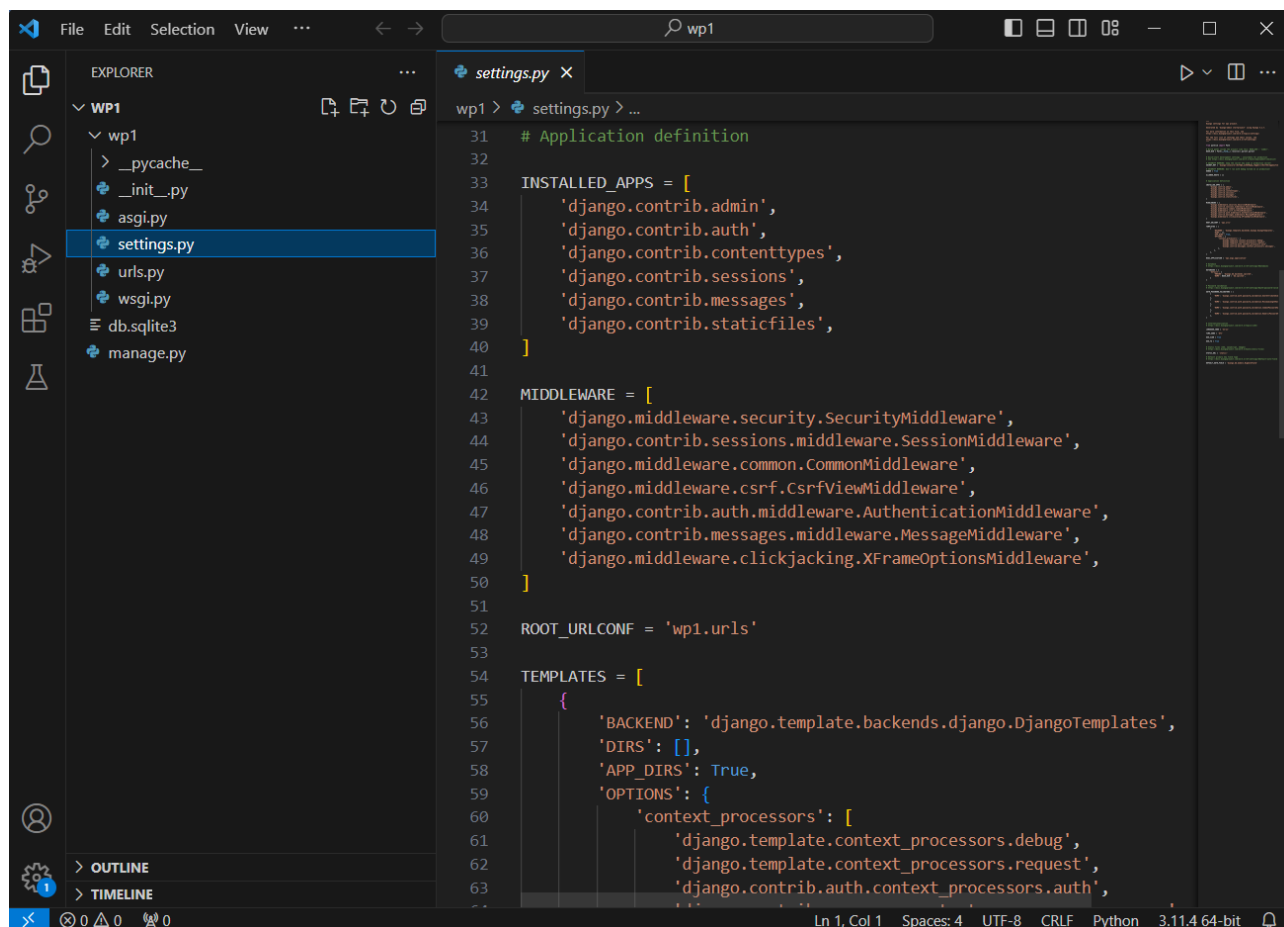


Рисунок 1.13 – Зміст файлу *settings.py*

Ми можемо створювати свої додатки, які реалізують певний функціонал. Для створення додатка у проєкті застосовується команда

```
python manage.py startapp app_name
```

Так, створимо в проєкті, створеному в минулій темі або новому, перший додаток, який називатиметься *hello*. Для цього виконаємо в командному рядку/терміналі наступну команду:

```
python manage.py startapp hello
```

```

Administrator: Командний рядок

c:\django\wp1>cd c:\django
c:\django>.venv\scripts\activate
(.venv) c:\django>cd c:\django\wp1
(.venv) c:\django\wp1>python manage.py startapp hello
(.venv) c:\django\wp1>

```

Рисунок 1.14 – Створення нового додатка

В результаті в проєкті *Django* буде створено програму *hello*. І після виконання цієї команди ми побачимо в проєкті нову папку, яка зберігатиме всі файли створеної програми:

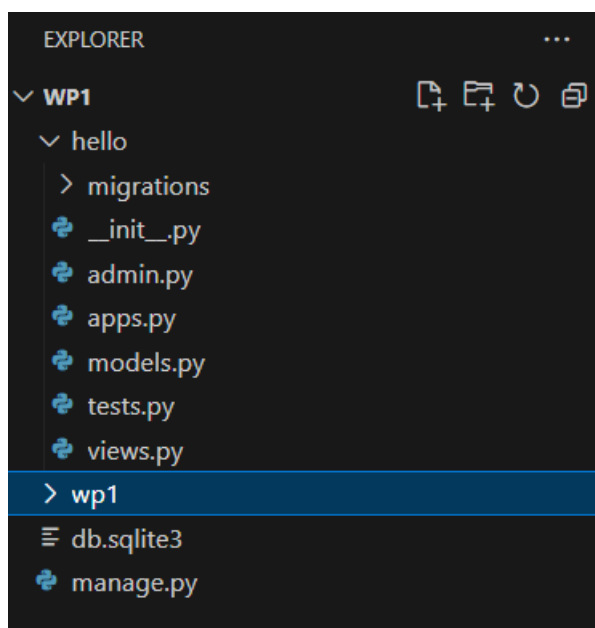


Рисунок 1.15 – Оновлення структура проєкта

Розглянемо коротко його структуру:

папка *migrations*: призначена для зберігання міграцій - скриптів, які дозволяють синхронізувати структуру бази даних із визначенням моделей

__init__.py: вказує інтерпретатору python, що поточний каталог буде розглядатися як пакет

admin.py: призначений для адміністративних функцій, зокрема тут наводиться реєстрація моделей, які використовуються в інтерфейсі адміністратора

apps.py: визначає конфігурацію програми

models.py: зберігає визначення моделей, які описують дані, що використовуються в додатку

tests.py: зберігає тести програми

views.py: визначає функції, які отримують запити користувачів, обробляють їх та повертають відповідь

Але поки що додаток ніяк не задіюється. Його треба зареєструвати у проєкті Django. Для цього відкриємо файл *settings.py* і додамо до кінця масиву *INSTALLED_APPS* нашу програму:

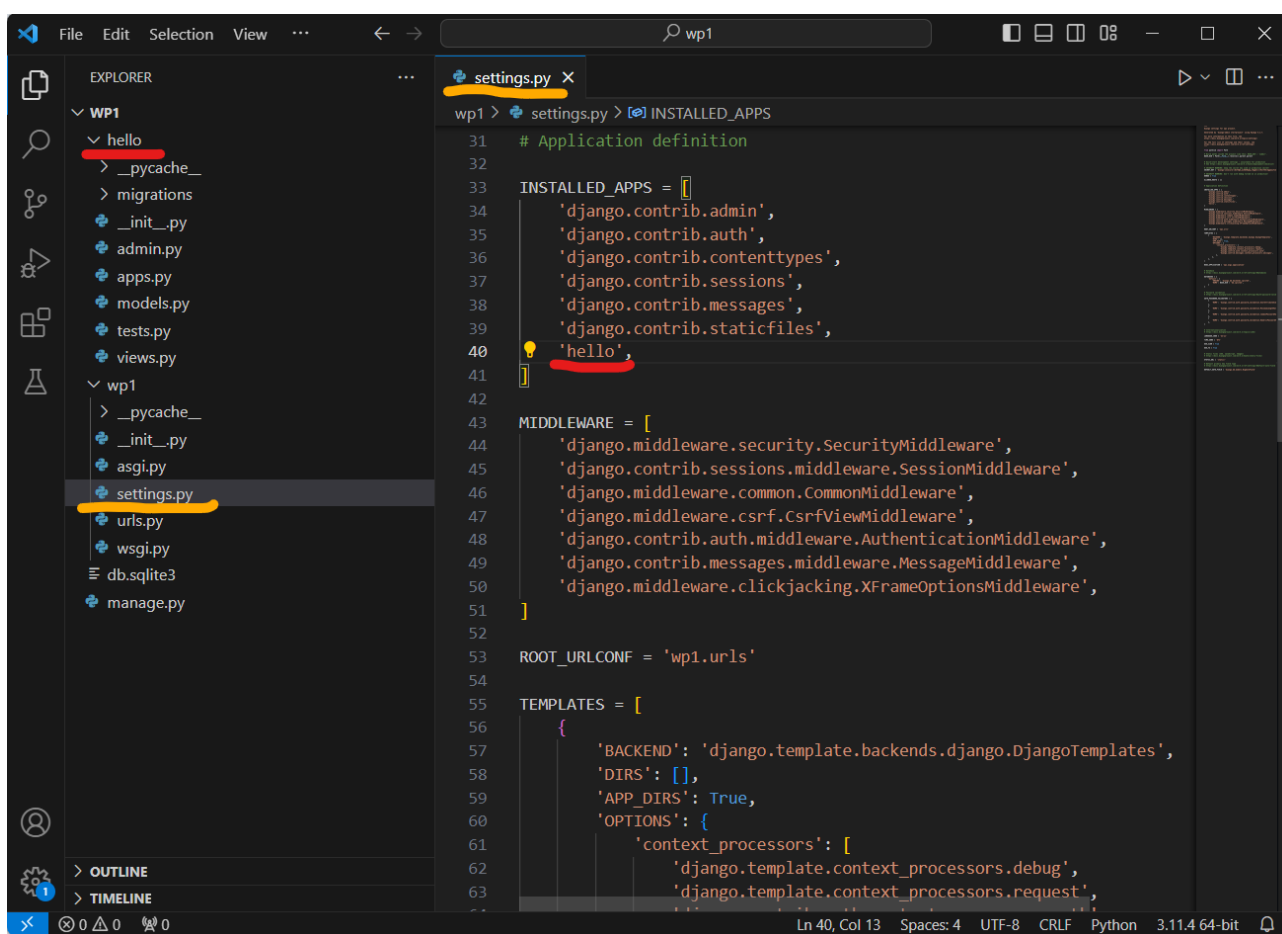
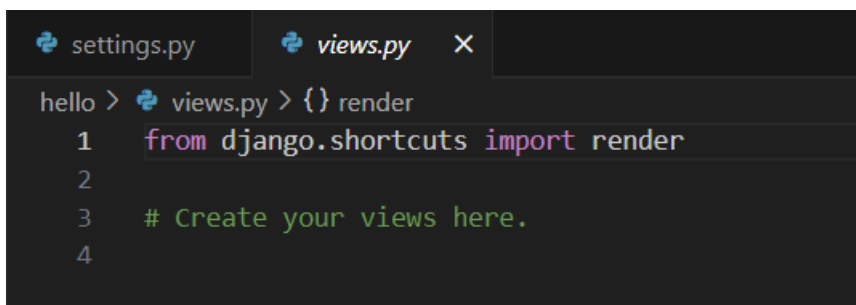


Рисунок 1.15 – Додавання додатка hello до проєкту

У проєкті може бути кілька додатків, і кожен з них треба додавати таким чином.

Тепер визначимо якісь найпростіші дії, які виконуватиме цей додаток, наприклад, надсилати у відповідь користувачеві рядок *"Hello World"*. Для цього перейдемо у проєкті програми *hello* до файлу *views.py*, який за умовчанням має виглядати так:



```

settings.py  views.py  X
hello > views.py > {} render
1  from django.shortcuts import render
2
3  # Create your views here.
4

```

Рисунок 1.16 – Зміст файлу *views.py* за замовченням

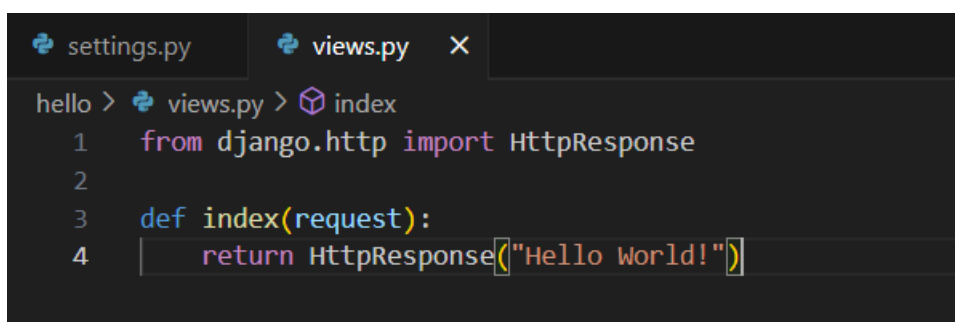
Змінимо код на наступний:

```
from django.http import HttpResponse
```

```
def index(request):
```

```
    return HttpResponse("Hello World!")
```

В результаті отримаємо:



```

settings.py  views.py  X
hello > views.py > index
1  from django.http import HttpResponse
2
3  def index(request):
4      return HttpResponse("Hello World!")

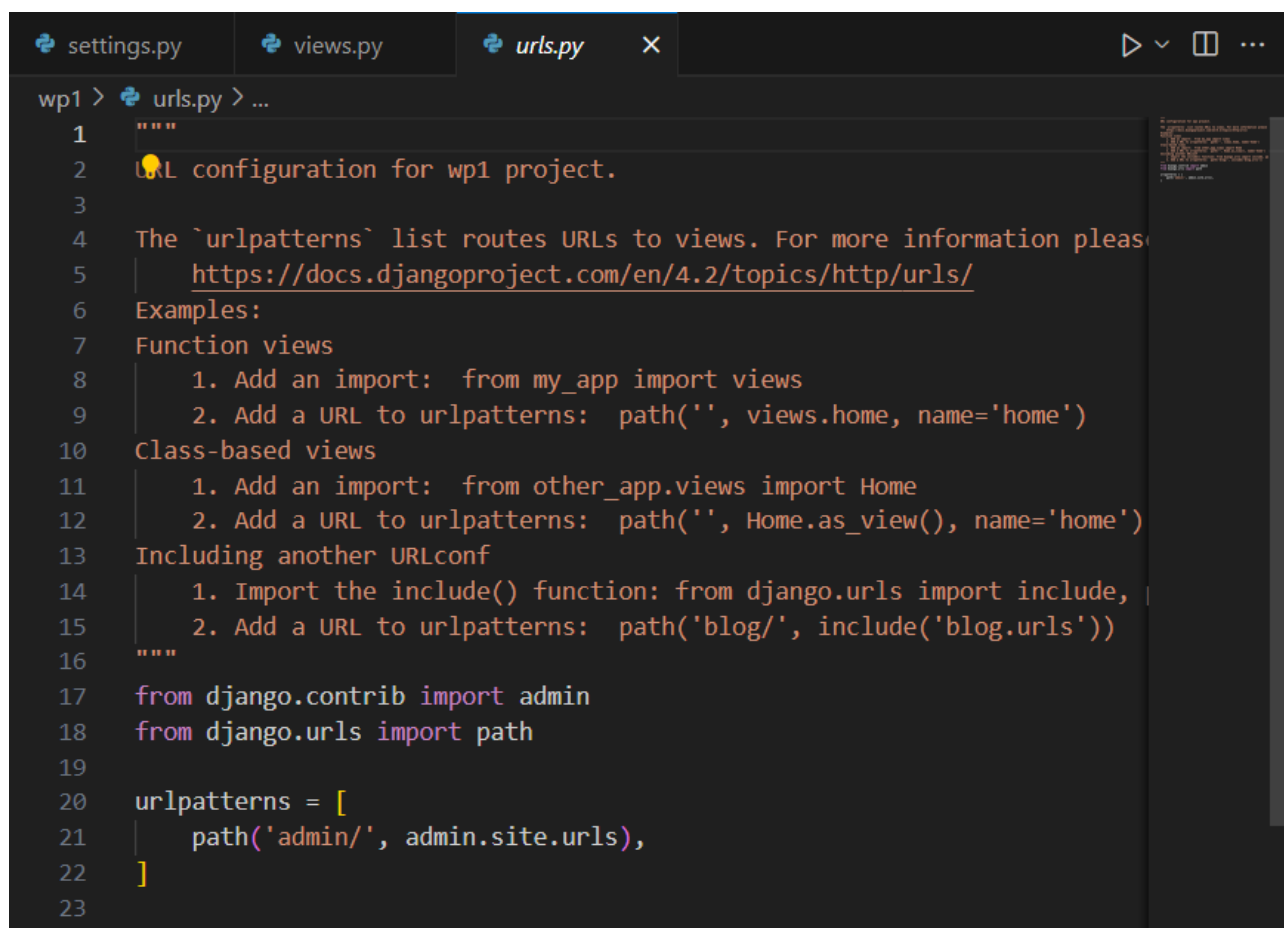
```

Рисунок 1.16 – Оновлений зміст файлу *views.py*

У цьому випадку ми імпортуємо клас *HttpResponse* із стандартного пакету *django.http*. Потім визначається функція *index()*, яка як параметр отримує об'єкт запиту *request*. Клас *HttpResponse* призначений для створення відповіді, яка

надсилається користувачеві. І за допомогою виразу `return HttpResponse("Hello World!")` ми відправляємо користувачеві рядок `"Hello World!"`

Тепер в основному проєкті *Django* відкриємо файл `urls.py`, який дозволяє зіставити маршрути з уявленнями, які будуть обробляти запит по цих маршрутах. За замовчуванням цей файл виглядає так:



```

wp1 > settings.py  views.py  urls.py  X
1  """
2  URL configuration for wp1 project.
3
4  The `urlpatterns` list routes URLs to views. For more information please see
5  https://docs.djangoproject.com/en/4.2/topics/http/urls/
6  Examples:
7  Function views
8      1. Add an import: from my_app import views
9      2. Add a URL to urlpatterns: path('', views.home, name='home')
10 Class-based views
11     1. Add an import: from other_app.views import Home
12     2. Add a URL to urlpatterns: path('', Home.as_view(), name='home')
13 Including another URLconf
14     1. Import the include() function: from django.urls import include,
15     2. Add a URL to urlpatterns: path('blog/', include('blog.urls'))
16 """
17 from django.contrib import admin
18 from django.urls import path
19
20 urlpatterns = [
21     path('admin/', admin.site.urls),
22 ]
23

```

Рисунок 1.17 – Зміст файлу `urls.py` за замовченням

Першим рядком із модуля `django.contrib` імпортується клас `AdminSite`, який надає можливості роботи з інтерфейсом адміністратора. Другим рядком із модуля `django.urls` імпортується функція `path`. Ця функція визначає порівняння певного маршруту з функцією обробки. Так, у разі маршрут `"admin/"` буде оброблятися методом `admin.site.urls`.

Але вище ми визначили функцію *index* у файлі *views.py*, який повертає користувачеві рядок “*Hello World!*”.

Тому змінимо зміст на такий:

```
from django.urls import path
from hello import views

urlpatterns = [
    path("", views.index, name='home'),
]
```

Файл *urls.py* має виглядати наступним чином:

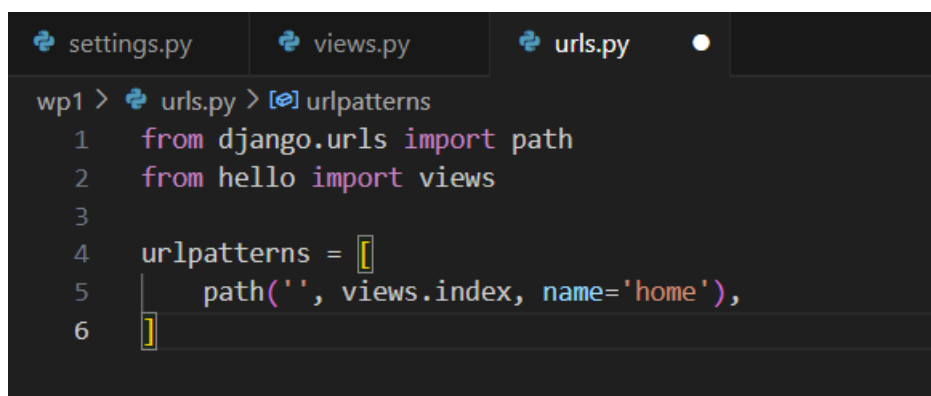
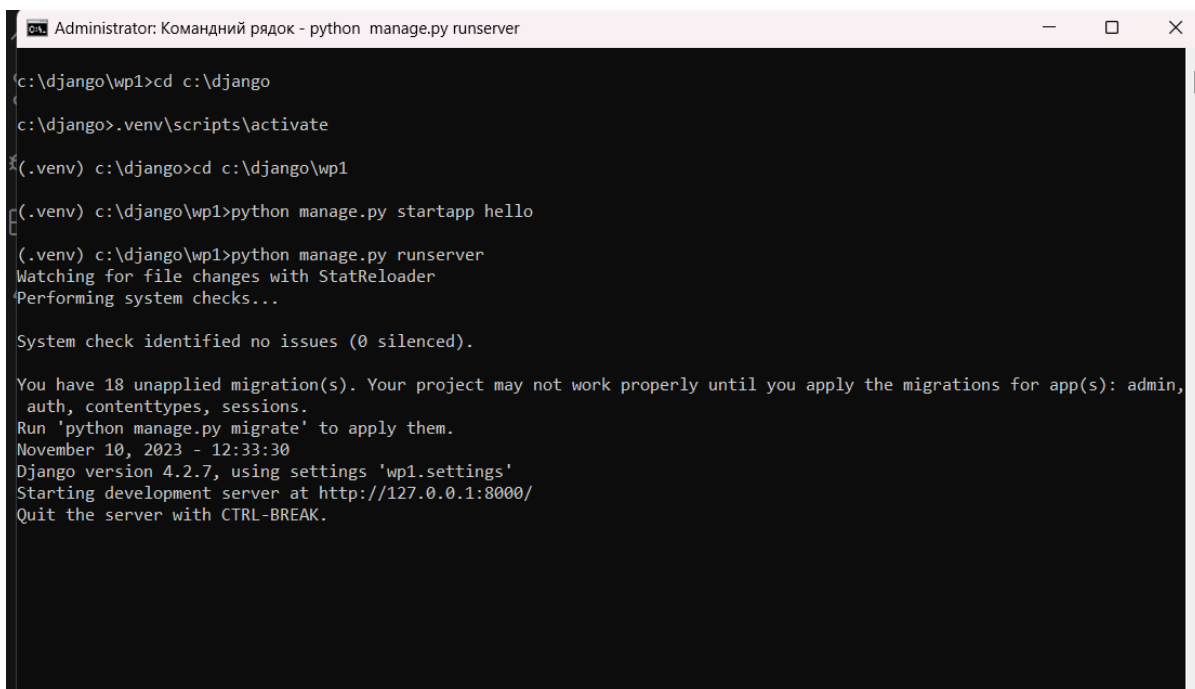


Рисунок 1.18 – Оновлений зміст файлу *urls.py*

Таким чином, щоб використовувати функцію *views.index*, спочатку імпортуємо модуль *views*. Потім визначаємо зіставлення маршруту "" і функції *views.index* а також додатково ім'я для маршруту: *name='home'*. По суті, маршрут "" буде зіставлятися із запитом до кореня додатка.

Тепер знову запустимо програму командою *python manage.py runserver*



```

Administrator: Командний рядок - python manage.py runserver

c:\django\wp1>cd c:\django
c:\django>.venv\scripts\activate
(.venv) c:\django>cd c:\django\wp1
(.venv) c:\django\wp1>python manage.py startapp hello
(.venv) c:\django\wp1>python manage.py runserver
Watching for file changes with StatReloader
Performing system checks...

System check identified no issues (0 silenced).

You have 18 unapplied migration(s). Your project may not work properly until you apply the migrations for app(s): admin,
auth, contenttypes, sessions.
Run 'python manage.py migrate' to apply them.
November 10, 2023 - 12:33:30
Django version 4.2.7, using settings 'wp1.settings'
Starting development server at http://127.0.0.1:8000/
Quit the server with CTRL-BREAK.

```

Рисунок 1.19 – Перезапуск серверу

І знову перейдемо в браузері за адресою *http://127.0.0.1:8000/*, і браузер відобразить рядок *"Hello World!"*:

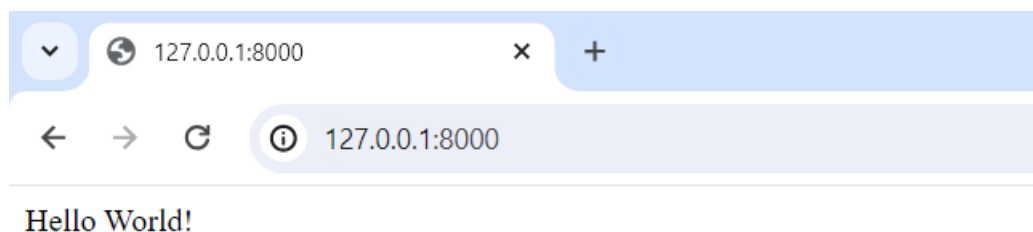


Рисунок 1.20 – Результат роботи оновленого серверу

2 ПРЕДСТАВЛЕННЯ ТА МАРШРУТИЗАЦІЯ

2.1 Обробка запиту

Центральним моментом будь-якої веб-програми є обробка запиту, який надсилає користувач. Django за обробку запиту відповідають подання або *views*. По суті представлення представляють функції обробки, які приймають дані запиту у вигляді об'єкта *HttpRequest* з пакету *django.http* і генерують деякий результат, який потім надсилається користувачеві.

За промовчанням подання розміщуються у програмі у файлі *views.py*.

Наприклад, візьмемо стандартний проект, до якого додано програму (наприклад, проект створений у попередній роботі). Запустити додаток можна виконавши наступні команди у командному рядку:

```
cd C:\django
.venv\Scripts\activate.bat
cd C:\django\wp2
python manage.py startapp hello
python manage.py runserver
```

Нагадаємо, що при створенні нового проекту файл *views.py* має такий вміст:

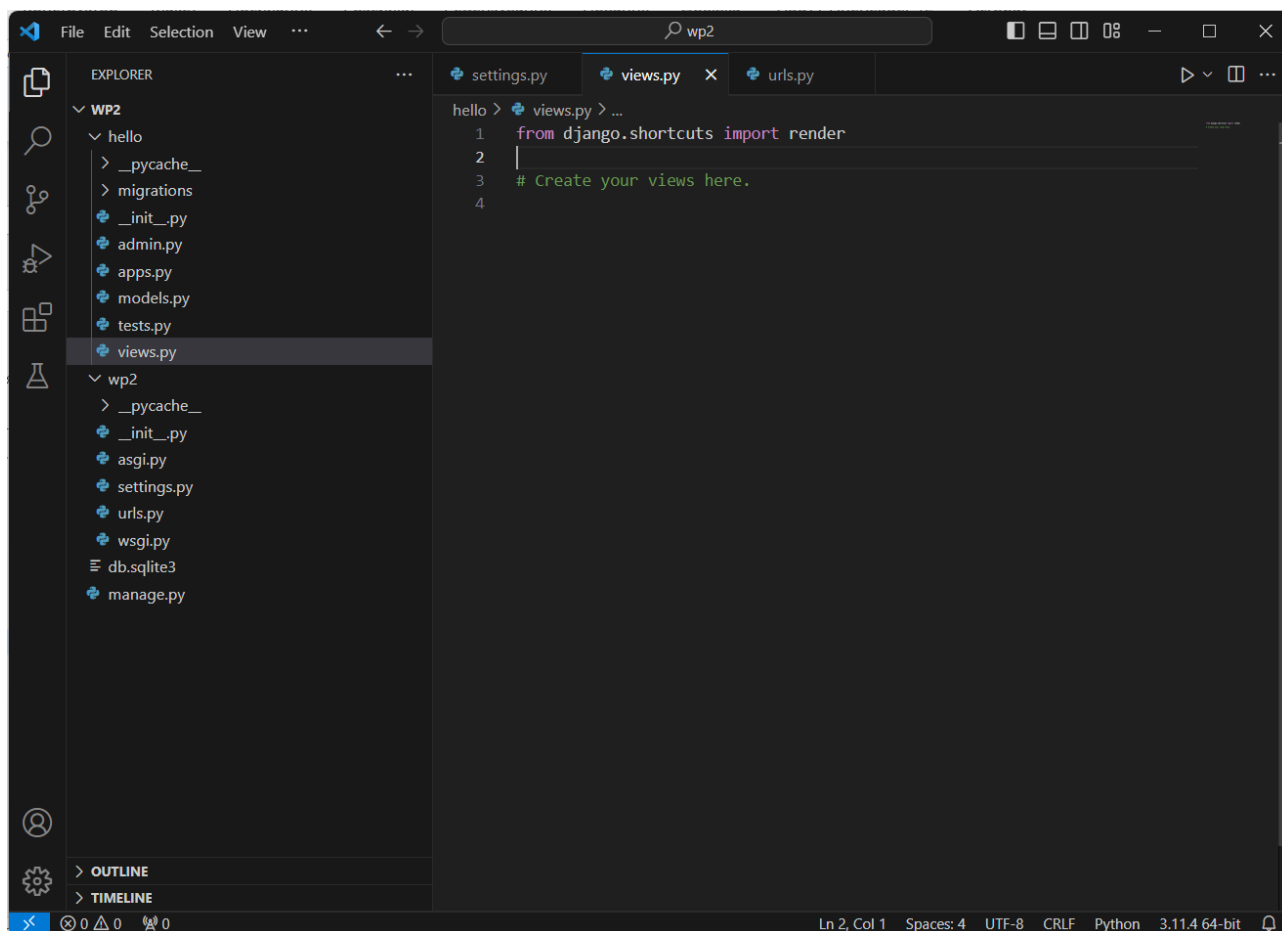


Рисунок 2.1 – Зміст файлу за замовченням

Цей код поки що ніяк не обробляє запити, він тільки імпортує функцію *render()*, яка може використовуватися для обробки.

Генерувати результат можна у різний спосіб. Один із них представляє використання класу *HttpResponse* з пакету *django.http*, який дозволяє надіслати текстовий вміст.

Так, змінимо файл *views.py* наступним чином:

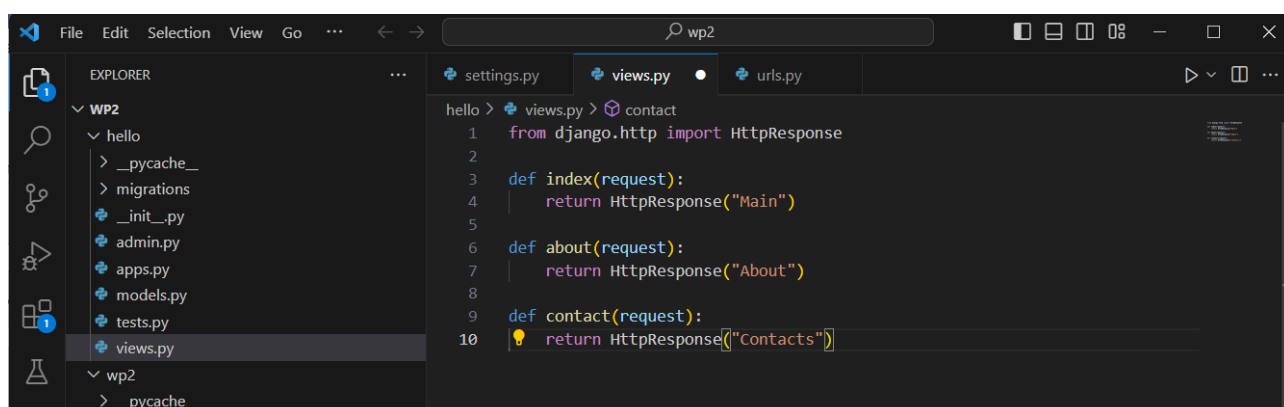


Рисунок 2.2 – Визначення функцій

У цьому випадку визначено три функції, які будуть опрацьовувати запити. Кожна функція приймає як параметр *request* об'єкт *HttpRequest*, який зберігає інформацію про запит. Однак у цьому випадку вона нам не потрібна, тому параметр ніяк не використовується. Для генерації відповіді конструктор об'єкта *HttpResponse* передається деякий рядок. Це може бути навіть код html у вигляді рядка.

Щоб ці функції зіставлялися із запитами, треба визначити для них маршрути у проекті у файлі *urls.py*. Зокрема, змінимо цей файл так:

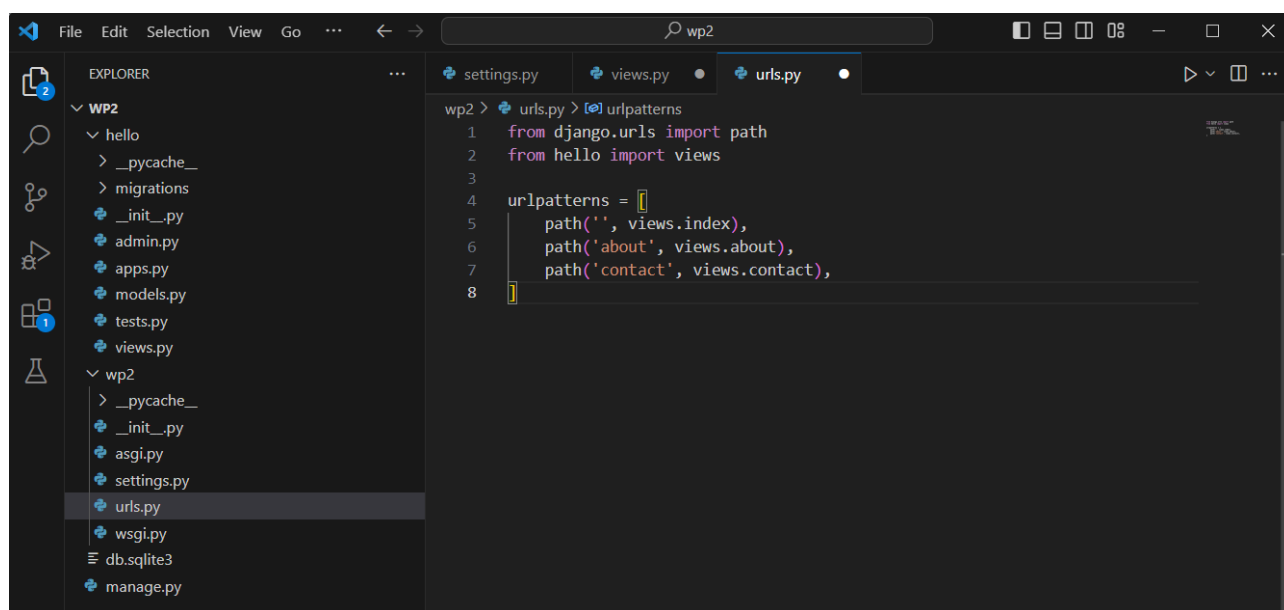


Рисунок 2.3 – Зв'язок функцій з запитами

Змінна *urlpatterns* визначає набір зіставлень функцій обробки з певними рядками запиту. Наприклад, запит до кореня веб-сайту оброблятиметься функцією *index*, запит на адресу "about" оброблятиметься функцією *about*, а запит "contact" - функцією *contact*.

Запустимо проект і звернемося на деякі з цих адрес.

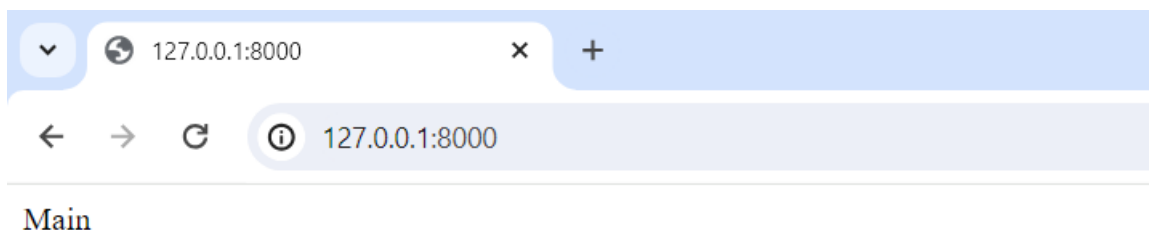


Рисунок 2.4 – Зміст сторінки за адресою <http://127.0.0.1:8000/>

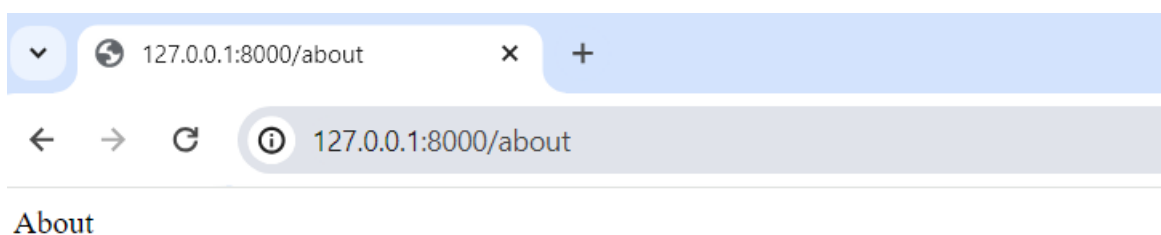


Рисунок 2.5 – Зміст сторінки за адресою <http://127.0.0.1:8000/about>

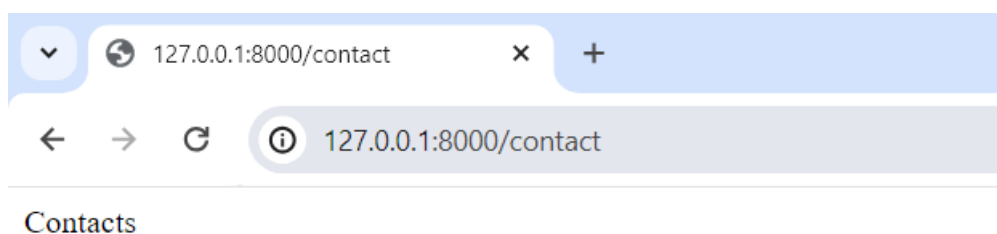


Рисунок 2.7 – Зміст сторінки за адресою <http://127.0.0.1:8000/contact>

При цьому ми можемо відправляти не простий текст, а, наприклад, код HTML, який потім інтерпретується браузером. Так, змінимо файл *views.py* наступним чином:

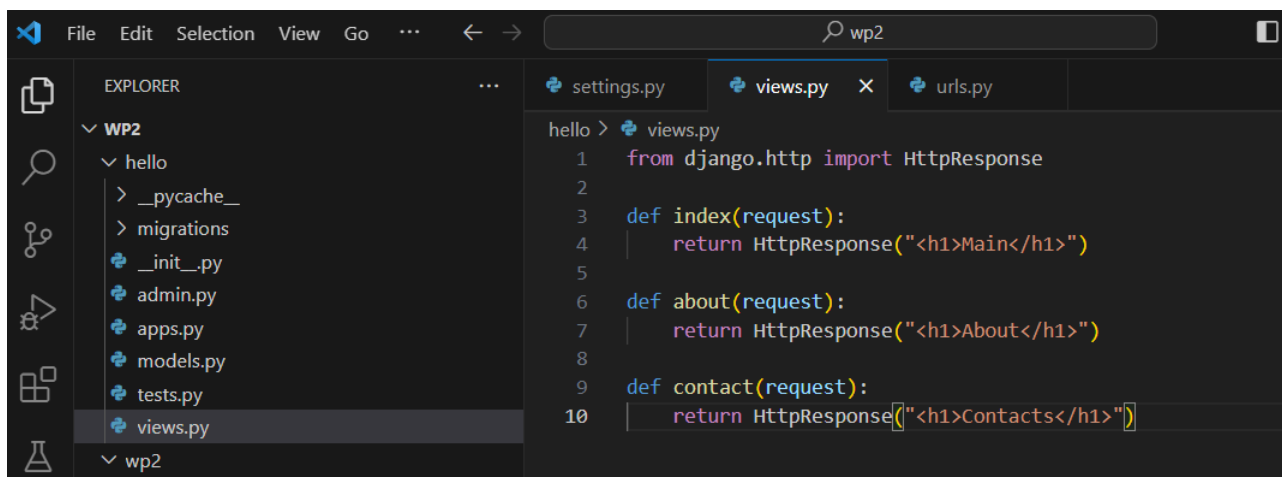


Рисунок 2.8 – Оновлене визначення функцій

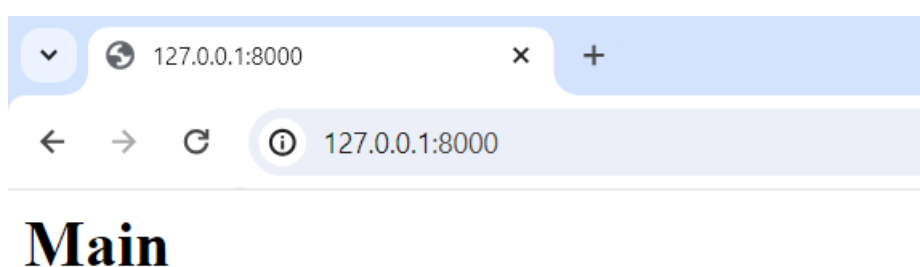


Рисунок 2.9 – Зміст сторінки за адресою <http://127.0.0.1:8000/>

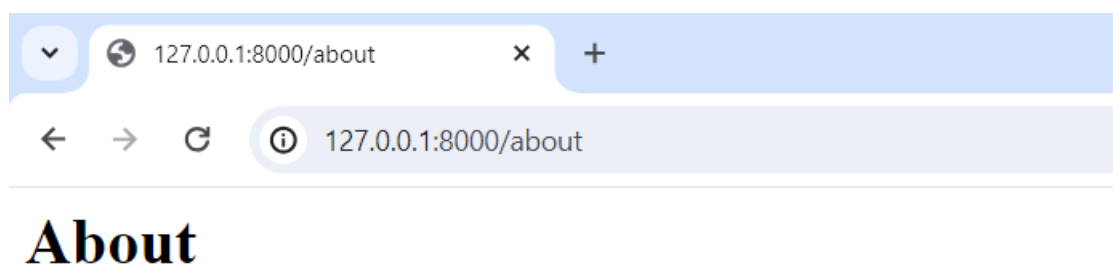
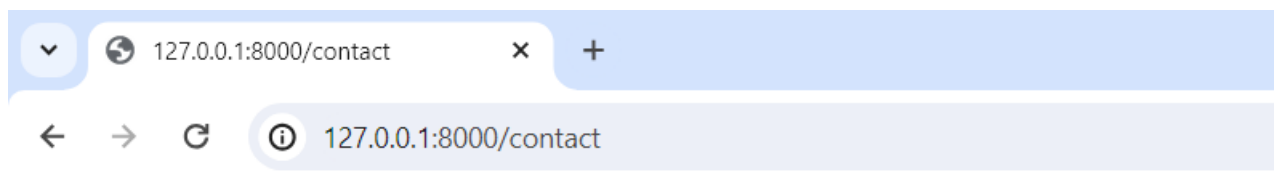


Рисунок 2.10 – Зміст сторінки за адресою <http://127.0.0.1:8000/about>



Contacts

Рисунок 2.11 – Зміст сторінки за адресою <http://127.0.0.1:8000/contact>

2.2 Визначення маршрутів та функції `path` та `re_path`

У 2.1 розглядалося зіставлення адрес URL та функцій, які обробляють запити на ці адреси. Наприклад, ми маємо відповідні функції у файлі `views.py`, код якого представлено на рис.2.8. Це так звані функції-подання або *view function*. І у файлі `urls.py` проекту вони зіставляються (рис. 2.3) з URL-адресами за допомогою функції `path()`.

За зіставлення шляхів та функцій-подання відповідає функція `path()`, яка розташовується в пакеті `django.urls` і яка приймає чотири параметри:

```
path(route, view, kwargs=None, name=None)
```

- `route`: представляє шаблон адреси URL, якій має відповідати запит
- `view`: функція-подання, яке обробляє запит
- `kwargs`: додаткові аргументи, які передаються у функцію-подання
- `name`: назва маршруту

У прикладі коду на рис.2.3 лише перші два параметри, які є обов'язковими: запитана URL-адреса та функція, яка обробляє запит за цією адресою. Додатково через третій параметр можна вказати ім'я маршруту:

```
path('', views.index, name='home'),
```

В даному випадку маршрут називатиметься *"home"*.

2.2.1 re_path()

Хоча ми можемо успішно застосовувати функцію `path()` для визначення маршрутів, вона досить обмежена за своєю дією. Запрошений шлях повинен точно відповідати вказаній в маршруті адресі URL. Так, у прикладі вище, щоб функція `views.about` могла обробляти запит, адреса повинна бути в точності `"about"`. Наприклад, варто нам вказати слеш наприкінці: `"about/"` і `django` вже не зможе зіставити шлях із запитом.

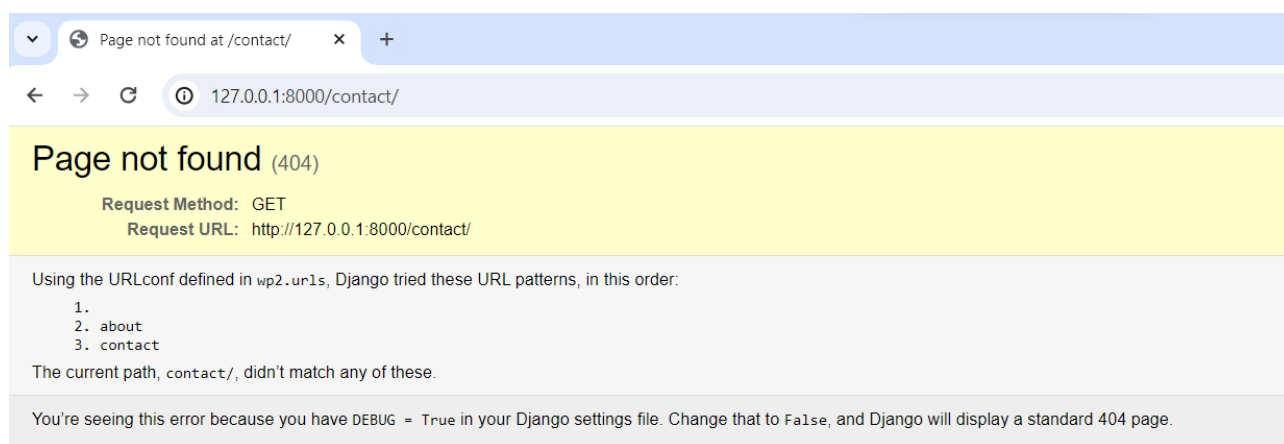


Рисунок 2.12 – Помилка в обробці запиту

В якості альтернативи для визначення маршрутів ми можемо використовувати функцію `re_path()`, яка також розміщується в пакеті `django.urls` і має той самий набір параметрів:

```
re_path(route, view, kwargs=None, name=None)
```

Її перевага полягає в тому, що вона дозволяє задати адреси URL за допомогою регулярних виразів. Наприклад, змінимо визначення файлу `urls.py` наступним чином:

Адреса в першому маршруті, як і раніше, утворюється за допомогою функції `path()` і вказує на корінь веб-програми.

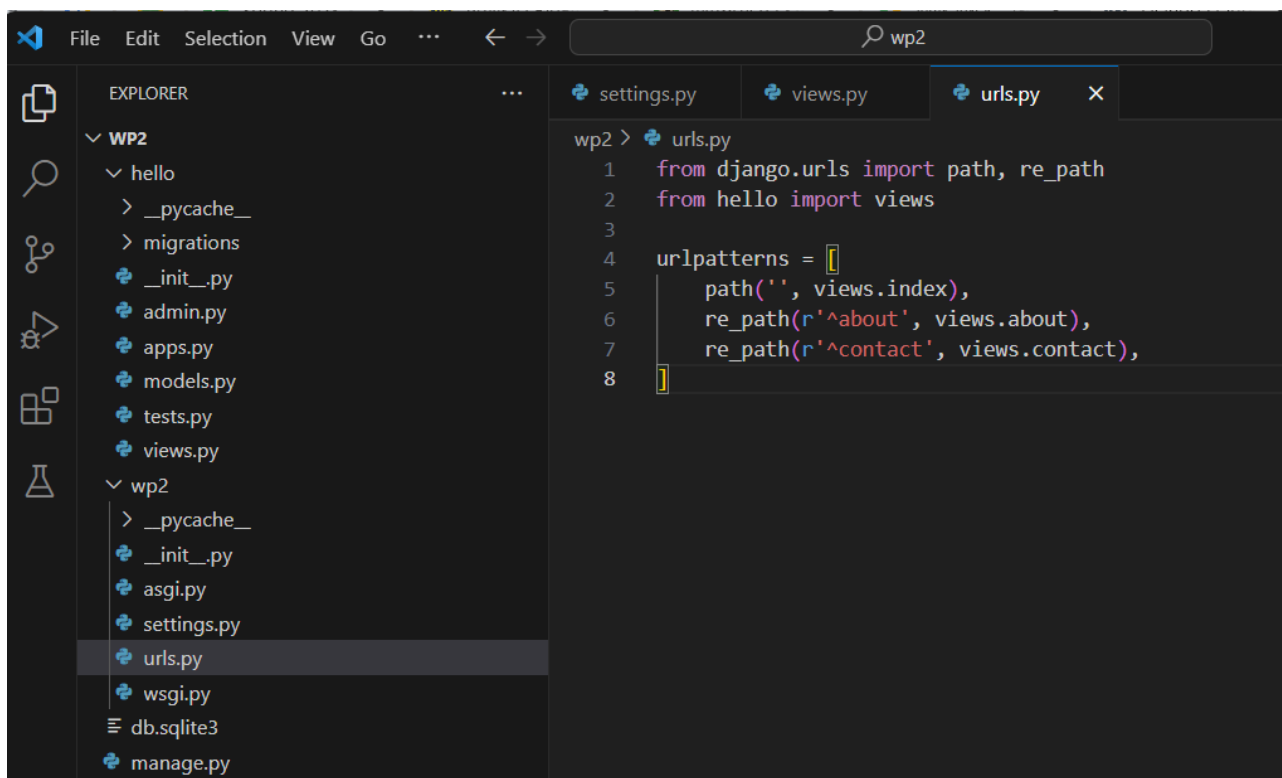


Рисунок 2.13 – Застосування *path()* та *re_path()*

Інші два маршрути утворюються за допомогою функції *re_path()*. Причому оскільки визначається регулярний вираз, то перед рядком з шаблоном адреси URL ставиться буква *r*. У шаблоні адреси можна використовувати різні елементи синтаксису регулярних виразів. Зокрема, вираз *^about* вказує, що адреса має починатися з *"about"*. Однак він необов'язково точно повинен відповідати рядку *"about"*, як це було у випадку з функцією *path()*.

Наприклад, ми можемо звернутися за будь-якою адресою, головне, щоб вона починалася з *"about"*, і тоді подібний запит буде оброблятися функцією *views.about*.

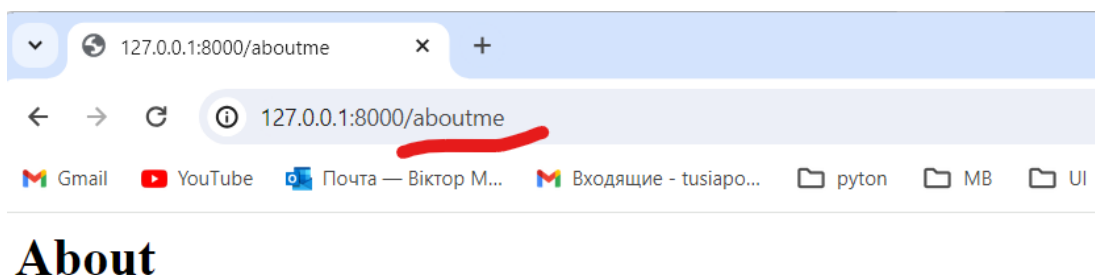


Рисунок 2.14 – Результат роботи методу *re_path()*

2.2.2 Черговість маршрутів

Коли запит приходить до додатка, то система перевіряє відповідність запиту маршрутам у міру їх визначення: спочатку порівнюється перший маршрут, якщо він не підходить, то порівнюється другий і так далі. Тому більш загальні маршрути повинні визначатися в останню чергу, а більш конкретні маршрути повинні йти на початку. Наприклад:

```
from django.urls import path, re_path
from hello import views

urlpatterns = [
    re_path(r'^about/contact/', views.contact),
    re_path(r'^about', views.about),
    path('', views.index),
]
```

У цьому випадку адреса "`^about/contact`" представляє більш конкретний маршрут порівняно з "`^about`". Тому вона визначається насамперед.

Якби було навпаки:

```
from django.urls import path, re_path
from hello import views

urlpatterns = [
    path('', views.index),
    re_path(r'^about', views.about),
    re_path(r'^about/contact', views.contact),
]
```

то запит за адресою "`about/contact`" оброблявся б функцією `views.about`

2.2.3 Основні елементи синтаксису регулярних виразів

Деякі базові елементи регулярних виразів, які можна використовувати для визначення адрес URL:

- $\wedge$ (початок адреси)
- $\$$ (кінець адреси)
- $+$ (1 і більше символів)
- $?$ (0 або 1 символ)
- $\{n\}$ (n символів)
- $\{n, m\}$ (від n до m символів)
- $.$ (будь-який символ)
- $\backslash d+$ (одна або кілька цифр)
- $\backslash D+$ (одна або кілька НЕ цифр)
- $\backslash w+$ (один або кілька буквених символів)

Розглянемо кілька можливих зіставлень шаблонів адрес і запитів:

Адреса	Запит
$r'^{\wedge}\$'$	<code>http://127.0.0.1/</code> (корінь сайту)
$r'^{\wedge}about'$	<code>http://127.0.0.1/about/</code> або <code>http://127.0.0.1/about/contact</code>
$r'^{\wedge}about/contact'$	<code>http://127.0.0.1/about/contact</code>
$r'^{\wedge}products/\backslash d+/'$	<code>http://127.0.0.1/products/23/</code> або <code>http://127.0.0.1/products/6459/abc</code> Але не відповідає запиту <code>http://127.0.0.1/products/abc/</code>
$r'^{\wedge}products/\backslash D+/'$	<code>http://127.0.0.1/products/abc/</code> або <code>http://127.0.0.1/products/abc/123</code> Але не відповідає запиту <code>http://127.0.0.1/products/123/</code> або <code>http://127.0.0.1/products/123/abc</code>
$r'^{\wedge}products/phones tablets/'$	<code>http://127.0.0.1/products/phones/1</code> або <code>http://127.0.0.1/products/tablets/</code> Але не відповідає запиту <code>http://127.0.0.1/products/clothes/</code>
$r'^{\wedge}products/\backslash w+/'$	<code>http://127.0.0.1/products/abc/</code> або <code>http://127.0.0.1/products/123/</code> Але не відповідає запиту <code>http://127.0.0.1/products/abc-123</code>
$r'^{\wedge}products/[-\backslash w]+/'$	<code>http://127.0.0.1/products/abc-123</code>
$r'^{\wedge}products/[A-Z]{2}'$	<code>http://127.0.0.1/products/UA</code> Але не відповідає запиту <code>http://127.0.0.1/products/Ua</code> або <code>http://127.0.0.1/products/UKR</code>

2.2.4 Передача значень у функцію

Вище були розглянуті всі параметри функцій `path()` і `re_path()`, крім одного - `kwargs`, який дає змогу передати до функції-представлення деякі значення. Наприклад, у файлі `views.py` визначимо такі функції:

```
from django.http import HttpResponse

def index(request):
    return HttpResponse("<h1>Main</h1>")

def about(request, name, age):
    return HttpResponse(f"""
        <h2>About</h2>
        <p>Name: {name}</p>
        <p>Age: {age}</p>
    """)
```

Тут функція *about()* також приймає два додаткові параметри: *name* і *age* (умовно ім'я та вік користувача). У функції їхні значення надсилаються користувачеві разом із рештою вмісту.

Змінимо файл *urls.py*:

```
from django.urls import path
from hello import views

urlpatterns = [
    path('', views.index),
    path('about', views.about, kwargs={"name": "Bill", "age": 45}),
]
```

За допомогою параметра *kwargs* у функцію *about()* передається словник із двома значеннями - для двох параметрів функції. Відповідно при зверненні до цієї функції ми побачимо в браузері відповідні дані:

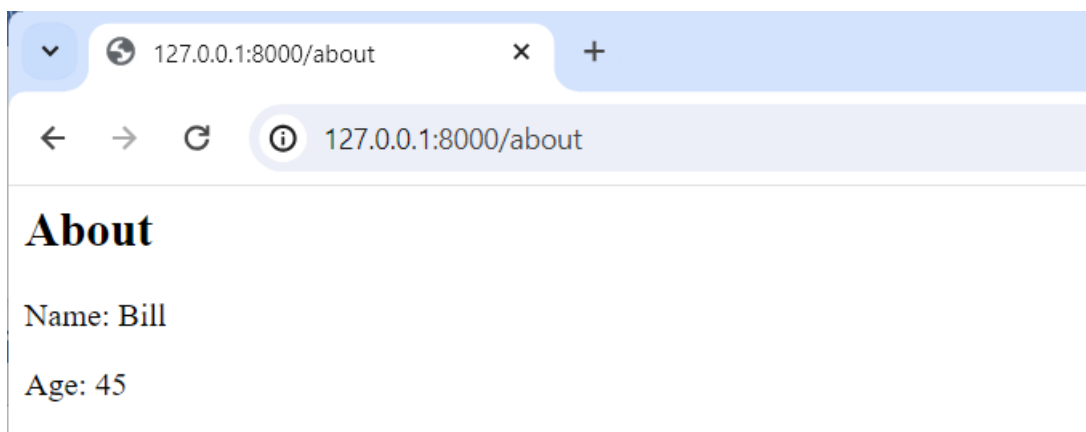


Рисунок 2.15 – Результат роботи

2.3 Отримання даних запиту. HttpRequest

Функції-представлення як обов'язковий параметр отримують об'єкт `HttpRequest`, який зберігає інформацію про запит. `HttpRequest` визначає низку атрибутів, які зберігають інформацію про запит. Виділимо такі з них:

- *scheme*: схема запиту (*http* або *https*)
- *body*: представляє тіло запиту у вигляді рядка байтів
- *path*: представляє шлях запиту
- *method*: метод запиту (*GET*, *POST*, *PUT* тощо)
- *encoding*: кодування
- *content_type*: тип вмісту запиту (значення заголовка *CONTENT_TYPE*)
- *GET*: об'єкт у вигляді словника, який містить параметри запиту *GET*
- *POST*: об'єкт у вигляді словника, який містить параметри запиту *POST*
- *COOKIES*: відправлені клієнтом куки
- *FILES*: відправлені клієнтом файли
- *META*: зберігає всі доступні заголовки *http* у вигляді словника. Набір заголовків залежить від клієнта і сервера, деякі з них:

- *CONTENT_LENGTH*: довжина вмісту.
- *CONTENT_TYPE*: *MIME*-тип запиту.
- *HTTP_ACCEPT*: типи відповіді, які приймає клієнт.
- *HTTP_ACCEPT_ENCODING*: кодування, у якому клієнт приймає відповідь.
- *HTTP_ACCEPT_LANGUAGE*: мова відповіді, яку приймає клієнт.
- *HTTP_HOST*: хост сервера.
- *HTTP_REFERER*: сторінка, з якої клієнт надіслав запит (за її наявності).
- *HTTP_USER_AGENT*: юзер-агент або інформація про браузер клієнта.
- *QUERY_STRING*: рядок запиту.
- *REMOTE_ADDR*: *IP*-адреса клієнта.
- *REMOTE_HOST*: ім'я хоста клієнта.
- *REMOTE_USER*: аутентифікаційні дані клієнта (за наявності).

- `REQUEST_METHOD`: тип запиту (`GET`, `POST`).
- `SERVER_NAME`: ім'я хоста сервера.
- `SERVER_PORT`: порт сервера.
- `headers`: заголовки запиту у вигляді словника.

Також `HttpRequest` визначає низку методів. Відзначимо такі з них:

- `get_full_path()`: повертає повний шлях запиту, включно з рядком запиту
- `get_host()`: повертає хост клієнта, для цього використовується значення заголовків `HTTP_X_FORWARDED_HOST` (якщо ввімкнено опцію `USE_X_FORWARDED_HOST`) і `HTTP_HOST`
- `get_port()`: повертає номер порту

Наприклад, отримаємо деяку інформацію про запит. Для цього у файлі `views.py`:

```
from django.http import HttpResponse

def index(request):
    host = request.META["HTTP_HOST"] # отримуємо адресу сервера
    user_agent = request.META["HTTP_USER_AGENT"] # отримуємо дані браузера
    path = request.path # отримуємо запитаний шлях

    return HttpResponse(f"""
        <p>Host: {host}</p>
        <p>Path: {path}</p>
        <p>User-agent: {user_agent}</p>
    """)
```

У цьому випадку отримуємо два заголовки `"HTTP_HOST"` і `"HTTP_USER_AGENT"` і запитаний шлях.

У файлі `urls.py` зареєструємо цю функцію:

```
from django.urls import path
from hello import views

urlpatterns = [
    path("index", views.index),
]
```

Результат роботи:

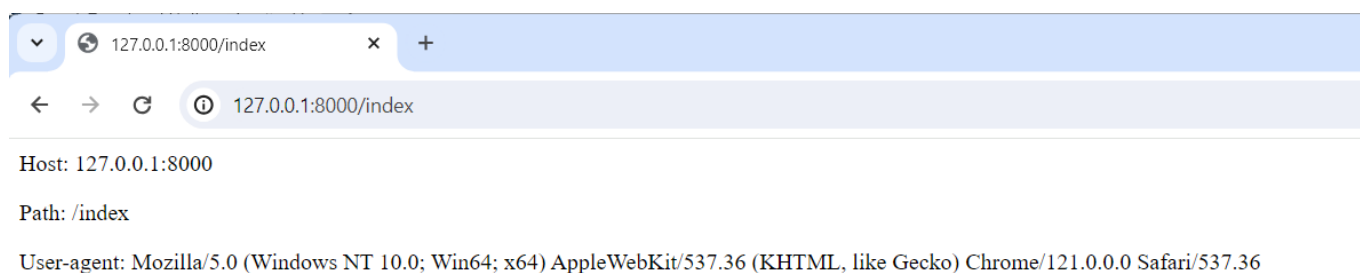


Рисунок 2.16 – Результат роботи

2.4 HttpResponse і надсилання відповіді

Для надсилання відповіді клієнту в Django застосовується клас *HttpResponse* з пакета *django.http*. У загальному випадку для надсилання деяких даних достатньо ці дані передати в конструктор *HttpResponse*. Наприклад, нехай у файлі *views.py* є найпростіша функція-представлення, яка надсилає відповідь клієнту:

```
from django.http import HttpResponse

def index(request):
    return HttpResponse("Hello")
```

І у файлі *urls.py* ця функція співвідноситься з деяким маршрутом:

```
from django.urls import path
from hello import views

urlpatterns = [
    path("", views.index),
]
```

Подібна функція просто передає в *HttpResponse* деякий текст, який користувач потім побачить у браузері. Однак подібною функціональністю *HttpResponse* не обмежується. Так, функція ініціалізації класу визначає кілька параметрів:

```
HttpResponse.__init__(content=b'', content_type=None, status=200,
reason=None, charset=None, headers=None)
```

Параметри:

- *content*: вміст відповіді у вигляді рядка байтів. Якщо передається інший вміст, то він конвертується в рядок байтів
- *content_type*: *MIME*-тип відповіді, встановлює *HTTP*-заголовок *Content-Type*. Якщо цей параметр не встановлено, то застосовується *mime*-тип *text/html* і значення налаштування *DEFAULT_CHARSET*, тобто в підсумку буде: *"text/html; charset=utf-8"*.
- *charset*: кодування відповіді у вигляді рядка. За замовчуванням *django* намагається встановити кодування з параметра *content_type*, а в разі невдачі для встановлення кодування застосовується налаштування *DEFAULT_CHARSET*.
- *status*: статусний код відповіді. За замовчуванням дорівнює 200
- *reason_phrase*: повідомлення, яке надсилається разом зі статусним кодом
- *headers*: заголовки відповіді у вигляді словника

Для зберігання даних, що надсилаються, він визначає низку атрибутів. Деякі з них:

- *content*: вміст відповіді у вигляді рядка байтів
- *headers*: відправлені заголовки у вигляді словника
- *charset*: кодування відповіді у вигляді рядка. За замовчуванням *django* намагається встановити кодування із заголовка *content_type*, а в разі невдачі для встановлення кодування застосовується налаштування *DEFAULT_CHARSET*.
- *status_code*: статусний код відповіді
- *reason_phrase*: повідомлення, яке надсилається разом зі статусним кодом

Розглянемо деякі можливості. Наприклад, змінимо визначення функції у файлі *views.py*:

```
from django.http import HttpResponse

def index(request):
    return HttpResponse("Hello WebPython", headers={"SecretCode": "qwerty123456"})
```

У цьому випадку також встановлюється заголовок *"SecretCode"*. Хоча в реальності в *HTTP* не існує такого заголовка, але ми можемо визначати кастомні заголовки, щоб передати через них клієнту якусь інформацію. Наприклад, після звернення до функції ми можемо в браузері через інструменти розробника проінспектувати відправлені сервером заголовки і знайти там, зокрема, заголовок *"SecretCode"*.

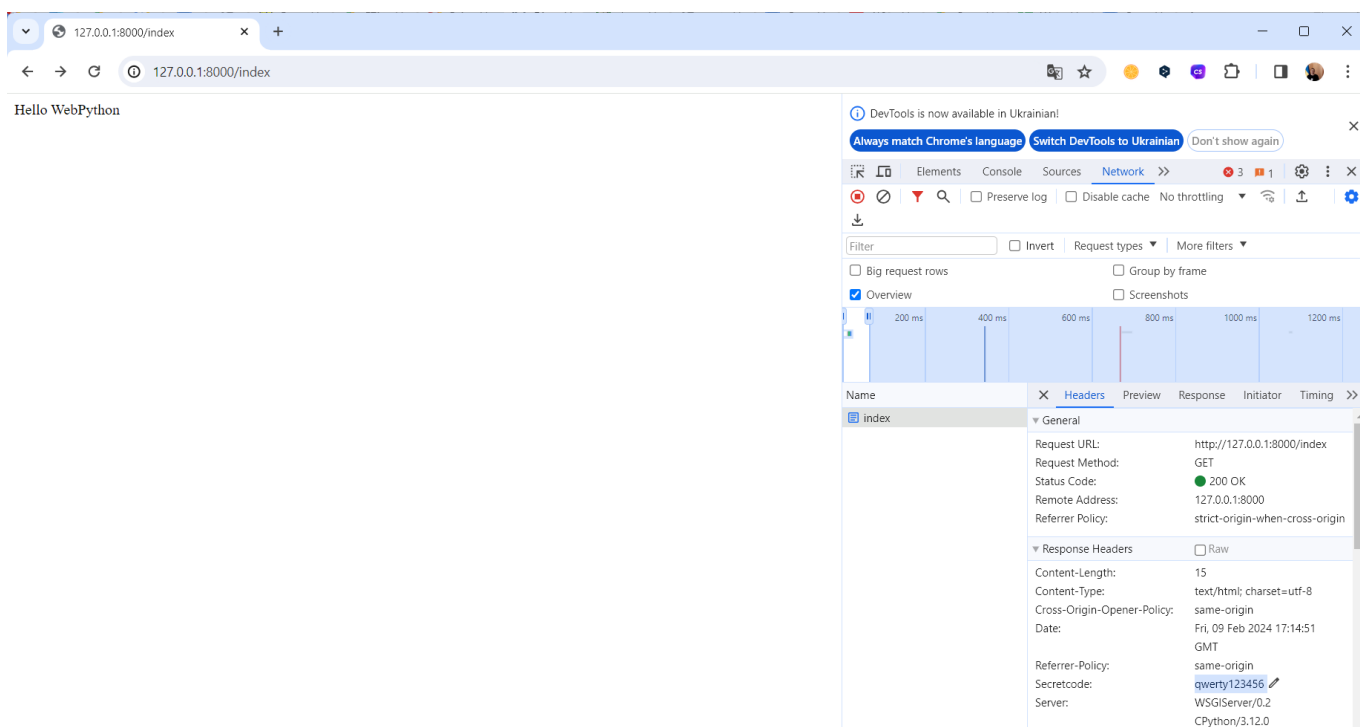


Рисунок 2.17 – Результат роботи

Подібним чином можна встановити інші параметри, наприклад, статусний код і повідомлення до нього:

```
from django.http import HttpResponse

def index(request):
    return HttpResponse("Сталася помилка", status=400, reason="Incorrect data")
```

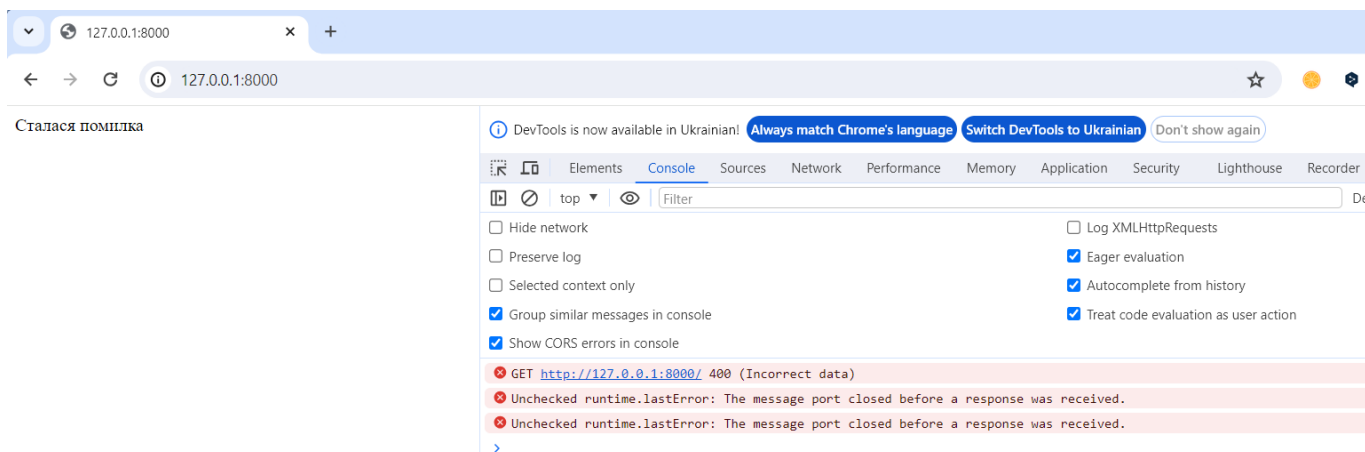


Рисунок 2.18 – Результат надсилання статусного коду

Встановлення вмісту та кодування:

```
from django.http import HttpResponse

def index(request):
    return HttpResponse("<h1>Hello</h1>", content_type="text/plain", charset="utf-8")
```

Хоча у вмісті відповіді застосовуються теги *html* (*<h1>*), але браузер тепер розглядатиме цей вміст як простий текст, тому що встановлено заголовок "*text/plain*":

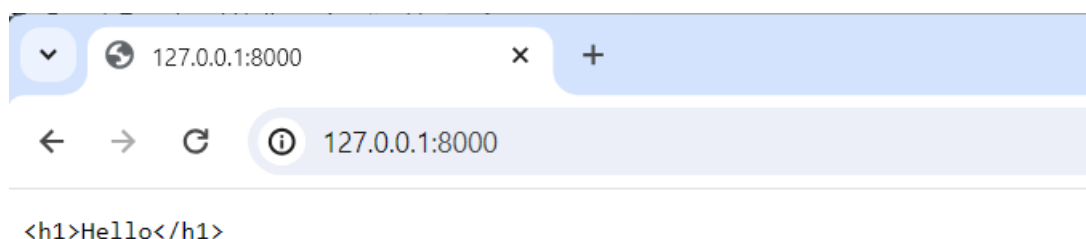


Рисунок 2.19 – Результат встановлення вмісту та кодування

2.5 Параметри подань

Функції-представлення можуть приймати параметри, через які можуть передаватися різні дані. Подібні параметри передаються в адресі *URL*. Наприклад, у запиті

```
http://127.0.0.1:8000/index/Bill/45/
```

останні два сегменти *Bill/45/* можуть представляти параметри *URL*, які можуть бути пов'язані з параметрами функції-представлення через систему маршрутизації. Подібні параметри ще можна назвати параметрами маршруту

2.5.1 Визначення параметрів через функцію `path()`

Визначимо у файлі *views.py* такі функції:

```
from django.http import HttpResponse

def index(request):
    return HttpResponse("<h2>Main</h2>")

def user(request, name):
    return HttpResponse(f"<h2>Name: {name}</h2>")
```

У цьому випадку функція *user* має два параметри. Для другого параметра - *name* ми будемо отримувати дані з рядка запиту. Тобто це буде параметр маршруту. І як будь-який інший параметр ми зможемо використовувати його всередині функції.

Далі у файлі *urls.py* визначимо такий код:

```
from django.urls import path
from hello import views

urlpatterns = [
    path("", views.index),
    path("user/<str:name>", views.user),
]
```

Для другого маршруту тут визначено параметр *name* (який відповідає параметру *name* функції *views.user*). Параметри укладаються в кутові дужки у форматі *<специфікатор:назва_параметра>*. Наприклад, тут параметр *name* має специфікатор *str*.

За замовчуванням *Django* надає такі специфікатори:

- *str*: відповідає будь-якому рядку за винятком символу *" / "*. Якщо специфікатор не вказано, то використовується за замовчуванням
- *int*: відповідає будь-якому позитивному числу
- *slug*: відповідає послідовності буквених символів *ASCII*, цифр, дефіса і символу підкреслення, наприклад, *building-your-1st-django-site*
- *uuid*: відповідає ідентифікатору *UUID*, наприклад, *075194d3-6885-417e-a8a8-6c931e272f00*
- *path*: відповідає будь-якому рядку, який також може включати символ *" / "* на відміну від специфікатора *str*

Також зазначу, що кількість і назва параметрів у шаблонах адрес URL відповідають кількості та назвам параметрів відповідних функцій, які обробляють запити за даними адресами.

Запустимо застосунок, звернемося до функції *views.user*, наприклад, за допомогою запиту *http://127.0.0.1:8000/user/Bill*, і через рядок запиту - через його третій сегмент ми зможемо передати значення для параметра *name*:

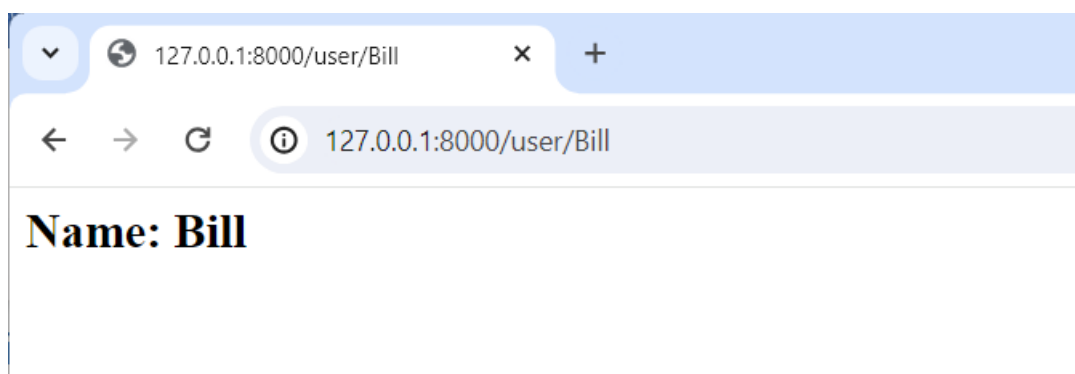


Рисунок 2.20 – Результат передачі значення параметру

Подібним чином можна визначити і більшу кількість параметрів. Наприклад, додамо другий параметр у функцію *user* у *views.py*:

```
from django.http import HttpResponse

def index(request):
    return HttpResponse("<h1>Main</h1>")

def user(request, name, age):
    return HttpResponse(f"<h2>Name: {name} Age:{age}</h2>")
```

А у файлі `urls.py` додамо в маршрут параметр `age`:

```
from django.urls import path
from hello import views

urlpatterns = [
    path("", views.index),
    path("user/<name>/<int:age>", views.user),
]
```

Передбачається, що параметр `age` представлятиме число, тому для нього використовується специфікатор `int`. У цьому випадку ми можемо звернутися до функції `user`, наприклад, за допомогою запиту:

```
http://127.0.0.1:8000/user/Bill/45
```

У цьому разі сегмент `"/Bill/"` буде зіставлено з параметром `name`, а `"/45"` - з параметром `age`.

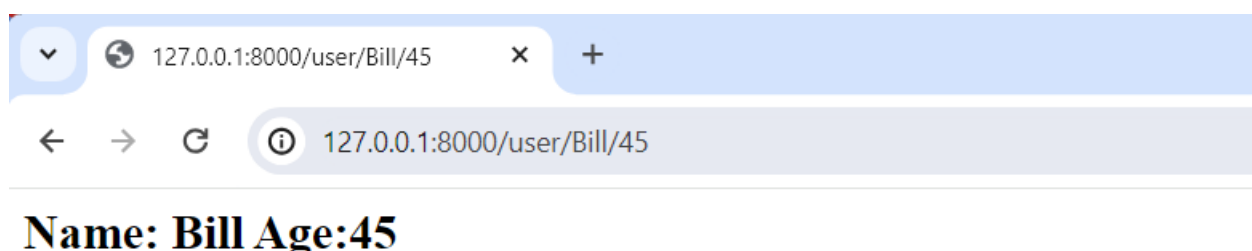


Рисунок 3.21 – Результат передаї значень двох параметрів

2.5.2 Значення для параметрів за замовчуванням

У прикладі вище використовувалися два параметри, але що, якщо ми не передамо для одного або обох параметрів значення?

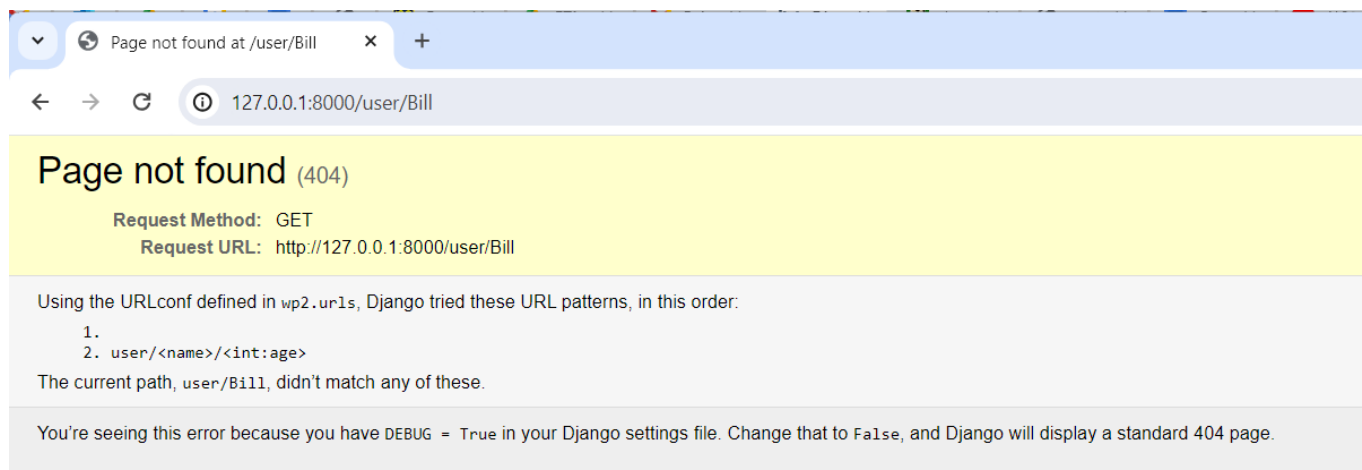


Рисунок 2.22 – Результат появи помилки при передачі не всіх параметрів

У цьому випадку ми отримаємо помилку. Параметри маршруту є частиною шаблону *URL*. Відповідно якщо в рядку запиту не передаються значення для параметрів, такий запит не відповідає маршруту. Тому *Django* не зможе знайти потрібний маршрут для обробки запиту, і ми отримаємо помилку 404 (ресурс не знайдено).

Однак така поведінка не завжди може бути бажаною. І ми можемо задати 4 для параметрів маршруту значення за замовчуванням на випадок, якщо через рядок запиту не передаються значення. Так, для функції *user* у *views.py* визначимо значення для параметрів за замовчуванням:

```
from django.http import HttpResponse

def index(request):
    return HttpResponse("<h1>Main</h1>")

def user(request, name="Undefined", age =0):
    return HttpResponse(f"<h2>Name: {name} Age: {age}</h2>")
```

У цьому випадку, якщо для параметра *name* не передається значення, то він отримує як значення рядок *"Undefined"*. Для параметра *age* значення за замовчуванням *0*.

У цьому випадку для функції *user* у файлі *urls.py* треба визначити додаткові маршрути, які не враховують необов'язкові параметри:

```
from django.urls import path
from hello import views

urlpatterns = [
    path("", views.index),
    path("user", views.user),
    path("user/<name>", views.user),
    path("user/<name>/<int:age>", views.user),
]
```

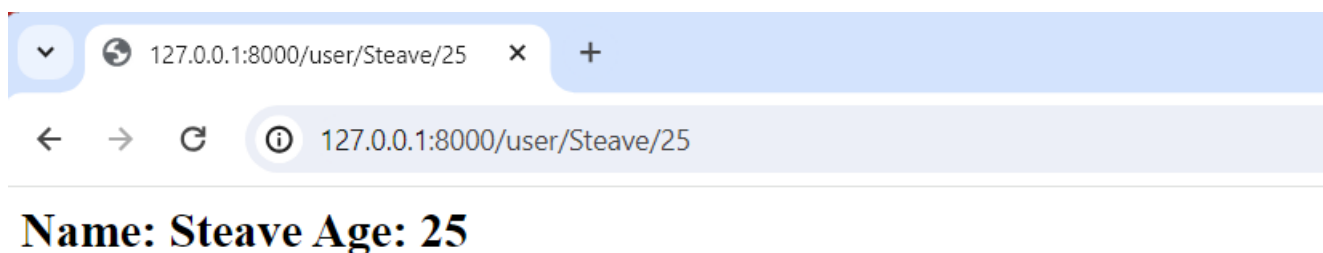


Рисунок 2.23 – Результат передачі значень двох параметрів



Рисунок 2.24 – Результат передачі значення одного параметра

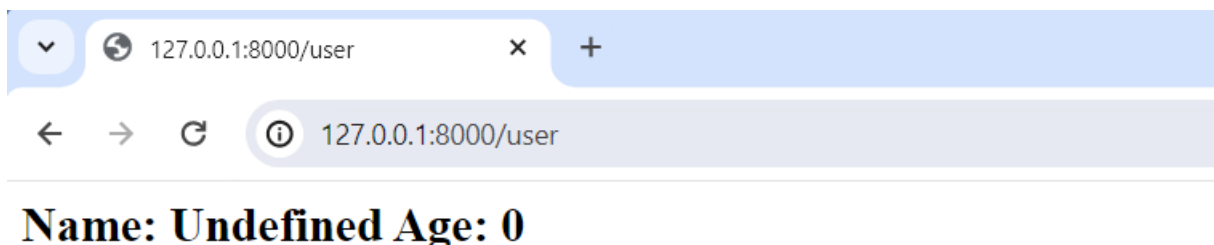


Рисунок 2.25 – Результат запиту без параметрів

2.5.3 Визначення параметрів через функцію `re_path()`

Подібним чином ми можемо використовувати функцію `re_path()` для визначення параметрів. Визначимо в додатку у файлі `views.py` такі функції:

```
from django.http import HttpResponse

def index(request):
    return HttpResponse("<h1>Main</h1>")

def user(request, name, age):
    return HttpResponse(f"<h2>Name: {name} Age: {age}</h2>")
```

Тепер змінимо файл `urls.py`, щоб він міг зіставити дані функції із запитами:

```
from django.urls import path, re_path
from hello import views

urlpatterns = [
    path("", views.index),
    re_path(r"^user/(?P<name>\D+)/(?P<age>\d+)", views.user),
]
```

Для представлення параметра в шаблоні адреси використовується вираз `?P<>`. Загальне визначення параметру відповідає формату `(?P<ім'я_параметра>регулярний_вираз)`. Між кутовими дужками поміщається назва параметра. Після кутової дужки, що закриває, йде регулярний вираз, якому має відповідати значення параметра.

У другому шаблоні адреси визначаються два параметри: *name* і *age*. При цьому параметр *age* має представляти число, а параметр *name* має складатися тільки з буквених символів.

Тепер ми можемо через адресний рядок передати дані в додаток:

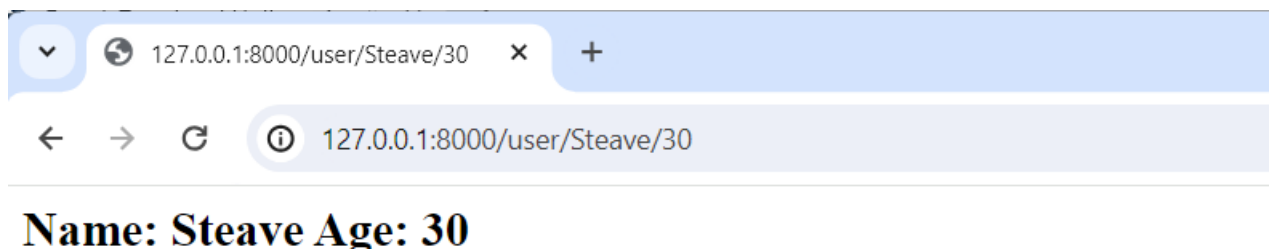


Рисунок 2.26 – Результат передачі значень параметрів

Також ми можемо вказати для певних параметрів значення за замовчуванням:

```
from django.http import HttpResponse

def index(request):
    return HttpResponse("<h1>Main</h1>")

def user(request, name="Undefined", age =0):
    return HttpResponse(f"<h2>Name: {name} Age: {age}</h2>")
```

У цьому разі треба додатково визначити ще маршрути у файлі *urls.py* для тих запитів, у яких не передаються значення для маршрутів:

```
from django.urls import path, re_path
from hello import views

urlpatterns = [
    path("", views.index),
    re_path(r"^user/(?P<name>\D+)/(?P<age>\d+)", views.user),
    re_path(r"^user/(?P<name>\D+)", views.user),
    re_path(r"^user", views.user),
]
```

Зверніть увагу на порядок розміщення маршрутів: на відміну від випадку з функцією *path()*, тепер спочатку розміщуються конкретніші маршрути з великою кількістю параметрів.

2.6 Вкладені маршрути

2.6.1 Функція *include()*

Функція *include()* дозволяє визначити вкладені маршрути або підмаршрути для деякого маршруту. Як параметр вона приймає набір маршрутів:

```
include(pattern_list)
```

Параметр *pattern_list* представляє набір викликів функцій *path()* та/або *re_path()*. Наприклад, визначимо у файлі *views.py* такі функції:

```
from django.http import HttpResponse

def index(request):
    return HttpResponse("Main")

def products(request):
    return HttpResponse("Products list")

def new(request):
    return HttpResponse("New")

def top(request):
    return HttpResponse("Top list of products")
```

Тут останні три функції функціонально належать до товарів.

І визначимо у файлі *urls.py* такі маршрути:

```
from django.urls import path, include
from hello import views

product_patterns = [
    path("", views.products),
    path("new", views.new),
    path("top", views.top),
]

urlpatterns = [
    path("", views.index),
    path("products/", include(product_patterns)),
]
```

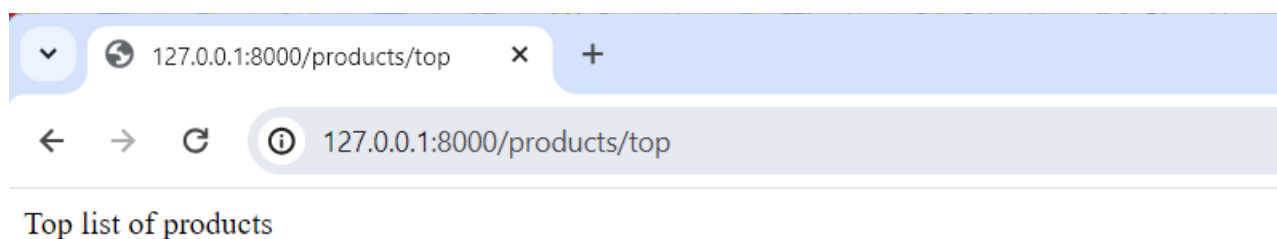
Тут у вигляді змінної *product_patterns* окремо визначено набір маршрутів, який стосується товарів.

Для встановлення цих маршрутів цей список передається у функцію *include()*.

```
path("products", include(product_patterns)),
```

Причому цей список буде асоційовано з шаблоном "product". У цьому разі шаблони вкладених маршрутів будуть об'єднані з шаблоном батьківського маршруту, і в такий спосіб буде сформовано загальний шаблон, якому має відповідати запит.

Наприклад, якщо надійде запит "*http://127.0.0.1:8000/products/top*", то його оброблятиме функція *top*.



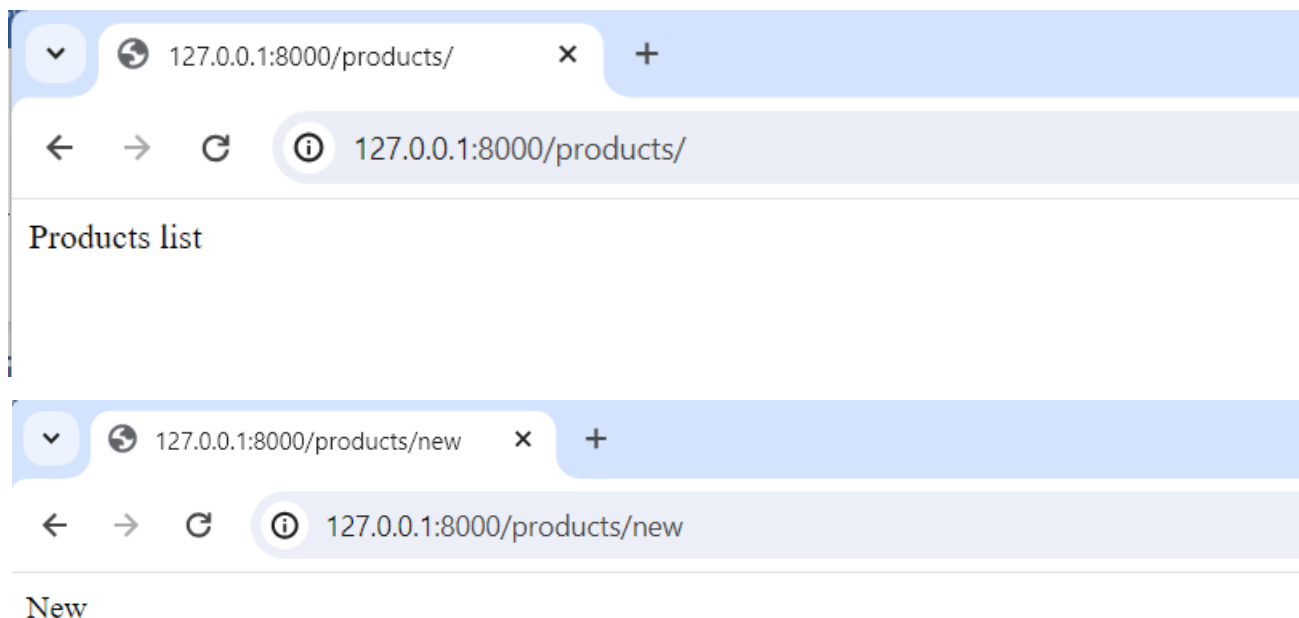


Рисунок 2.27 – Результат групування запитів

Таким чином, ми можемо згрупувати маршрути для запитів, які починаються з певного шаблону. Що також дає змогу уникнути повтору частин шаблону, коли шаблон *url* починається з одного й того самого сегмента.

2.6.2 Отримання параметрів

Вкладені маршрути отримують параметри, визначені в батьківських маршрутах. Наприклад, визначимо у файлі *views.py* такі функції-представлення:

```
from django.http import HttpResponse

def index(request):
    return HttpResponse("Home page")

def products(request, id):
    return HttpResponse(f"Product {id}")

def comments(request, id):
    return HttpResponse(f"Comments about the product {id}")

def questions(request, id):
    return HttpResponse(f"Questions about the product {id}")
```

Тут усі функції крім першої мають другий параметр - *id* - умовний номер товару.

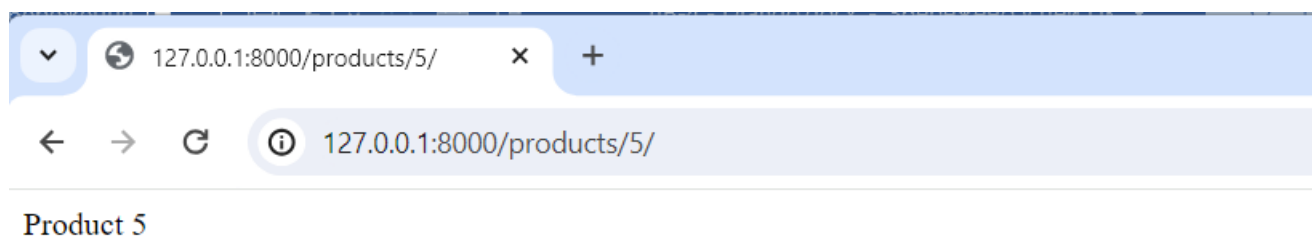
Змінимо файл *urls.py*:

```
from django.urls import path, include
from hello import views

product_patterns = [
    path("", views.products),
    path("comments", views.comments),
    path("questions", views.questions),
]

urlpatterns = [
    path("", views.index),
    path("products/<int:id>/", include(product_patterns)),
]
```

Тут для другого маршруту визначається числовий параметр *id*. Цей параметр передається всім вкладеним маршрутам, відповідно нам не треба визначати цей параметр у вкладених маршрутах:



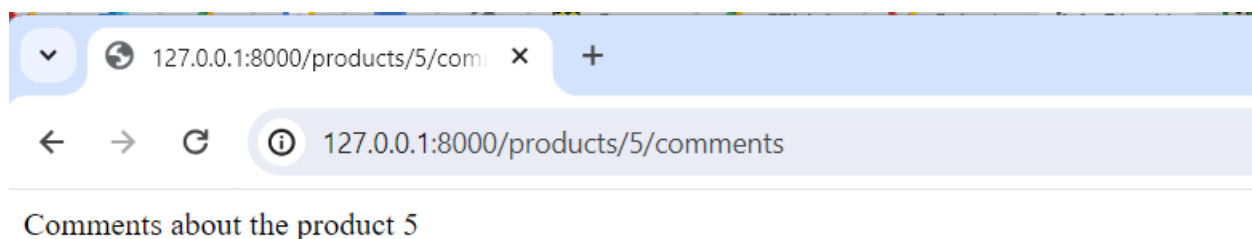
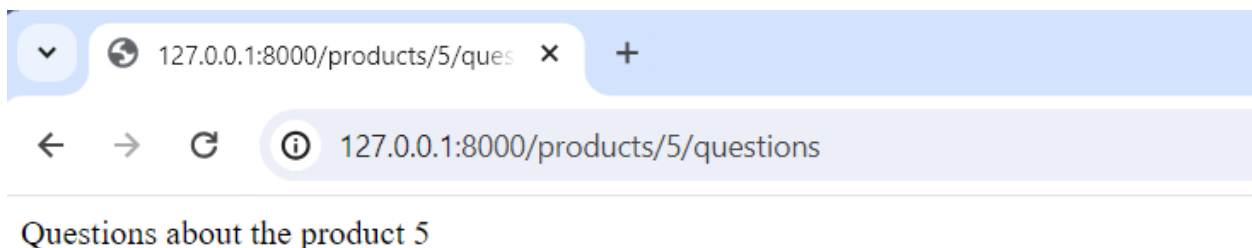


Рисунок 2.28 – результат обробки вкладених маршрутів

2.7 Параметри рядка запиту

Від параметрів, які передаються через адресу *URL*, слід відрізняти параметри, які передаються через рядок запиту. Наприклад, у запиті:

```
http://127.0.0.1:8000/index/3/Bill/
```

два останні сегменти - 3/Bill/ представляють параметри *URL* або параметри маршруту. А в запиті:

```
http://127.0.0.1:8000/index?id=3&name=Tom
```

ті ж самі значення 3 і Bill представляють параметри рядка запиту.

Параметри рядка запиту вказуються після знаке питання ?. Кожен параметр представляє пару ключ-значення, наприклад, в *id=3* : *id* - назва або ключ параметра, а 3 - його значення. Параметри в рядку запиту відокремлюються один від одного знаком амперсанду.

Для отримання параметрів із рядка запиту застосовується метод *request.GET.get()*, у який передається назва параметра.

Наприклад, визначимо у файлі *views.py* такі функції:

```
from django.http import HttpResponse

def index(request):
    return HttpResponse("<h1>Main</h1>")

def user(request):
    age = request.GET.get("age")
    name = request.GET.get("name")
    return HttpResponse(f"<h2>Name: {name} Age: {age}</h2>")
```

Функція *user* приймає витягує з рядка запиту два параметри: *name* і *age*.

У файлі *urls.py* визначимо такі маршрути:

```
from django.urls import path
from hello import views

urlpatterns = [
    path("", views.index),
    path("user/", views.user)
]
```

При зверненні до додатка за адресою *http://127.0.0.1:8000/user/?name=Jack&age=30*, параметр *name* матиме значення "Jack", а параметр *age* - 30.

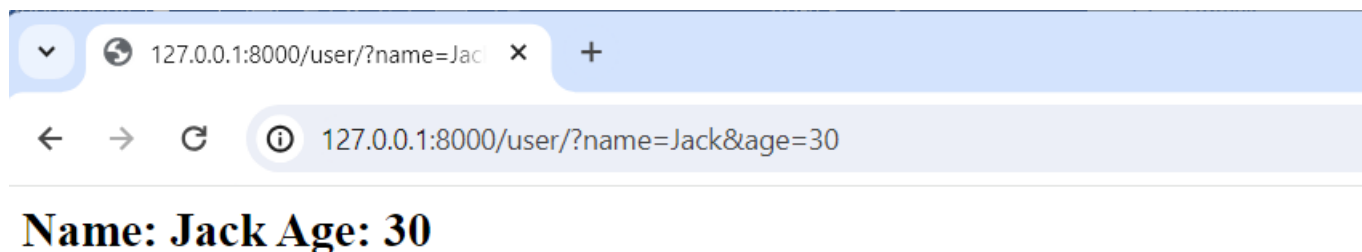


Рисунок 2.29 – Результат передачі значень параметрів

Однак можлива ситуація, коли для якихось параметрів не буде передано значення. У цьому випадку ми можемо вказати для подібних параметрів значення за замовчуванням. Для цього змінимо визначення функції *user* у *views.py*:

```
from django.http import HttpResponse

def index(request):
    return HttpResponse("<h1>Home page</h1>")

def user(request):
    age = request.GET.get("age", 0)
    name = request.GET.get("name", "Undefined")
    return HttpResponse(f"<h2>Name: {name} Age: {age}</h2>")
```

Другий параметр функції `request.GET.get()` представляє значення за замовчуванням для параметра, для якого не задано значення.

Відповідно при зверненні за адресою `http://127.0.0.1:8000/user/` параметри *name* і *age* отримають значення за замовчуванням:

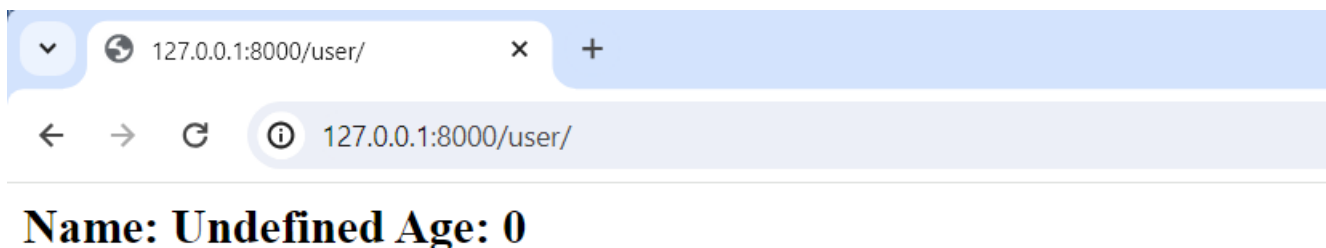


Рисунок 2.30 – Результат передачі значень параметрів за замовчень

2.8 Переадресація та надсилання статусних кодів

2.8.1 Переадресація

Під час переміщення документа з однієї адреси на іншу ми можемо скористатися механізмом переадресації, щоб вказати користувачам і пошуковику, що документ тепер доступний за новою адресою.

Переадресація буває тимчасова та постійна. Під час тимчасової переадресації ми вказуємо, що документ тимчасово переміщено на нову адресу. У цьому разі у відповідь надсилається статусний код 302. У разі постійної переадресації ми повідомляємо, що документ тепер постійно буде доступний за новою адресою.

Для створення тимчасової переадресації застосовується клас *HttpResponseRedirect*, а для постійної - клас *HttpResponsePermanentRedirect*, які розташовані в пакеті *django.http*.

Так, визначимо у файлі *views.py* такий код:

```
from django.http import HttpResponseRedirect, \
HttpResponsePermanentRedirect

def index(request):
    return HttpResponseRedirect("/")

def about(request):
    return HttpResponseRedirect("/about")

def contact(request):
    return HttpResponseRedirect("/")

def details(request):
    return HttpResponseRedirect("/")
```

При зверненні до функції *contact* вона буде перенаправляти за шляхом *"about"*, який буде оброблятися функцією *about()*. А функція *details()* використовуватиме постійну переадресацію і перенаправлятиме на корінь веб-додатку.

І також у файлі *urls.py* для тестування визначимо такі маршрути:

```
from django.urls import path
from hello import views

urlpatterns = [
    path("", views.index),
    path("about/", views.about),
    path("contact/", views.contact),
    path("details/", views.details),
]
```

2.8.2 Надсилення статусних кодів

Також у пакеті *django.http* є низка класів, які дають змогу надсилати певний статусний код:

Статусний код	Клас
304 (Not Modified)	<code>HttpResponseNotModified</code>
400 (Bad Request)	<code>HttpResponseBadRequest</code>
403 (Forbidden)	<code>HttpResponseForbidden</code>
404 (Not Found)	<code>HttpResponseNotFound</code>
405 (Method Not Allowed)	<code>HttpResponseNotAllowed</code>
410 (Gone)	<code>HttpResponseGone</code>
500 (Internal Server Error)	<code>HttpResponseServerError</code>

У функцію конструктора цих класів можна передати деякі дані, наприклад, повідомлення про помилку, яке побачить користувач:

```
HttpResponseNotModified()  
HttpResponseBadRequest("Bad Request")  
HttpResponseForbidden("Forbidden")  
HttpResponseNotFound("Not Found")  
HttpResponseNotAllowed("Method is not allowed")  
HttpResponseGone("Content is no longer here")  
HttpResponseServerError("Server Error")
```

Наприклад, визначимо такий файл *views.py*:

```
from django.http import HttpResponse, HttpResponseNotFound, HttpResponseForbidden,
HttpResponseBadRequest

def index(request, id):
    people = ["Sir", "Alex", "Ferguson"]
    # якщо користувача знайдено, повертаємо його
    if id in range(0, len(people)):
        return HttpResponse(people[id])
    # якщо ні, то повертаємо помилку 404
    else:
        return HttpResponseNotFound("Not Found")

def access(request, age):
    # якщо вік НЕ входить у діапазон 1-100, надсилаємо помилку 400
    if age not in range(1, 101):
        return HttpResponseBadRequest("Incorrect data")
    # якщо вік понад 17, то доступ дозволено
    if(age > 17):
        return HttpResponse("Access is allowed")
    # якщо ні, то повертаємо помилку 403
    else:
        return HttpResponseForbidden("Access blocked: not enough years")
```

Функція *index()* приймає параметр *id*. Це буде індекс елемента в списку *people*. Однак користувач може передати і недійсний індекс, наприклад, 100, хоча в прикладі в списку тільки 3 елементи. Тому, якщо передано дійсний індекс, то повертаємо елемент за цим індексом. Якщо ж індекс недійсний, то за допомогою класу *HttpResponseNotFound* повертаємо помилку 404 і повідомлення про помилку.

Аналогічно функція *access()* приймає параметр *age*, який представляє умовний повернення користувача. Якщо вік виходить за деякі розумні межі (1-100), то за допомогою класу *HttpResponseBadRequest* повертаємо помилку 400. Якщо вік укладається в ці рамки, але він менший за 18, то за допомогою класу *HttpResponseForbidden* повертаємо помилку 403 про те, що доступ заборонено.

Для тесту у файлі *urls.py* визначимо такі маршрути:

```
from django.urls import path
from hello import views

urlpatterns = [
    path("index/<int:id>", views.index),
    path("access/<int:age>", views.access),
]
```

Звернутися до функції `index()`, передавши для параметра `id` коректні та некоректні значення. І залежно від значення параметра ми побачимо або дані зі списку *people*, або помилку 404:

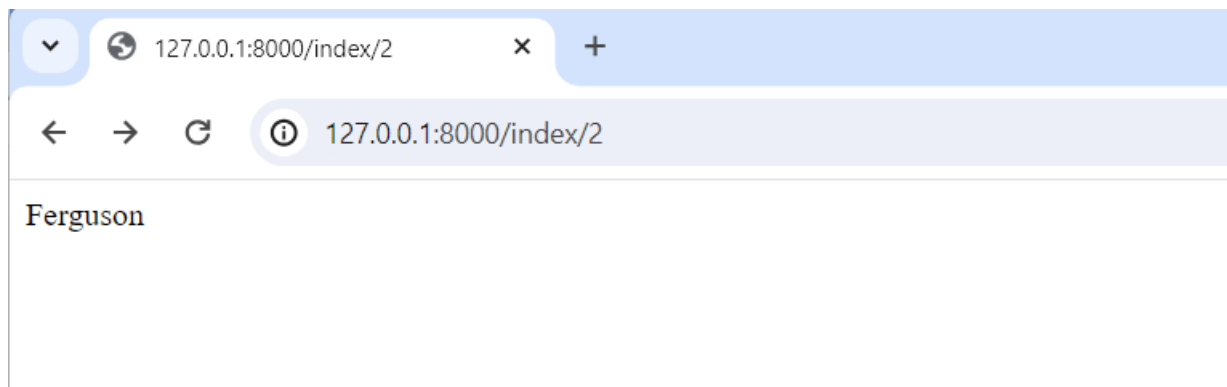


Рисунок 2.31 – Результат обробки коректного запиту

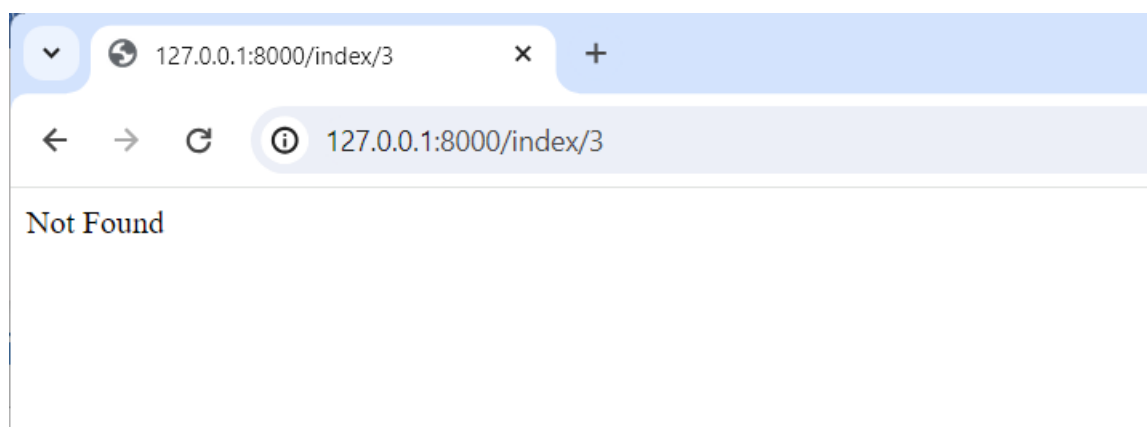
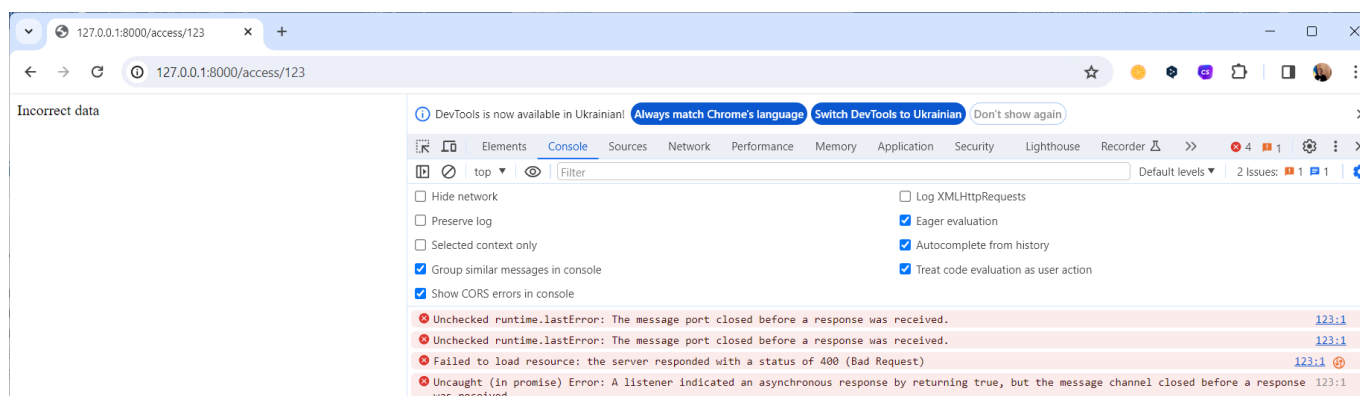
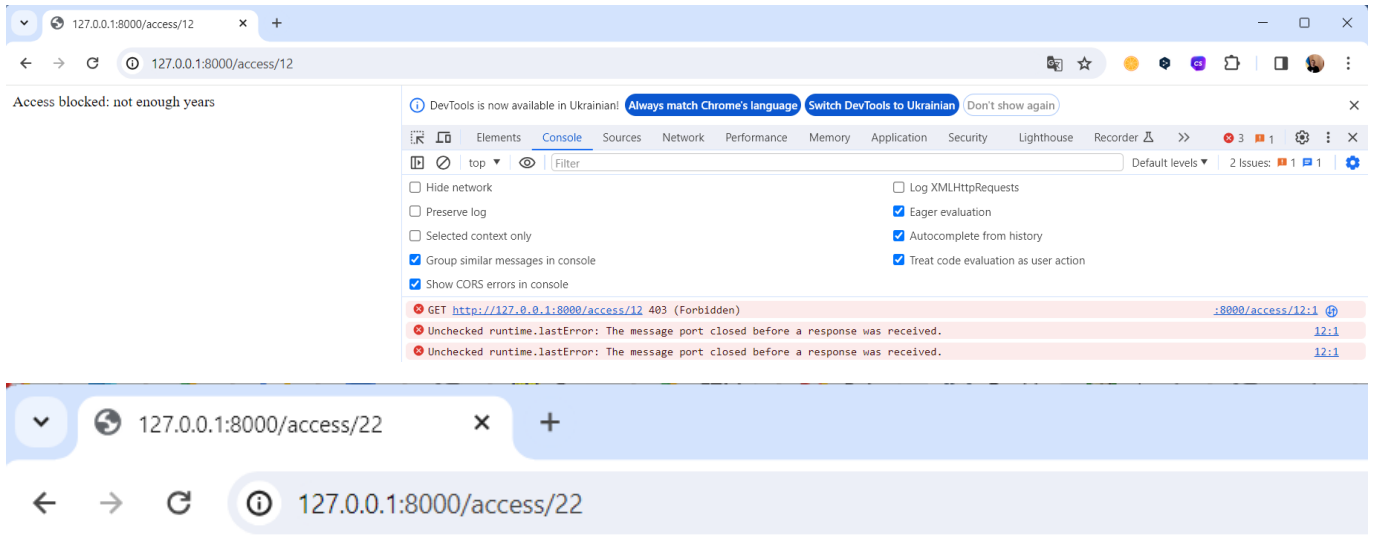


Рисунок 2.32 – Результат обробки некоректного запиту

Аналогічно під час звернення за адресою `"/access"` залежно від значення параметра `age` ми побачимо або повідомлення про помилку, або звичайне повідомлення. Варто зазначити, що статусні повідомлення про помилки також відображаються в консолі запущеного додатка:





Access is allowed

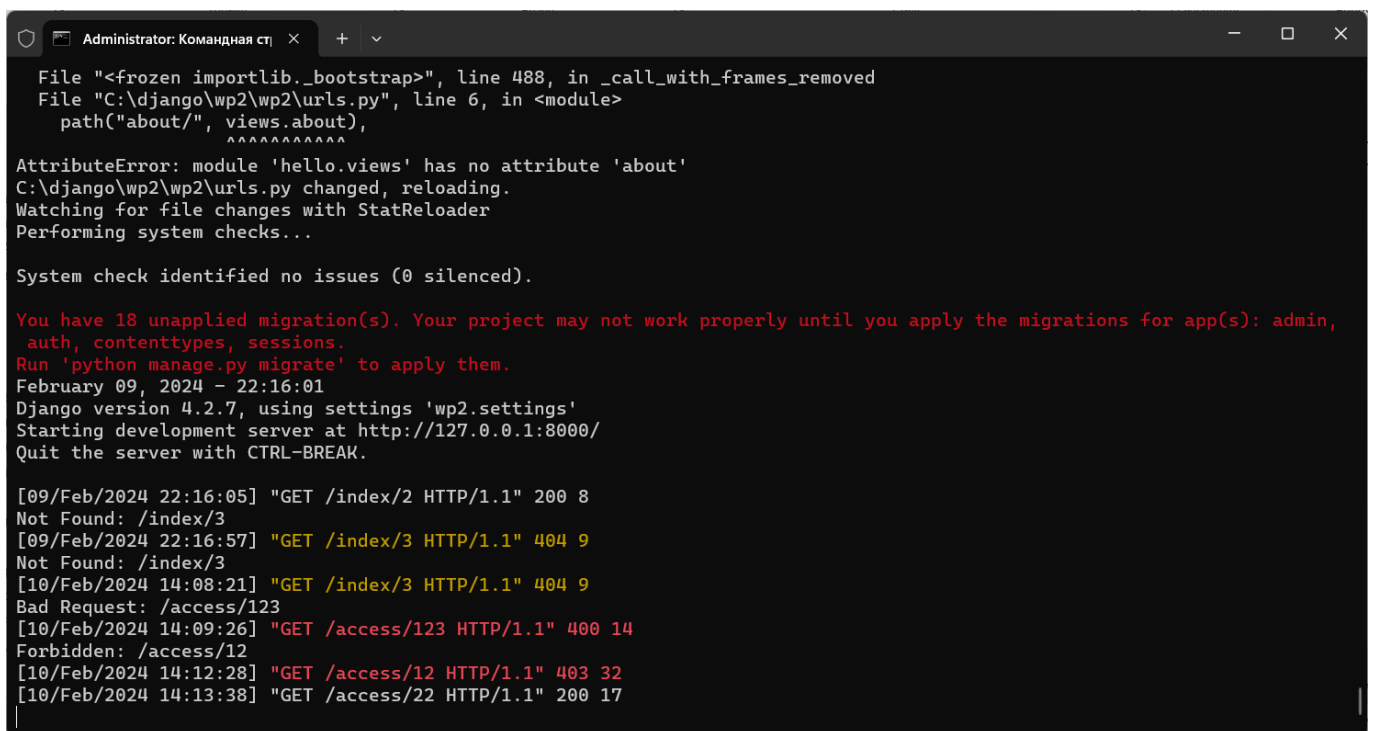


Рисунок 2.32 – Результати відповідей серверу

2.9 Надсилення json

За надсилення клієнту даних у форматі *JSON* у *Django* відповідає спеціальний клас - *JsonResponse*, який, по суті, є підкласом *HttpResponse*. Основна особливість *JsonResponse* полягає в тому, що під час надсилення даних він автоматично встановлює для заголовка *Content-Type* (тип вмісту) значення *application/json*.

2.9.1 Клас *JsonResponse*

Його конструктор приймає низку параметрів:

```
def __init__(data, encoder=DjangoJSONEncoder, safe=True,
             json_dumps_params=None, **kwargs)
```

- *data*: дані, що надсилаються
- *encoder*: серіалізатор, який серіалізує дані, що надсилаються, у формат *JSON*.

За замовчуванням представляє тип *django.core.serializers.json.DjangoJSONEncoder*

- *safe*: представляє булеве значення. Якщо дорівнює *False*, то серіалізації підлягає будь-який об'єкт. Якщо ж дорівнює *True*, то дані, що надсилаються, мають представляти тип *dict* - тобто словник. За замовчуванням дорівнює *True*
- *json_dumps_params*: словник аргументів, який передається у функцію *json.dumps()* для генерації відповіді

Наприклад, надішлемо якісь дані у форматі *JSON*. Для цього визначимо у файлі *views.py* такий код:

```
from django.http import JsonResponse

def index(request):
    return JsonResponse({"name": "Tom", "age": 38})
```

У цьому випадку надсилається словник із двома елементами *name* і *age*.

Відкоригуємо файлі *urls.py* наступним змістом:

```
from django.urls import path
from hello import views

urlpatterns = [
    path("", views.index),
]
```

І при зверненні в браузері до функції *index()* ми побачимо ці дані:

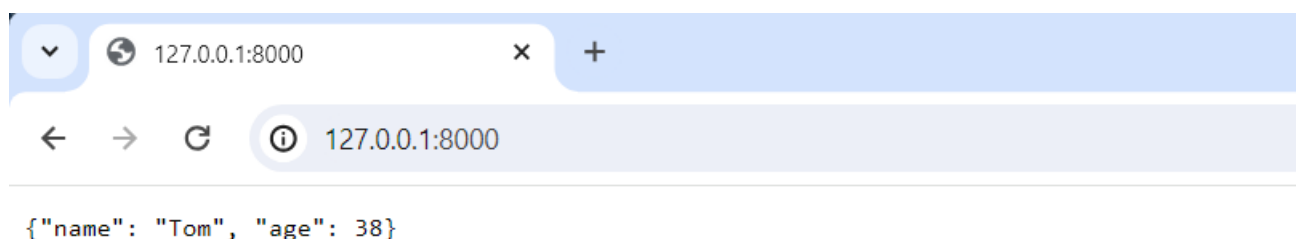


Рисунок 2.33 – Результат передачі json

2.9.2 Серіалізація довільних об'єктів

За замовчуванням `JsonResponse` серіалізує і надсилає тільки словники. Однак що, якщо ми хочемо надіслати об'єкт якогось свого типу? У цьому разі необхідно визначити клас-серіалізатор, який міститиме логіку серіалізації об'єкта в json. Наприклад:

```
from django.http import JsonResponse
from django.core.serializers.json import DjangoJSONEncoder

def index(request):
    bob = Person("Bill", 45)
    return JsonResponse(bob, safe=False, encoder=PersonEncoder)

class Person:

    def __init__(self, name, age):
        self.name = name      # ім'я людини
        self.age = age        # вік людини

class PersonEncoder(DjangoJSONEncoder):
    def default(self, obj):
        if isinstance(obj, Person):
            return {"name": obj.name, "age": obj.age}
            # return obj.__dict__
        return super().default(obj)
```

У цьому випадку *JsonResponse* надсилає об'єкт типу *Person*, у якого визначено два атрибути: *name* і *age*.

Об'єкт *Person* не є словником, тому параметр *safe* має значення *False*. Крім того, параметр *encoder* вказує на серіалізатор, який буде серіалізувати дані в *json*. У цьому випадку це клас *PersonEncoder*.

Клас серіалізатора успадковується від *django.core.serializers.json.DjangoJSONEncoder*. Він реалізує метод *default*, який повертає серіалізований об'єкт. Зокрема, у цьому методі спочатку перевіряємо, чи представляє параметр об'єкт *Person*. І якщо представляє, то повертаємо словник зі значень атрибутів об'єкта:

```
return {"name": obj.name, "age": obj.age}
```

Варто зазначити, що в цьому разі ми можемо просто повернути подання об'єкта у вигляді словника:

```
return obj.__dict__
```

Однак в окремих ситуаціях може знадобитися більш тонке налаштування серіалізації.

Якщо ж об'єкт не представляє тип *Person*, то передаємо його в реалізацію методу *default* батьківського класу.

Результат при зверненні в браузері:

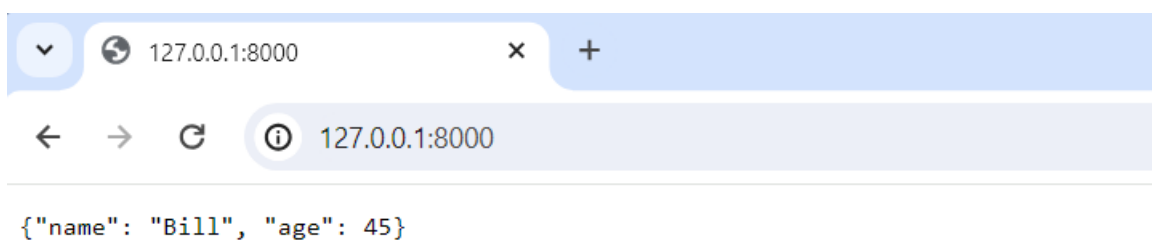


Рисунок 2.34 – Результат серіалізації

2.10 Надсилання та отримання кук

Куки представляють найпростіший спосіб зберегти дані користувача. Куки зберігаються на комп'ютері користувача і можуть встановлюватися як на сервері, так і на клієнті. Оскільки куки надсилаються з кожним запитом на сервер, то їхній максимальний розмір обмежений 4096 байтами. Розглянемо, як надіслати клієнту й отримати від клієнта куки в додатку на *Django*.

2.10.1 Встановлення куки

За встановлення куки та надсилання їх клієнту відповідає метод `set_cookie()` класу *HttpResponse*. Цей метод має таку сигнатуру:

```
set_cookie(key, value='', max_age=None, expires=None, path='/',
domain=None, secure=False, httponly=False, samesite=None)
```

Параметри методу:

- *key*: ключ або ім'я куки
- *value*: значення куки
- *max_age*: максимальний час життя куки в секундах. Це може бути або об'єкт *timedelta*, або число, або значення *None* (обмежує час життя куки поточною сесією браузера, є значенням за замовчуванням).
- *expires*: час і дата, коли закінчується дія куки. Повинен представляти рядок у форматі "Wdy, DD-Mon-YY HH:MM:SS GMT" або об'єкт *datetime.datetime*
- *path*: шлях, для якого встановлюються куки
- *domain*: домен, до якого застосовуються куки
- *secure*: встановлює використовуваний протокол. Так, якщо має значення *True*, то куки надсилатимуться на сервер тільки в запиті за протоколом *https*
- *httponly*: встановлює доступність для скриптів *javascript* на клієнті. Так, значення *httponly=True* запобігає доступу до куки з коду *javascript* на клієнті

– *samesite*: встановлює дозволи на надсилання куки в кросдоменних запитах. Так, значення *samesite*='Strict' і *samesite*='Lax' вказують браузеру не надсилати куки в кросдоменних запитах. Значення за замовчуванням *samesite*='None' дозволяє надсилання куки в кросдоменних запитах

Також клас *HttpResponse* надає ще один метод для встановлення кук:

```
set_signed_cookie(key, value, salt='', max_age=None, expires=None,
path='/', domain=None, secure=False, httponly=False,
samesite=None)
```

Цей метод приймає ті самі параметри, його відмінність у тому, що він застосовує шифрування. Необов'язковий параметр *salt* дає змогу задати сіль для шифрування.

Наприклад, встановимо куки. Нехай у файлі *views.py* є така функція:

```
from django.http import HttpResponse

# встановлення куки
def set(request):
    # отримуємо з рядка запиту ім'я користувача
    username = request.GET.get("username", "Undefined")
    # створюємо об'єкт відповіді
    response = HttpResponse(f"Hello {username}")
    # передаємо його в куки
    response.set_cookie("username", username)
    return response
```

З рядка запиту отримуємо значення параметра "username" і передаємо його в куки за однойменним ключем

Нехай у файлі *urls.py* ця функція викликається в запиті за шляхом *"/set"*:

```
from django.urls import path
from hello import views

urlpatterns = [
    path("set", views.set),
]
```

Звернемося за адресою `"/set"`, передавши через рядок запиту параметр `username`, і сервер встановить куки, а браузер збереже їх. І ми зможемо їх побачити через інструменти розробника:

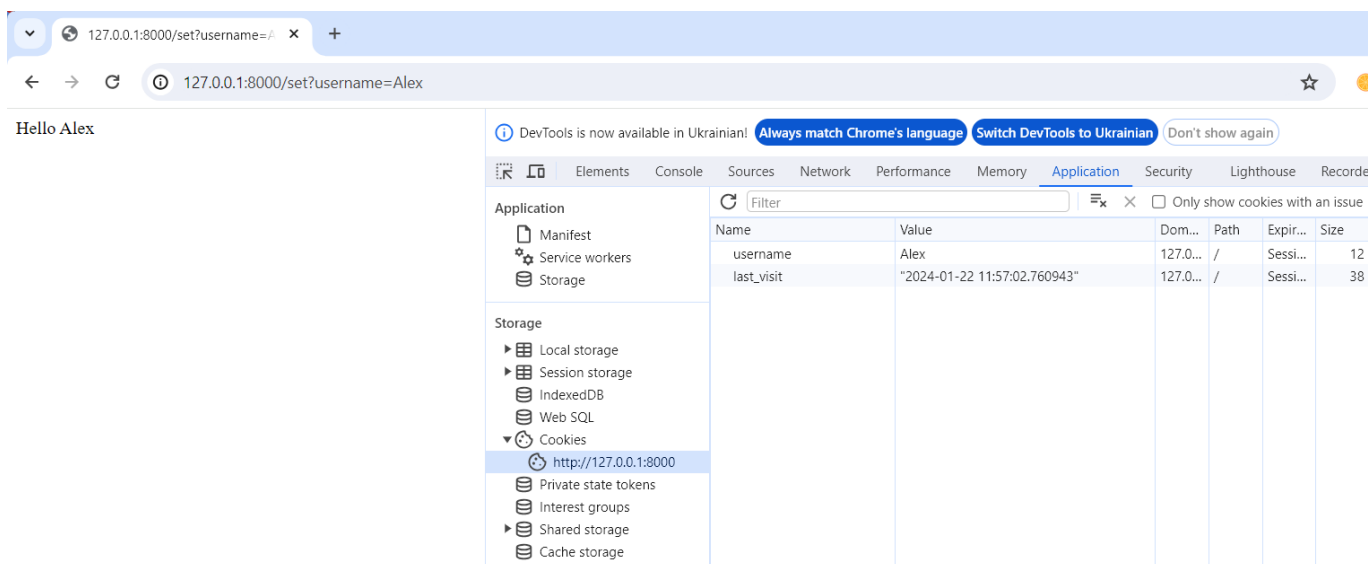


Рисунок 2.35 – Результ встановлення куки

2.10.2 Отримання куки

Якщо куки не шифровані, то для їхнього отримання в об'єкта `HttpRequest` можна використовувати атрибут `COOKIES`, який представляє словник.

Якщо куки шифровані методом `set_signed_cookie`, то для їх отримання в класі `HttpRequest` застосовується метод:

```
get_signed_cookie(key, default=RAISE_ERROR, salt='', max_age=None)
```

Параметри методу:

- `key`: ключ куки, які треба отримати
- `default`: значення за замовчуванням, якщо куки із зазначеним ключем відсутні в запиті
- `max_age`: максимальний час життя куки в секундах
- `salt`: сіль шифрування, повинен мати те ж саме значення, яке передавалося в `set_signed_cookie`

Наприклад, отримаємо раніше встановлені куки за ключем. Визначимо у файлі `views.py` додаткову функцію `get()`:

```
from django.http import HttpResponseRedirect

# встановлення куки
def set(request):
    # отримуємо з рядка запиту ім'я користувача
    username = request.GET.get("username", "Undefined")
    response = HttpResponseRedirect(f"Hello {username}")
    # передаємо його в куки
    response.set_cookie("username", username)
    return response

# отримання куки
def get(request):
    # отримуємо куки з ключем username
    username = request.COOKIES["username"]
    return HttpResponseRedirect(f"Hello {username}")
```

За допомогою виразу `request.COOKIES["username"]` отримуємо куки за ключем `"username"` і передаємо їхнє значення у відповідь клієнту.

Нехай у файлі `urls.py` ця функція викликається в запиті за шляхом `"/get"`:

```
from django.urls import path
from hello import views

urlpatterns = [
    path("set", views.set),
    path("get", views.get),
]
```

Звернемося до за адресою `"/get"` і отримаємо раніше встановлену куку `username`:

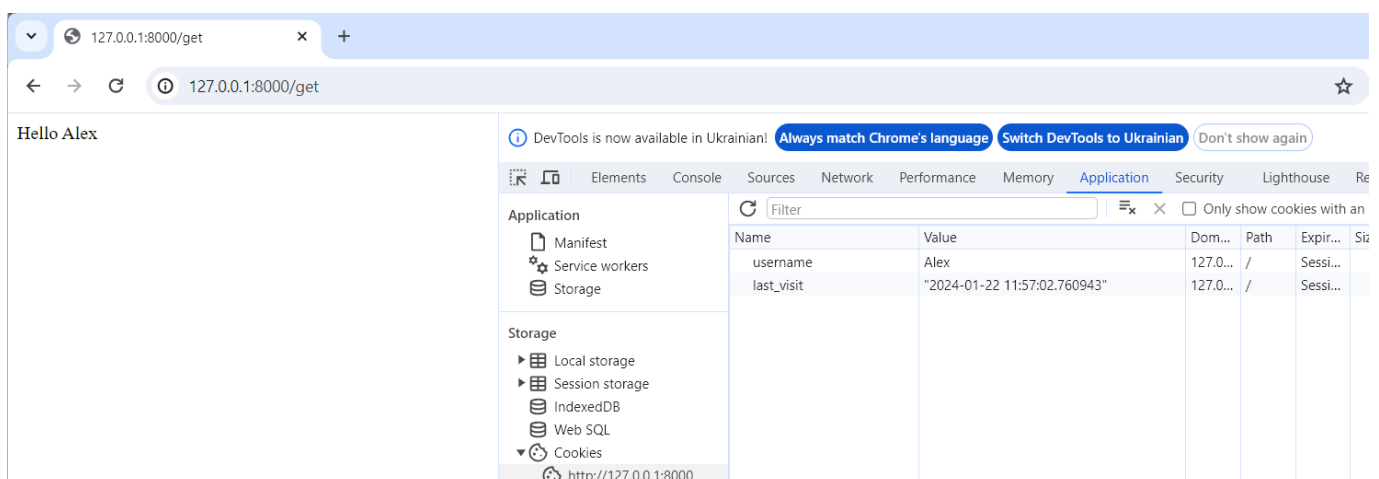


Рисунок 2.36 – Результат отримання куки

3 ШАБЛОНИ

3.1 Створення та використання шаблонів

Шаблони (*templates*) відповідають за формування зовнішнього вигляду програми. Вони надають спеціальний синтаксис, який дає змогу впроваджувати дані в код *HTML*.

Припустимо, у нас є проєкт *wp2*, і в ньому визначено один застосунок - *hello*:

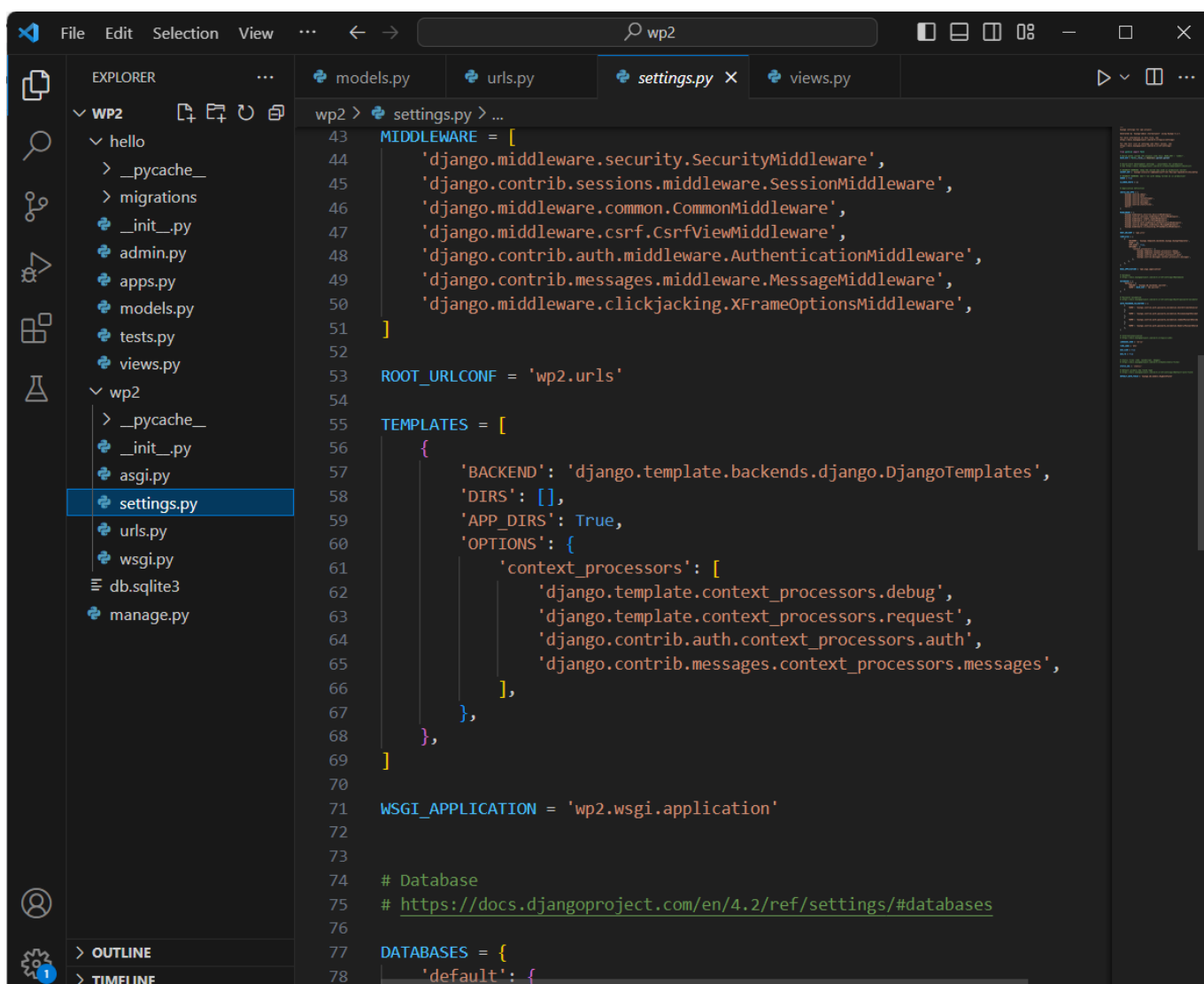


Рисунок 3.1 – Структура проєкту

Налаштування функціональності шаблонів у проєкті *Django* здійснюється у файлі `settings.py`. за допомогою змінної *TEMPLATES*. Так, за замовчуванням змінна *TEMPLATES* у файлі `settings.py` має таке визначення:

```
TEMPLATES = [
    {
        'BACKEND': 'django.template.backends.django.DjangoTemplates',
        'DIRS': [],
        'APP_DIRS': True,
        'OPTIONS': {
            'context_processors': [
                'django.template.context_processors.debug',
                'django.template.context_processors.request',
                'django.contrib.auth.context_processors.auth',
                'django.contrib.messages.context_processors.messages',
            ],
        },
    },
]
```

Ця змінна приймає список конфігурацій для кожного движка шаблонів. За замовчуванням визначено одну конфігурацію, яка має такі параметри:

- *BACKEND*: движок шаблонів. За замовчуванням застосовується вбудований движок *django.template.backends.django.DjangoTemplates*
- *DIRS*: визначає список каталогів, де рушій шаблонів буде шукати файли шаблонів. За замовчуванням порожній список
- *APP_DIRS*: вказує, чи буде рушій шаблонів шукати шаблони всередині папок додатків у папці *templates*.
- *OPTIONS*: визначає додатковий список параметрів

Отже, у конфігурації за замовчуванням параметр *APP_DIRS* має значення *True*, а це значить, що рушій шаблонів буде також шукати потрібні файли шаблонів у папці застосунку в каталозі *templates*. Тобто за замовчуванням ми вже маємо налаштовану конфігурацію, готову до використання шаблонів. Тепер визначимо самі шаблони.

Додамо в папку додатка каталог *templates*. А в ньому визначимо файл *index.html*:

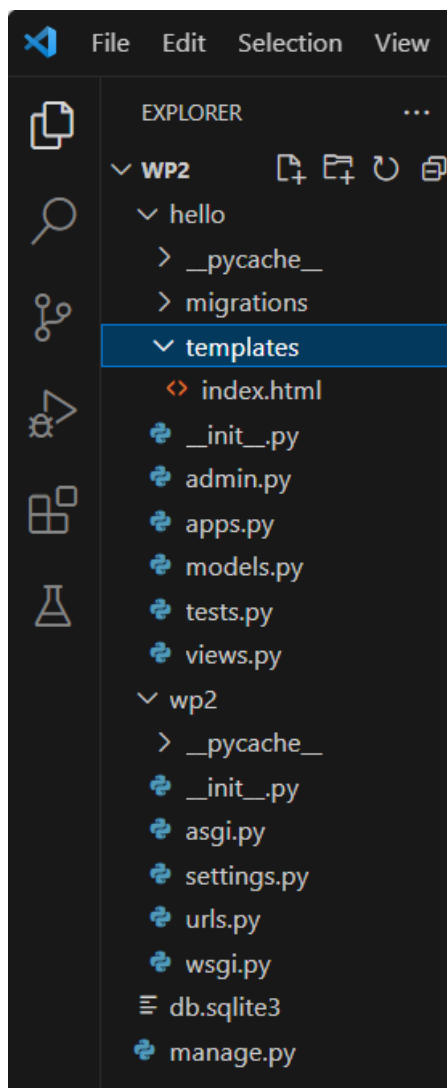


Рисунок 3.2 – Результат додавання нової теки та файлу

Далі у файлі *index.html* визначимо такий код:

```
<!DOCTYPE html>
<html>
<head>
  <title>Django on WebPython</title>
  <meta charset="utf-8" />
</head>
<body>
  <h1>Hello WebPython</h1>
</body>
</html>
```

3.1.1 Функція `render()`

По суті це звичайна веб-сторінка, яка містить код *html*. Тепер використаємо цю сторінку для надсилання відповіді користувачеві. І для цього перейдемо в додатку *hello* до файлу *views.py*, який визначає функції для обробки запиту. Змінимо цей файл таким чином:

```
from django.shortcuts import render

def index(request):
    return render(request, "index.html")
```

З модуля *django.shortcuts* імпортується функція *render()*.

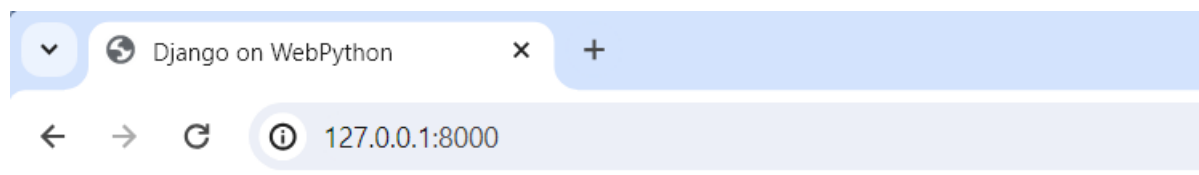
Функція *index()* викликає функцію *render()*, якій передаються об'єкт запиту *request* і шлях до файлу шаблону в рамках папки *templates* - "*index.html*".

У файлі *urls.py* проєкту пропишемо зіставлення функції *index()* із запитом до кореня веб-додатка:

```
from django.urls import path
from hello import views

urlpatterns = [
    path("", views.index),
]
```

І запусимо проєкт на виконання і перейдемо до застосунку в браузері (якщо проєкт запущено, то його треба перезапустити):



Hello WebPython

Рисунок 3.3 – Результ відображення шаблону

Подібним чином можна вказати й інші шаблони. Наприклад, у папку *templates* додамо ще дві сторінки: *about.html* і *contact.html*

І також у файлі *views.py* визначимо функції, які використовують ці шаблони:

```
from django.shortcuts import render

def index(request):
    return render(request, "index.html")

def about(request):
    return render(request, "about.html")

def contact(request):
    return render(request, "contact.html")
```

А у файлі *urls.py* зв'яжемо функції з маршрутами:

```
from django.urls import path
from hello import views

urlpatterns = [
    path("", views.index),
    path("about/", views.about),
    path("contact/", views.contact),
]
```

3.1.2 Клас `TemplateResponse`

Вище для генерації шаблону застосовувалася функція *render()*, яка є найбільш поширеним варіантом. Однак також ми можемо використовувати клас *TemplateResponse*:

```
from django.template.response import TemplateResponse

def index(request):
    return TemplateResponse(request, "index.html")
```

3.2 Передання даних у шаблони

Однією з переваг шаблонів є те, що ми можемо передати в них динамічно з подань різні дані. Для виведення даних у шаблоні можуть використовуватися різні способи.

3.2.1 Передавання простих даних

Для виведення найпростіших даних застосовується подвійна пара фігурних дужок:

```
{{ назва_об'єкта }}
```

Наприклад, нехай у проєкті в нас є папка *templates*, у якій міститься шаблон *index.html*.

Визначимо у файлі *index.html* такий код:

```
<!DOCTYPE html>
<html>
<head>
  <meta charset="utf-8" />
  <title>Django on WebPython</title>
</head>
<body>
  <h2>{{ header }}</h2>
  <p>{{ message }}</p>
</body>
</html>
```

Тут використовується дві змінні: *message* і *header*. Вони передаватимуться з подання.

Щоб із функції-представлення передати дані в шаблон застосовується третій параметр функції *render()*, який ще називається *context* і який представляє словник. Наприклад, змінимо файл *views.py* таким чином:

```
from django.shortcuts import render

def index(request):
    data = {"header": "Hello Django", "message": "Welcome to WebPython"}
    return render(request, "index.html", context=data)
```

У шаблоні використовуються дві змінні, відповідно словник, який передається у функцію `render()` через параметр `context`, тепер містить два значення з ключами *header* і *message*.

У результаті під час звернення до кореня веб-додатка ми побачимо такий висновок у браузері:

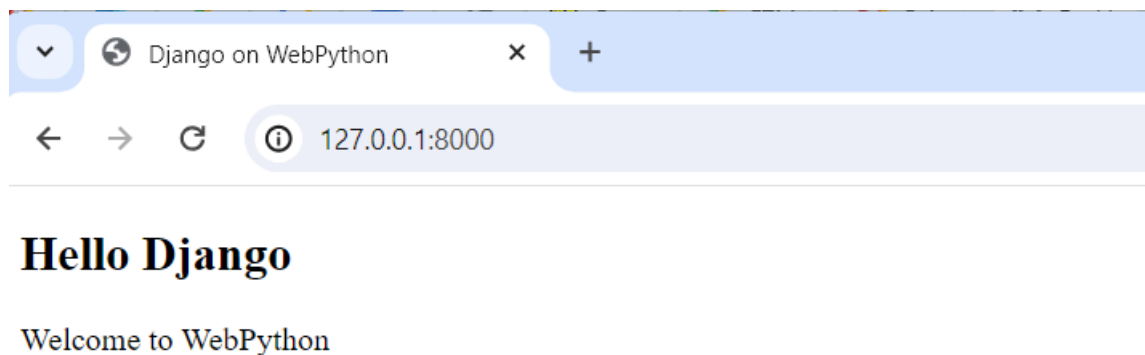


Рисунок 3.4 – Результат передавання даних у шаблон

3.2.2 Передання складних даних

Розглянемо передавання складніших даних. Припустимо, у поданні передаються такі дані:

```
from django.shortcuts import render

def index(request):
    header = "Дані користувача"           # звичайна змінна
    langs = ["Python", "Java", "C#"]      # список
    user = {"name": "Alex", "age": 25}     # словник
    address = ("Політехнічна", 512, 1024) # кортеж

    data = {"header": header, "langs": langs, "user": user, "address": address}
    return render(request, "index.html", context=data)
```

Як третій параметр у функцію *render()* нам треба передати словник, тому всі дані обертаються в словник і в такому вигляді передаються в шаблон.

У цьому разі шаблон міг би мати, наприклад, такий вигляд:

```
<!DOCTYPE html>
<html>
<head>
  <meta charset="utf-8" />
  <title>Django on WebPytnon</title>
</head>
<body>
  <h1>{{ header }}</h1>
  <p>Им'я: {{ user.name }} Вік: {{user.age}}</p>
  <p>Адреса: вул. {{address.0}}, буд. {{address.1}}, кв. {{address.2}}</p>
  <p>Навички: {{langs.0}}, {{langs.1}}</p>
</body>
</html>
```

Оскільки об'єкти *langs* і *address* представляють відповідно масив і кортеж, то ми можемо звернутися до їхніх елементів через індекси, як ми б працювали з ними в коді на *Python*, наприклад, перший елемент кортежу *address*: *address.0*.

Подібним чином, оскільки об'єкт *user* представляє словник, то ми можемо звернутися до його елементів за ключами *name* і *age*: *{{ user.name }}* *{{user.age}}*.

У підсумку ми отримаємо такий висновок у веб-браузері:



Дані користувача

Ім'я: Alex Вік: 25

Адреса: вул. Політехнічна, буд. 512, кв. 1024

Навички: Python, Java, C#

Рисунок 3.5 – Результат передачі складних даних

3.2.3 Клас `TemplateResponse`

Якщо для генерації шаблону застосовується клас `TemplateResponse`, то в його конструктор також через третій параметр можна передати дані для шаблону:

```
from django.template.response import TemplateResponse

def index(request):
    header = "Дані користувача"           # звичайна змінна
    langs = ["Python", "Java", "C#"]      # список
    user = {"name" : "Steave", "age" : 45} # словик
    address = ("Політехнічна", 256, 512)  # кортеж

    data = {"header": header, "langs": langs, "user": user, "address": address}
    return TemplateResponse(request, "index.html", data)
```

3.2.4 Передача об'єктів класів

Подібним чином можна передавати в шаблони об'єкти своїх класів. Наприклад, визначення функції-представлення:

```
from django.shortcuts import render

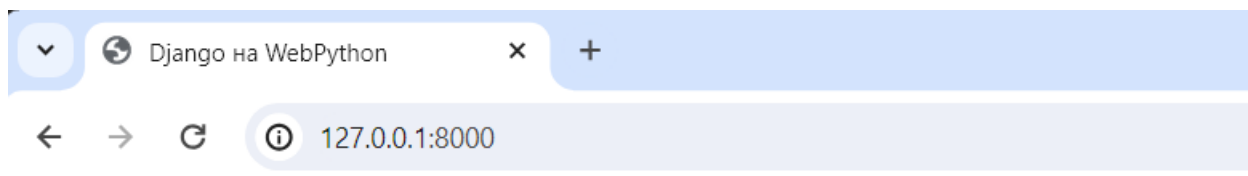
def index(request):
    return render(request, "index.html", context = {"person": Person("Dallas")})

class Person:

    def __init__(self, name):
        self.name = name      # ім'я
```

Тут визначається клас `Person`, у конструкторі якого передається деяке значення для атрибута `name`. У функції `index()` у шаблон передається об'єкт із ключем `"person"`.

У шаблоні `index.html` ми можемо звертатися до функціональності об'єкта, наприклад, до його атрибута `name`:



Person Dallas

Рисунок 3.6 – Результат передачі об’єктів класу

3.3 Вбудовані теги шаблонів

Django надає можливість використовувати в шаблонах низку спеціальних тегів, які спрощують виведення деяких даних. Розглянемо деякі найбільш використовувані теги.

3.3.1 autoescape

Тег *autoescape* дає змогу автоматично екранувати низку символів *HTML* і в такий спосіб зробити виведення на сторінку безпечнішим. Зокрема, здійснюються такі заміни:

- < замінюється на <
- > замінюється на >
- ' (одинарна лапка) замінюється на '
- " (подвійна лапка) замінюється на "
- & замінюється на &

Наприклад, нехай у нас буде визначено такий шаблон *index.html*:

```
<!DOCTYPE html>
<html>
<head>
    <meta charset="utf-8" />
    <title>Django on WebPython</title>
</head>
<body>
    {{ body }}

    {% autoescape off %}
        {{ body }}
    {% endautoescape %}
</body>
</html>
```

Тут у шаблоні два рази виводиться вміст змінної *body*. Однак спочатку вона виводиться сама по собі, у другому випадку її виведення поміщається між тегами *{% autoescape %}* і *{% endautoescape %}*. Тег як параметр приймає одне зі значень: *on* (екранування ввімкнене) і *off* (екранування вимкнене). У цьому випадку тегу передається значення *off*, а це означає, що всередині блоку тега *autoescape* екранування вимкнено.

Визначимо таку функцію-представлення, яка використовує цей шаблон і передає в нього деякі дані:

```
from django.shortcuts import render

def index(request):
    return render(request, "index.html", context = {"body": "<h1>Hello
WebPython!</h1>"})
```

Тут у шаблон передається деякий рядок із кодом *HTML*. За замовчуванням шаблони застосовують екранування. Тому на веб-сторінці ми побачимо код *html* у текстовому вигляді, а в другому випадку передані дані будуть інтпретовані безпосередньо як код *html*:

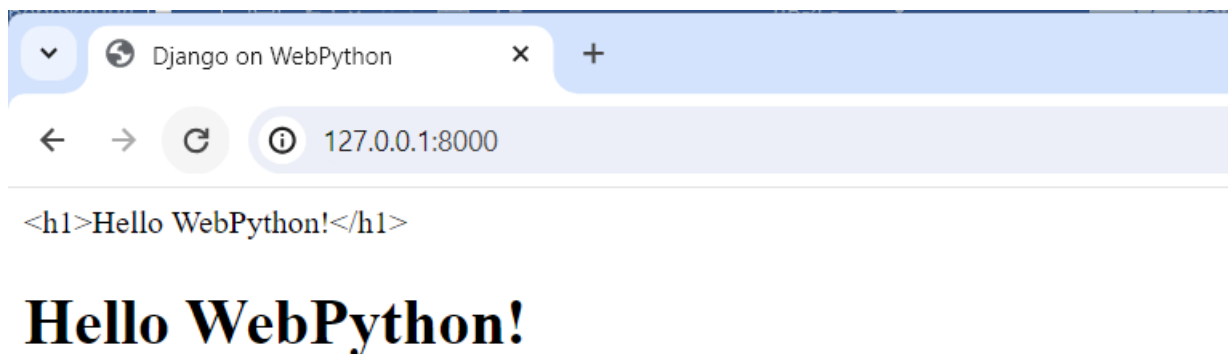


Рисунок 3.7 – Результат дії тегу *autoescape*

3.3.2 Коментарі

Для визначення коментарів у шаблоні застосовується тег *comment*: усе, що поміщається між тегами `{% comment %}` і `{% endcomment %}`, ігнорується під час генерації *html*-сторінки.

3.3.3 if..else

Тег *if* оформляється у вигляді блоку:

```
{% if умова %}
    вміст блоку
{% endif %}
```

Цей тег оцінює деяку умову, яка має повертати *True* або *False*: якщо повертається *True*, то виводиться вміст блоку `{% if %}`.

Наприклад, нехай у поданні передаються в шаблон деякі значення:

```
from django.shortcuts import render

def index(request):
    data = {"n" : 5}
    return render(request, "index.html", context=data)
```

У шаблоні залежно від значення змінної *n* ми можемо виводити певний контент:

```
{% if n > 0 %}
    <p>Число позитивне</p>
{% endif %}
```

Тобто якщо *n* більше за 0, то буде виводитися, що число позитивне. Якщо *n* менше або дорівнює 0, нічого не буде виводитися.

За допомогою додаткового тега `{% else %}` можна вивести контент у разі, якщо умова після *if* дорівнює *False*:

```
{% if n > 0 %}
    <p>Число позитивне</p>
{% else %}
    <p>Число від'ємне або дорівнює нулю</p>
{% endif %}
```

За допомогою тега `{% elif %}` можна перевірити додаткові умови, якщо умова в *if* дорівнює *False*:

```
{% if n > 0 %}
    <p>Число позитивне</p>
{% elif n < 0 %}
    <p>Число від'ємне</p>
{% else %}
    <p>Число дорівнює нулю</p>
{% endif %}
```

Варто зазначити, що в умові, яку передають тегу *if*, можна застосовувати низку операторів, які повертають *True* або *False*: `==`, `!=`, `<`, `>`, `<=`, `>=`, `in`, `not in`, `is`, `is not`, `and`, `or`, `not`.

```
{% if a > 0 and b > 0 %}
    <p>{{ a }}</p>
{% else %}
    <p>{{ b }}</p>
{% endif %}
```

3.3.4 Цикли

Тег *for* дозволяє створювати цикли. Цей тег приймає як параметр деяку колекцію і пробігається по цій колекції, обробляючи кожен її елемент.

```
{% for element in collection %}
    дії з element
{% endfor %}
```

Наприклад, нехай із подання в шаблон передається деякий список:

```
from django.shortcuts import render

def index(request):
    langs = ["Python", "JavaScript", "Java", "C#", "C++"]
    return render(request, "index.html", context={"langs": langs})

def about(request):
    return render(request, "about.html")

def contact(request):
    return render(request, "contact.html")
```

Виведемо елементи списку *langs* у шаблоні за допомогою тега *for*:

```
<!DOCTYPE html>
<html>
<head>
  <meta charset="utf-8" />
  <title>Django on WebPython</title>
</head>
<body>

  <ul>
    {% for lan in langs %}
    <li>{{ lan }}</li>
    {% endfor %}
  </ul>

</body>
</html>
```

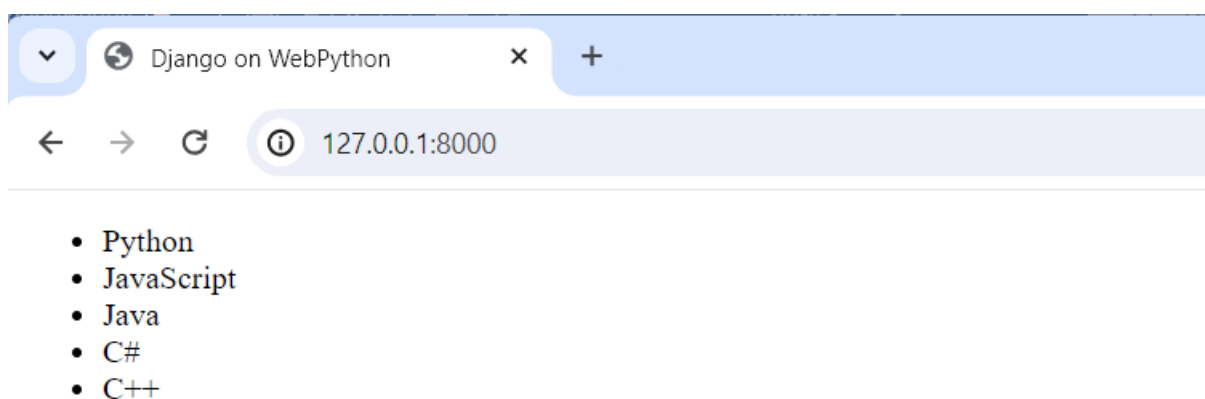


Рисунок 3.8 – Результат застосування циклу в шаблоні

Цілком можливо, що передана з подання в шаблон колекція виявиться порожньою. На цей випадок ми можемо використовувати тег *{% empty %}*:

```
<ul>
  {% for lan in langs %}
  <li>{{ lan }}</li>
  {% empty %}
  <li>Langs array is empty</li>
  {% endfor %}
</ul>
```

Подібним чином можна перебирати в шаблоні інші колекції, наприклад, словники:

```
<!DOCTYPE html>
<html>
<head>
    <meta charset="utf-8" />
    <title>Django on WebPython</title>
</head>
<body>
    {% for key, value in data.items %}
        <p>{{ key }}: {{ value }}</p>
    {% endfor %}
</body>
</html>
```

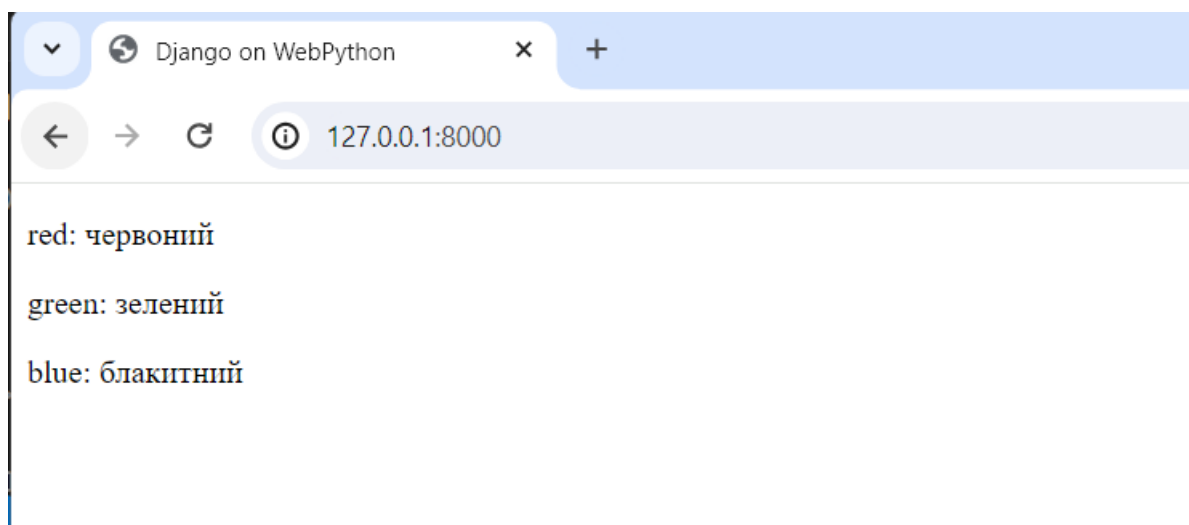
У цьому випадку перебирається словник *data*, який передається з подання:

```
from django.shortcuts import render

def index(request):
    data = {"red": "червоний", "green": "зелений", "blue": "блакитний"}
    return render(request, "index.html", context={"data": data})

def about(request):
    return render(request, "about.html")

def contact(request):
    return render(request, "contact.html")
```



Рисинок 3.9 – Результат перебирання словника

3.3.5 Визначення змінних

Тег `{% with %}` дає змогу визначити змінну і використовувати її всередині блоку тега. Наприклад:

```
{% with name="Steve" age=30 %}
  <div>
    <p>Name: {{ name }}</p>
    <p>Age: {{ age }}</p>
  </div>
{% endwith %}
```

У цьому випадку визначено дві змінні: `name` і `age`, які ми можемо використовувати всередині блоку `{% with %}` ... `{% endwith %}`. Однак поза цим блоком ці змінні використовуватися не можуть.

3.3.6 Дати

Тег `{% now "формат_даних" %}` дає змогу вивести системний час. Як параметр тегу `now` передається формат даних, який вказує, як форматувати час і дату.

```
<!DOCTYPE html>
<html>
<head>
  <meta charset="utf-8" />
  <title>Django on WebPython</title>
</head>
<body>
  <p>{% now "Y" %}</p>
  <p>{% now "F j Y" %}</p>
  <p>{% now "N, j, Y" %}</p>
  <p>{% now "N j, Y, P" %}</p>
</body>
</html>
```

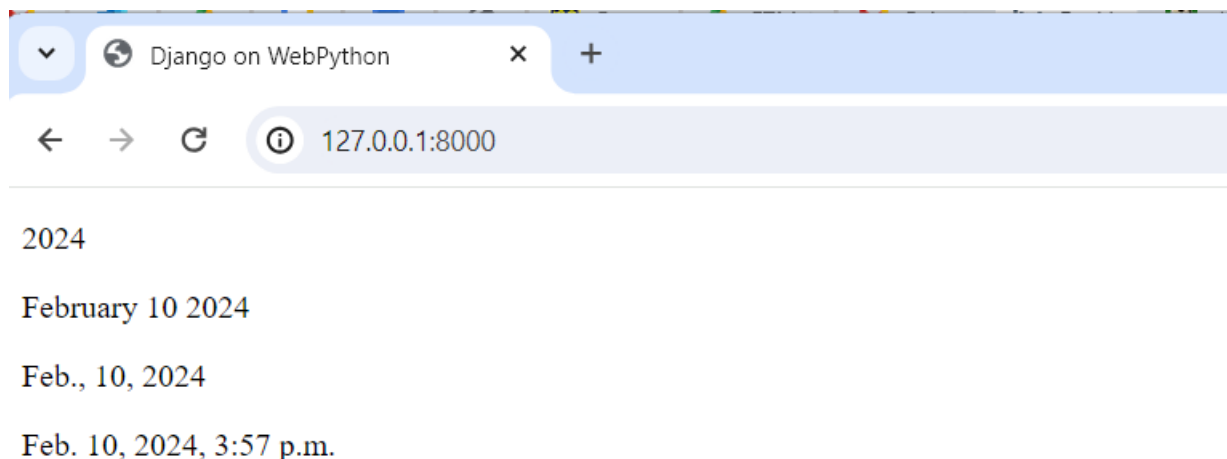


Рисунок 3.10 – Результат виведення даних у різних форматах

Символ "Y" передає рік у вигляді чотирьох цифр, "y" - бере з року останні дві цифри. "F" - повна назва місяця, "N" - скорочена назва місяця. "j" - день місяця. "P" - час. Усі можливі формати для виведення дати і часу можна подивитися в документації

Варто зазначити, що як час *Django* бере значення змінної з файлу *settings.py*, яка за замовчуванням має таке визначення:

```
TIME_ZONE = 'UTC'
```

3.4 Фільтри шаблонів

На додаток до тегів *Django* також підтримує фільтри, які дають змогу здійснити деяку просту обробку значень усередині шаблонів. Повний список фільтрів можна знайти в документації. Тут же розглянемо тільки деякі найбільш використовувані.

3.4.1 add

Фільтр *add* додає до одного значення інше.

```
значення1 | add: значення2
```

Наприклад:

```
<!DOCTYPE html>
<html>
<head>
  <meta charset="utf-8" />
  <title>Django on WebPython</title>
</head>
<body>
  <div>
{% with message="Hello, " %}
  <h2>{{ message|add:" Django" }}</h2>
{% endwith %}
  </div>
</body>
</html>
```

У цьому разі визначається змінна *message* (це також могли б бути дані, що передаються з подання). Потім під час виведення значення цієї змінної до неї додається рядок "*Django*"

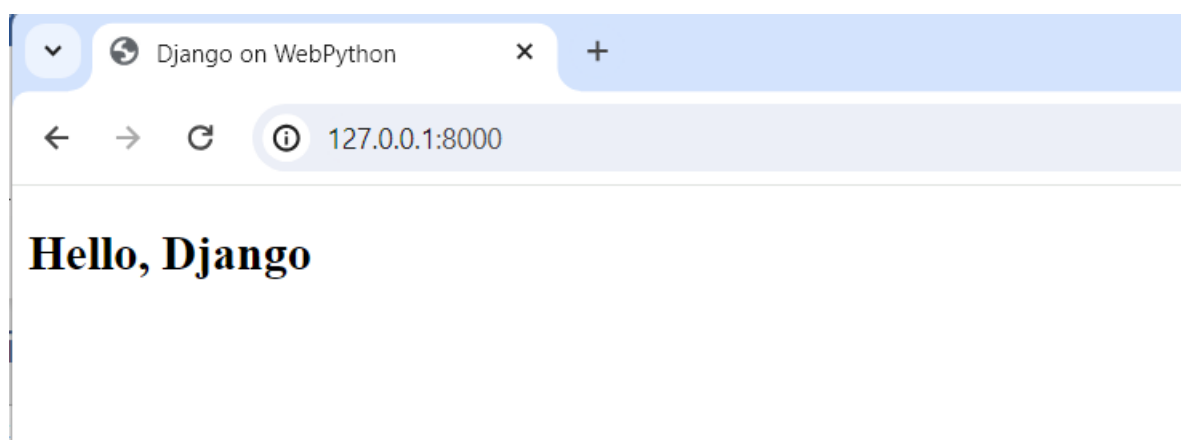


Рисунок 3.11 – Результат використання фільтра `add`

3.4.2 `capfirst`

Фільтр *capfirst* робить першу букву великою:

```
{{ message|capfirst }}
```

Наприклад, якщо *message* дорівнює "django", то після застосування фільтра на веб-сторінці буде виведено "Django".

3.4.3 cut

Фільтр *cut* видаляє з рядка певний підрядок:

```
{% with message="Я був вдома" %}
    {{ message|cut: "був" }}
{% endwith %}
```

У цьому випадку з рядка "Я був удома" видаляється підрядок "був", тому шаблон виведе "Я вдома"

3.4.4 Перевірка значення і значення за замовчуванням

Фільтр *default* перевіряє значення, і якщо воно дорівнює *False*, то повертає деяке значення за замовчуванням.

```
{% with isEnabled=False %}
    {{ isEnabled|default: "unabled" }}
{% endwith %}
```

У цьому випадку змінна *isEnabled* дорівнює *False*, тому шаблон виведе "unabled"

При цьому сама змінна необов'язково повинна зберігати саме *True* або *False*, це може бути будь-який вираз, який можна привести до значення *True/False*. Наприклад:

```
{% with name="" %}
    {{ name|default: "Undefined" }}
{% endwith %}
```

У цьому випадку змінна *name* представляє порожній рядок, тому під час застосування фільтра отримаємо *False*, а фільтр поверне рядок "*Undefined*".

Схожу дію має фільтр *default_if_none* - він повертає значення за замовчуванням, якщо передане йому значення дорівнює *None*:

```
{% with user=None %}
    {{ user|default_if_none: "Undefined" }}
{% endwith %}
```

3.4.5 floatformat

floatformat форматує числа з плаваючою крапкою.

Якщо фільтру не передається аргумент, то число округляється до одного знака після коми:

Значення	Шаблон	Результат
34.23234	{{ value floatformat }}	34.2
34.00000	{{ value floatformat }}	34
34.26000	{{ value floatformat }}	34.3

Фільтру можна передати числовий аргумент, який вказує, до скількох знаків після коми треба округляти:

Значення	Шаблон	Результат
34.23234	{{ value floatformat:3 }}	34.232
34.00000	{{ value floatformat:3 }}	34.000
34.26000	{{ value floatformat:3 }}	34.260

Якщо аргумент - від'ємне число, то число також округлюється до певної кількості знаків після коми. Однак якщо дробова частина складається з одних нулів, то нулі при цьому відсікаються:

Значення	Шаблон	Результат
34.23234	{{ value floatformat:"-3" }}	34.232
34.00000	{{ value floatformat:"-3" }}	34
34.26000	{{ value floatformat:"-3" }}	34.260

Аргумент 0 дозволяє округлити число до найближчого цілого:

Значення	Шаблон	Результат
34.23234	{{ value floatformat:"0" }}	34
34.00000	{{ value floatformat:"0" }}	34
39.56000	{{ value floatformat:"0" }}	40

Якщо аргумент має суфікс *g*, то застосовується групування з використанням роздільника розрядів відповідно до поточної локалі:

Значення	Шаблон	Результат
34232.34	{{ value floatformat:"2g" }}	34,232.34
34232.06	{{ value floatformat:"g" }}	34,232.1
34232.00	{{ value floatformat:"-3g" }}	34,232

Під час виведення застосовується поточна локаль проєкту. Наприклад, якщо ми відкриємо файл *settings.py*, то можемо там знайти змінну *LANGUAGE_CODE*, яка визначає локаль:

```
LANGUAGE_CODE = 'en-us'
```

За замовчуванням застосовується американський варіант англійської мови і відповідні їм налаштування локалі. Відповідно роздільником між цілою і дробовою частиною буде крапка, а між розрядами - кома. Але, наприклад, змінимо на українську культуру:

```
LANGUAGE_CODE = 'uk'
```

Тепер для коду:

```
<!DOCTYPE html>
<html>
<head>
  <meta charset="utf-8" />
  <title>Django on WebPython</title>
</head>
<body>
  <div>
    {% with value=34232.34 %}
    {{ value|floatformat:"2g" }}
    {% endwith %}
  </div>
</body>
</html>
```

Отримаємо:

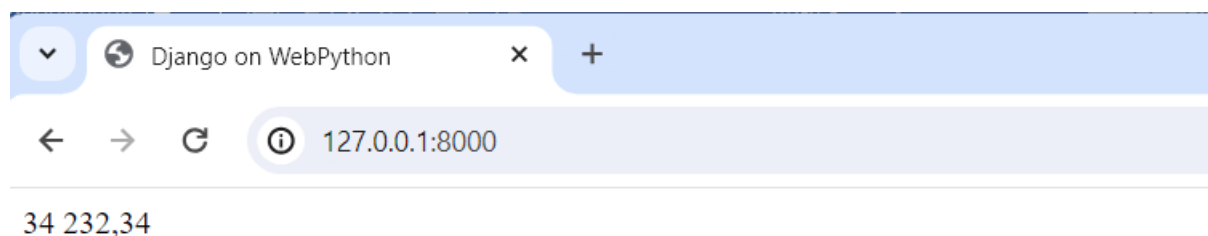


Рисунок 3.12 – Результат зміни локалі

Якщо ж необхідно відключити локалізацію, аргументу додається суфікс *u*:

```
{{ value|floatformat:"2u" }}
```

3.4.6 Форматування дат

Фільтр *date* застосовується до об'єкта *datetime* і як аргумент отримує такі специфікатори дати і часу:

- *d*: день місяця у вигляді двох цифр (одна цифра доповнюється нулем зліва).

Діапазон значень від '01' до '31'

- *j*: день місяця без доповнення нулем. Діапазон значень від '1' до '31'
- *D*: день тижня в текстовому вигляді у вигляді трьох символів, наприклад, 'Fri'

- I: день тижня в повному вигляді, наприклад, 'Friday'
- S: повертає англійський суфікс для порядкових числівників для дня місяця.

Можливі значення: 'st', 'nd', 'rd' і 'th'

- w: номер дня тижня. Значення в діапазоні від '0' (неділя) до '6' (субота)
- z: номер дня року. Діапазон значень від 1 до 366
- W: номер тижня року (першим днем тижня вважається понеділок). Значення від 1 до 53

- m: номер місяця у вигляді двох цифр. Діапазон значень від '01' до '12'
- n: номер місяця у вигляді двох цифр без передування нулем. Діапазон значень від '1' до '12'

- M: текстове значення місяця у вигляді трьох символів. Наприклад, 'Jan'
- b: текстове значення місяця у вигляді трьох символів у нижньому регістрі.

Наприклад, 'jan'

- E: локалізована назва місяця. Наприклад, 'серпня' ("серпень" у родовому відмінку)

- F: повна назва місяця. Наприклад, 'January'
- N: аббревіатура місяця. Наприклад, 'Jan.', 'Feb.', 'March', 'May'
- t: число днів у поточному місяці. Значення від 28 до 31
- u: двочисловий код року з доповненням нулем. Діапазон значень від '00' до '99'

- Y: чотиричисловий код року з доповненням нулем. Діапазон значень від '0001' до '9999'

- L: чи є рік високосним. Значення: True(є) і False (не є)
- o: ISO-8601 week-numbering year,
- g: година в 12-годинному форматі без доповнення нулем. Діапазон значень від '1' до '12'

- G: година у 24-годинному форматі без доповнення нулем. Діапазон значень від '0' до '23'

- h: година в 12-годинному форматі з доповненням нулем. Діапазон значень від '01' до '12'

- H: година в 24-годинному форматі з доповненням нулем. Діапазон значень від '00' до '23'
 - i: хвилини від '00' до '59'
 - s: секунди від '00' до '59'
 - u: мікросекунди від 000000 до 999999
 - a: 'a.m.' або 'p.m.'
 - A: 'AM' або 'PM'
 - f: час у 12-годинному форматі з хвилинами. Наприклад, '1:30'
 - P: час у 12-годинному форматі з хвилинами та "a.m."/"p.m.". Наприклад, '12:30 p.m.'
 - e: часова зона, наприклад, '', 'GMT', '-500', 'US/Eastern', etc.
 - O: зміщення в годинах відносно гринвіча. Наприклад, '+0200'
 - T: часова зона поточного комп'ютера. 'EST', 'MDT'
 - c: дата і час у форматі ISO 8601. Наприклад, 2008-01-02T10:30:00.000123+02:00 або 2008-01-02T10:30:00.000123
 - r: дата і час у форматі RFC 5322
- Також можна передати низку попередньо встановлених констант:
- *DATE_FORMAT*
 - *DATETIME_FORMAT*
 - *SHORT_DATE_FORMAT*
 - *SHORT_DATETIME_FORMAT*

Наприклад, у поданні в шаблон передається деяка дата:

```
from datetime import datetime
from django.shortcuts import render

def index(request):
    return render(request, "index.html", context={"my_date": datetime.now()})

def about(request):
    return render(request, "about.html")

def contact(request):
    return render(request, "contact.html")
```

Виведемо цю дату в шаблоні:

```
<!DOCTYPE html>
<html>
<head>
  <meta charset="utf-8" />
  <title>Django on WebPython</title>
</head>
<body>
  <div>
<h2>{{ my_date|date:"d.m.Y" }}</h2>
<h2>{{ my_date|date:"H:i" }}</h2>
<h2>{{ my_date|date:"c" }}</h2>
<h2>{{ my_date|date:"SHORT_DATE_FORMAT" }}</h2>
</body>
</html>
```

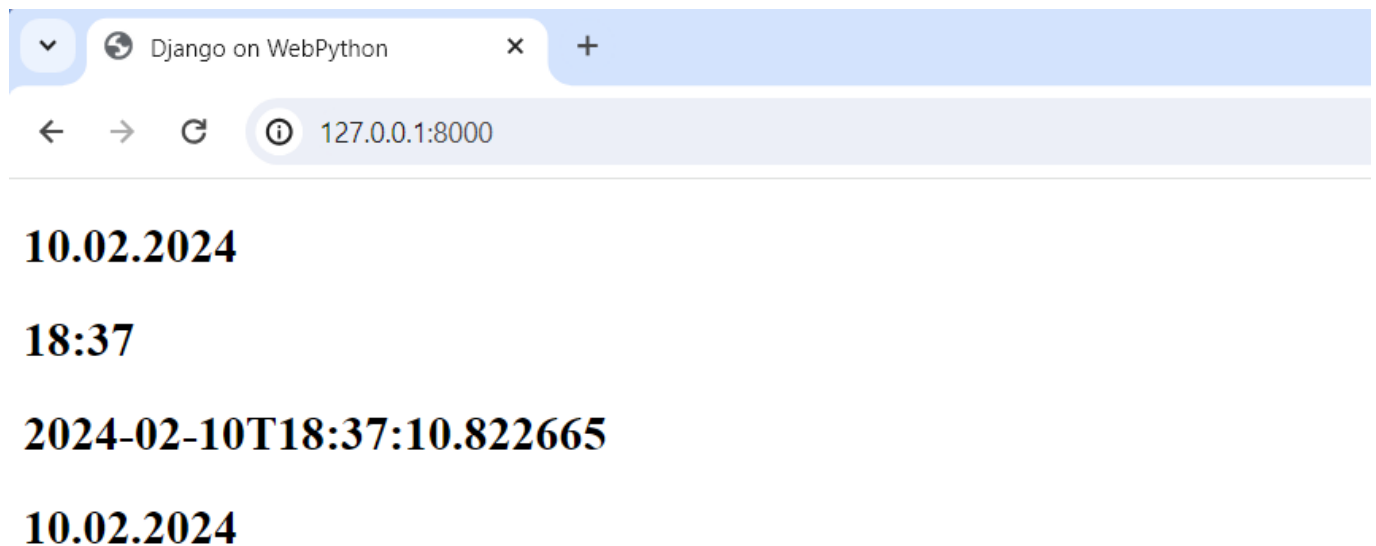


Рисунок 3.13 – Результат форматування дати

3.4.7 Операції зі списками

Низка фільтрів призначені для роботи зі списками.

Фільтр *join* об'єднує елементи списку, використовуючи певний роздільник:

```
{{ users|join:", "}}
```

Припустимо, що тут *users* - це список `["Sir", "Alexander", "Chapman", "Ferguson"]`, то після застосування фільтра вийде рядок `"Sir, Alexander, Chapman, Ferguson"`.

Фільтр *slice* отримує частину списку. Для вилучення частини списку він отримує початковий і кінцевий індекси для вилучення елементів:

```
slice: "start:"
slice: ":end"
slice: "start:end"
```

параметр *start* вказує на індекс елемента, починаючи з якого треба скопіювати елементи, а через параметр *end* передається індекс елемента, до якого потрібно копіювати список. Якщо *start* або *end* не вказуються, то *start* за замовчуванням дорівнює 0, а *end* - на довжину списку. Наприклад:

```
{{ {{ users|slice: "1:3" }}}}
```

Тут витягуються елементи з 1 по 3 індекс (не включаючи), тобто вийде список `["Alexander", "Chapman"]`.

Фільтр *length* повертає довжину списку (також цей фільтр можна застосовувати для знаходження довжини рядка).

```
{{ users|length}}
```

Тут фільтр поверне 4.

3.5 Статичні файли

Веб-додаток, як правило, використовує різні статичні файли - зображення, файли стилів *css*, скриптів *javascript* тощо. Розглянемо, як ми можемо використовувати подібні файли.

3.5.1 Розміщення за замовченням

При створенні проекту *Django* він уже має деяке базове налаштування для роботи зі статичними файлами. Зокрема, у файлі `settings.py` визначено змінну `STATIC_URL`, яка зберігає шлях до каталогу зі статичними файлами:

```
# Static files (CSS, JavaScript, Images)
# https://docs.djangoproject.com/en/4.2/howto/static-files/

STATIC_URL = '/static/'
```

А серед встановлених додатків у змінній `INSTALLED_APPS` вказано додаток `django.contrib.staticfiles`

```
# Application definition

INSTALLED_APPS = [
    'django.contrib.admin',
    'django.contrib.auth',
    'django.contrib.contenttypes',
    'django.contrib.sessions',
    'django.contrib.messages',
    'django.contrib.staticfiles',
    'hello',
]
```

Змінна `STATIC_URL` має значення `"static/"`, а це значить, що нам достатньо створити в папці додатка каталог з ім'ям `"static"` і додати в нього необхідні нам статичні файли. Але, звісно, за необхідності через це налаштування ми можемо змінити розташування каталогу статичних файлів.

Отже, додамо в папку програми новий каталог `static`. Щоб не звалювати всі статичні файли до купи, визначимо для кожного типу файлів окремі папки. Зокрема, створимо в папці `static` для зображень каталог `images`, а для стилів - каталог `css`. Подібним чином можна створювати папки і для інших типів файлів.

У папку *static/images* додамо якесь зображення - у моєму випадку це буде файл *2024-k-12-winter.jpg*. А в папці *static/css* визначимо новий файл *styles.css*, який матиме якісь найпростіші стилі, наприклад:

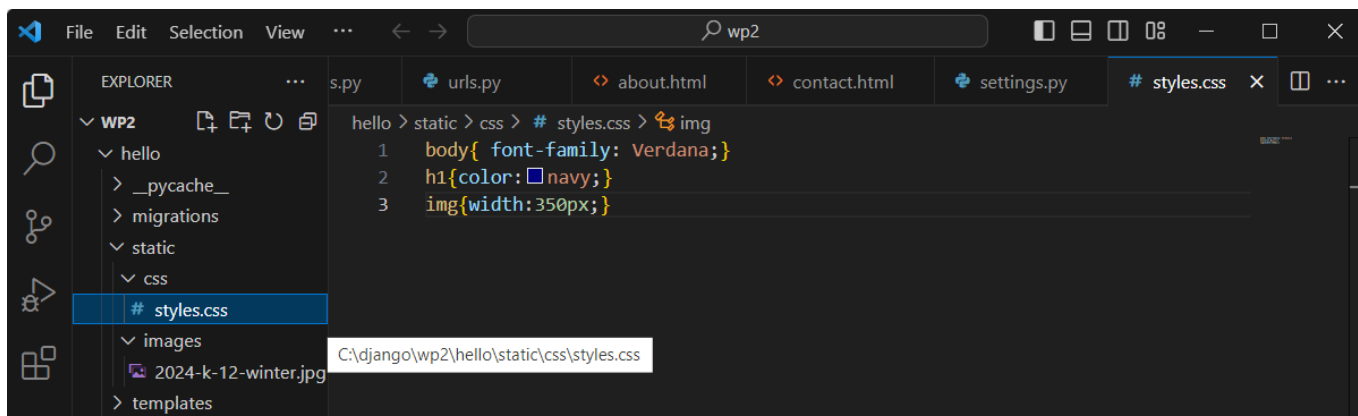


Рисунок 3.14 – Зміст теки *static*

Тепер використовуємо ці файли в шаблоні. Для цього на початку файлу шаблону необхідно визначити інструкцію

```
{% load static %}
```

При цьому цей код має йти після тега *DOCTYPE*.

Для визначення шляху до статичних файлів використовуються вирази типу

```
{% static "шлях до файлу всередині папки static" %}
```

Так, нехай у додатку в папці *templates* визначено шаблон *index.html*, який має такий код:

```
<!DOCTYPE html>
{% load static %}
<html>
<head>
    <meta charset="utf-8" />
    <link rel="stylesheet" href="{% static 'css/styles.css' %}" />
    <title>Django on WebPython</title>
</head>
<body>
    <h1>Зимовий універ</h1>
    
    <div id="block1">
        <p>Протокол засідання ректорату від 5 лютого 2024 року</p>
        <a href="https://www.kpi.ua/2024-02-05">Відкрити протокол</a></div>
</body>
</html>
```

Під час запуску програми шаблон *index.html* генеруватиметься в наступну веб-сторінку, яка використовуватиме зображення та застосовуватиме стилі:

3.5.2 Налаштування шляхів до файлів

Якщо нас не влаштовує зберігання файлів у каталозі за замовчуванням - каталозі *static*, або ми хочемо вказати кілька папок, то ми можемо у файлі *settings.py* задати всі необхідні каталоги за допомогою змінної *STATICFILES_DIRS*, яка приймає список шляхів:

```
STATICFILES_DIRS = [
    BASE_DIR / "static",
    "/var/www/static/",
    "/somefolder/"
]
```

3.6 Клас `TemplateView`

У попередніх темах, коли надходив запит, система маршрутизації обирала потрібне подання, і те вже своєю чергою використовувало шаблон для генерації відповіді. Але в принципі якщо у відповідь на запит нам просто треба повернути користувачеві вміст шаблону, то для цього необов'язково визначати функцію-представлення. Можна скористатися вбудованим класом *TemplateView*.

3.6.1 Визначення шаблонів

Так, визначимо кілька найпростіших шаблонів. Нехай це буде файл *about.html* із таким кодом:

```
<!DOCTYPE html>
<html>
<head>
  <meta charset="utf-8" />
  <title>Hello, Django students</title>
</head>
<body>
  <h1>About</h1>
</body>
</html>
```

І також визначимо шаблон *contact.html* з таким кодом:

```
<!DOCTYPE html>
<html>
<head>
  <meta charset="utf-8" />
  <title>Hello, WebPython students</title>
</head>
<body>
  <h1>Contact</h1>
</body>
</html>
```

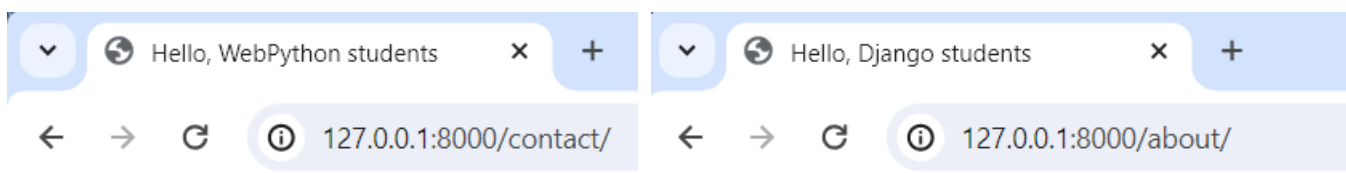
Змінимо файл *urls.py* наступним чином:

```
from django.urls import path
from hello import views
from django.views.generic import TemplateView

urlpatterns = [
    path("", views.index),
    path("about/", TemplateView.as_view(template_name="about.html")),
    path("contact/", TemplateView.as_view(template_name="contact.html")),
]
```

Фактично *TemplateView* сам по собі надає функціональність подання. За допомогою методу *as_view()* через параметр *template_name* встановлюється шлях до шаблону, який буде використовуватися як відповідь.

У підсумку при зверненні за відповідними адресами *URL* ми отримаємо такі результати:



Contact

About

3.6.2 Передання даних у шаблони

За допомогою параметра *extra_context* у метод *as_view* можна передати дані для шаблону. Дані повинні представляти словник. Наприклад:

```
from django.urls import path
from hello import views
from django.views.generic import TemplateView

urlpatterns = [
    path("", views.index),
    path("about/", TemplateView.as_view(template_name="about.html",
        extra_context={"header": "About project"})),
    path("contact/", TemplateView.as_view(template_name="contact.html")),
]
```

Тут у шаблон *"about.html"* передається об'єкт *header*, який представляє рядок *"About project"*. І в цьому разі ми можемо використовувати цей об'єкт у шаблоні:

```
<!DOCTYPE html>
<html>
<head>
  <meta charset="utf-8" />
  <title>Hello, Django students</title>
</head>
<body>
  <h1>{{ header }}</h1>
</body>
</html>
```

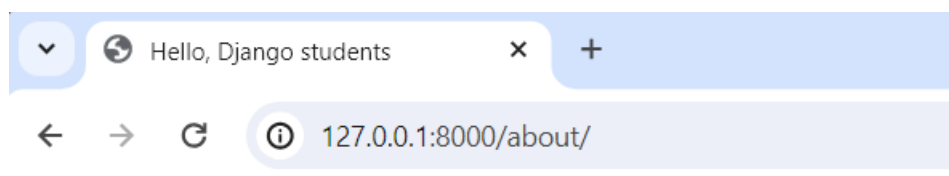


Рисунок 3.15 – Результат передавання даних у шаблон

3.7 Конфігурація шаблонів

За конфігурацію шаблонів у проєкті відповідає змінна *TEMPLATES* у файлі *settings.py*.

3.7.1 Налаштування за замовченням

За замовчуванням вона має такий вміст:

```
TEMPLATES = [
    {
        'BACKEND': 'django.template.backends.django.DjangoTemplates',
        'DIRS': [],
        'APP_DIRS': True,
        'OPTIONS': {
            'context_processors': [
                'django.template.context_processors.debug',
                'django.template.context_processors.request',
                'django.contrib.auth.context_processors.auth',
                'django.contrib.messages.context_processors.messages',
            ],
        },
    },
]
```

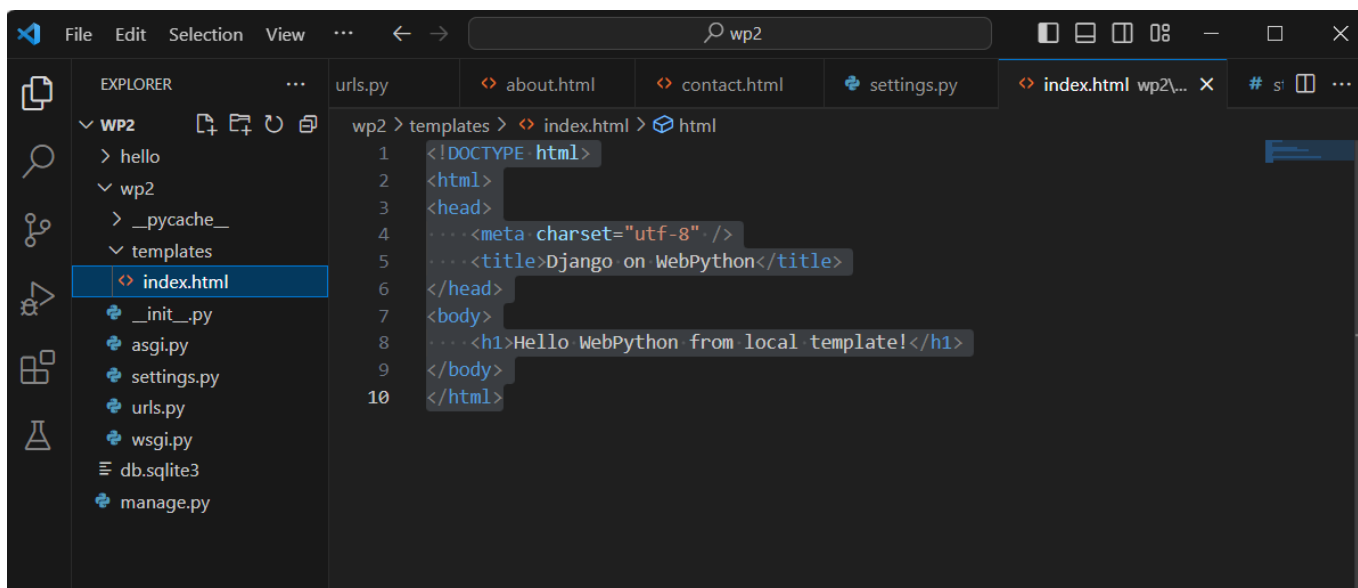
Ця змінна приймає список конфігурацій для кожного движка шаблонів. За замовчуванням визначено одну конфігурацію, яка має такі параметри:

- *BACKEND*: движок шаблонів. За замовчуванням застосовується вбудований движок `django.template.backends.django.DjangoTemplates`
- *DIRS*: визначає список каталогів, де рушій шаблонів буде шукати файли шаблонів. За замовчуванням порожній список
- *APP_DIRS*: вказує, чи буде рушій шаблонів шукати шаблони всередині папок додатків у папці `templates`.
- *OPTIONS*: визначає додатковий список параметрів. Зокрема, вказує, які обробники (процесори) будуть використовуватися під час обробки шаблонів.

3.7.2 Шляхи до шаблонів

За замовчуванням параметр *APP_DIRS* має значення *True*, а це означає, що рушій шаблонів буде також шукати потрібні файли шаблонів у папці застосунку в каталозі *templates*. Це досить зручно, якщо для кожної програми призначені якісь свої шаблони. Це також дає змогу уникнути проблем із найменуваннями.

Проте іноді виникає необхідність розміщення шаблонів десь в іншому місці. Наприклад, у нас є загальні шаблони для всього проєкту - для всіх застосунків, і було б логічно розмістити їх у якомусь одному спільному місці, наприклад, у папці проєкту. Наприклад, визначимо в каталозі проєкту папку *templates* (назва папки може бути будь-якою). І в цій папці визначимо файл *index.html* з яким-небудь найпростішим кодом:



У навчальному прикладі проєкт має назву *wp2*, який розташовується в однойменній папці. У проєкті знаходиться папка *templates* з одним шаблоном. Але щоб цей шаблон задіяти, треба налаштувати шляхи до цієї папки у файлі *setting.py*. Так, змінимо змінну *TEMPLATES*:

```
TEMPLATES = [
    {
        'BACKEND': 'django.template.backends.django.DjangoTemplates',
        'DIRS': [
            BASE_DIR / "wp2/templates"
        ],
        'APP_DIRS': True,
        'OPTIONS': {
            'context_processors': [
                'django.template.context_processors.debug',
                'django.template.context_processors.request',
                'django.contrib.auth.context_processors.auth',
                'django.contrib.messages.context_processors.messages',
            ],
        },
    },
]
```

```
    },
},
]
```

У цьому разі до списку папок `DIRS` додається один шлях - `BASE_DIR / "wp2/templates"`. Для утворення шляху береться визначена на початку файлу `settings.py` змінна `BASE_DIR`:

```
BASE_DIR = Path(__file__).resolve().parent.parent
```

яка представляє шлях до каталогу, в якому розташовується папка проекту. І до цього шляху додається папка проекту (`wp2`) і каталог `templates`.

Нехай у `views.py` визначено функцію, яка використовує шаблон `index.html`:

```
from django.shortcuts import render

def index(request):
    return render(request, "index.html")

def about(request):
    return render(request, "about.html")

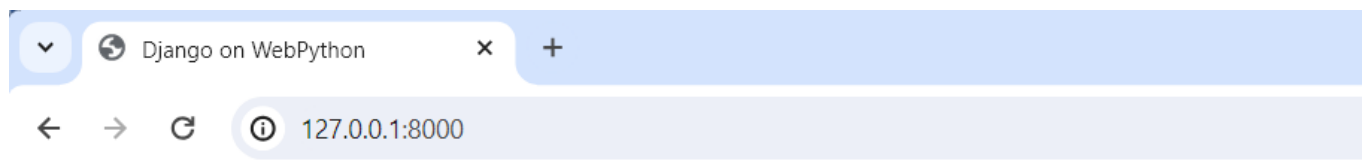
def contact(request):
    return render(request, "contact.html")
```

І у файлі `urls.py` визначено маршрут для цієї функції:

```
from django.urls import path
from hello import views

urlpatterns = [
    path("", views.index),
```

Відповідно при зверненні до функції `index()` ми побачимо в браузері вміст файлу `index.html`:



Hello WebPython from local template!

Рисунок 3.16 – Результат зміни шляху до шаблону

Подібним чином ми можемо вказати як сховище шаблонів будь-який каталог усередині проєкту або додатка чи навіть набір каталогів.

3.8 Розширення шаблонів і фільтр `extends`

Нерідко шаблони повинні мати однакову базову структуру, одні й ті самі блоки, при цьому визначати для окремих блоків різний вміст. Це дає змогу сформувати однаковий стиль сайту, коли веб-сторінки мають одні й ті самі структурні елементи - меню, хедер, футер, сайдбари тощо.

У цьому разі ми можемо визначати всі шаблони окремо. Однак якщо виникне необхідність змінити якийсь блок, наприклад, додати в загальне меню ще один пункт, тоді доведеться міняти всі шаблони, яких може бути досить багато. І в цьому разі оптимальніше повторно використовувати один базовий шаблон, який визначає всі основні блоки.

Наприклад, визначимо шаблон, який назовемо *base.html*:

```
<!DOCTYPE html>
<html>
<head>
  <meta charset="utf-8" />
  <title>{% block title %}Default title{% endblock title %}</title>
</head>
<body>
  <div><a href="/">Home page</a> | <a href="/contacts">Contacts</a></div>
  <h1>{% block header %}{% endblock header %}</h1>
  <div>{% block content %}{% endblock content %}</div>
  <div>MyCorp. 2024. All rights reserved.</div>
</body>
</html>
```

За допомогою елементів *{% block назва_блока %}{% endblock назва_блока %}* визначаються окремі блоки шаблонів. При цьому для кожного блоку визначається елемент, що відкриває *{% block назва_блока %}* і елемент, що закриває *{% endblock назва_блока %}*.

Наприклад, блок *title*:

```
{% block title %}Default title{% endblock title %}
```

Коли інші шаблони застосовуватимуть цей шаблон, то вони можуть визначити для блоку *title* якийсь свій вміст.

Для кожного блоку можна визначити вміст за замовчуванням. Так, для блоку *title* це рядок *"Default title"*. І якщо інші шаблони, які використовуватимуть цей шаблон, не визначать вміст для блоку *title*, то цей блок використовуватиме рядок *"Default title"*.

Подібним чином тут визначено блоки *header* і *content*. Вміст за замовчуванням для блоків визначати не обов'язково. Самих блоків за потреби можна визначити скільки завгодно.

Крім того, в базовому шаблоні визначено навігаційне меню з двох посилань. Але оскільки воно буде загальним для всіх шаблонів, то для нього не визначено окремий блок. І також визначено футер, який теж буде спільним для всіх сторінок.

Тепер застосуємо цей базовий шаблон. Наприклад, створимо новий шаблон *index.html*:

```
{% extends "base.html" %}
{% block title %}Index{% endblock title %}
{% block header %}Main{% endblock header %}
```

За допомогою виразу `{% extends "base.html" %}` визначаємо, який базовий шаблон буде розширюватися. Потім визначається вміст для блоків *title* і *header*. Варто зазначити, що необов'язково вказувати вміст для всіх блоків базового шаблону.

Також створимо шаблон *contacts.html*:

```
{% extends "base.html" %}
{% block title %}Contacts{% endblock title %}
{% block header %}Contacts{% endblock header %}

{% block content %}
<p>Phone: +380987654321</p>
<p>Email: admin@webpython.ua</p>
{% endblock content %}
```

Цей шаблон також розширює базовий шаблон *base.html*. На відміну від *index.html* тут також визначається вміст для блоку *content*.

Таким чином, подібна функціональність дає нам змогу уникнути повторення одних і тих самих елементів у різних шаблонах. І водночас якщо нам потрібно щось змінити, наприклад, структуру сторінки або якийсь загальний елемент, то достатньо це зробити в одному файлі - *base.html*.

Нехай у файлі *views.py* є функції, які використовують ці шаблони:

```
from django.shortcuts import render

def index(request):
    return render(request, "index.html")

def contacts(request):
    return render(request, "contacts.html")
```

А у файлі *urls.py* ці функції зіставляються з певними маршрутами:

```
from django.urls import path
from hello import views

urlpatterns = [
    path("", views.index),
    path("contacts/", views.contacts),
]
```

У підсумку незалежно від того, до якої функції ми звернемося, ми отримаємо на веб-сторінці одну й ту саму структуру зі спільними меню і футером:

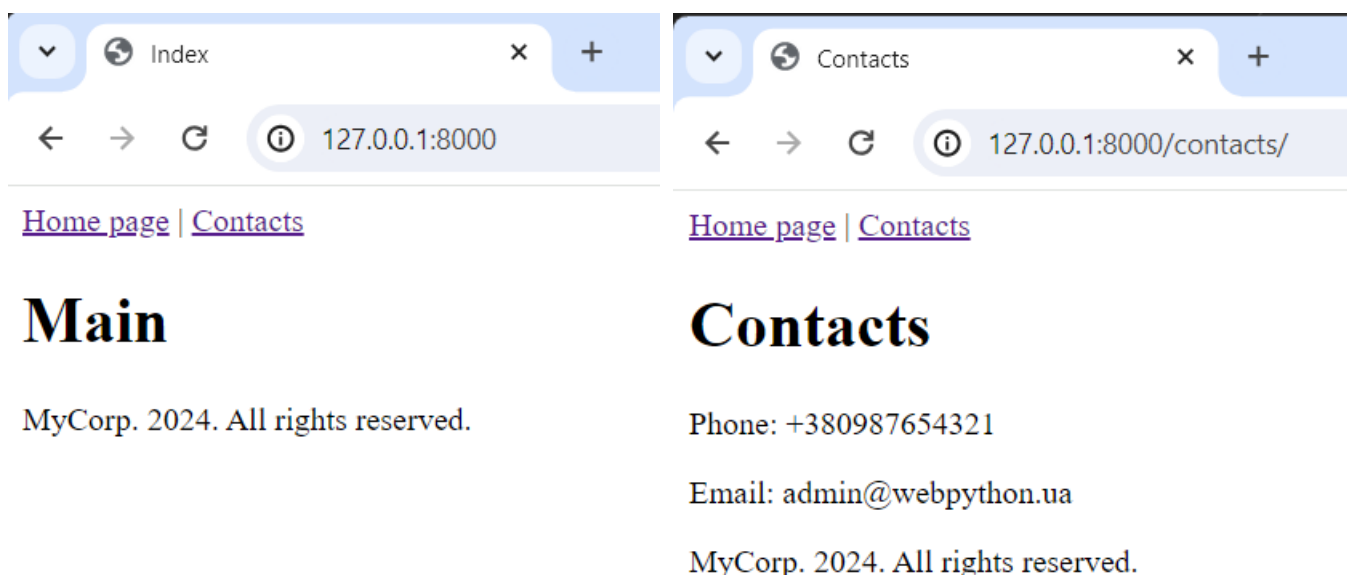


Рисунок 3.17 – Результати дії фільтру `extends`

У підсумку, якщо нам потрібно змінити структуру всіх веб-сторінок сайту, додати нові елементи або прибрати старі, то достатньо буде змінити один базовий шаблон.

3.9 Вкладені шаблони

3.9.1 Фільтр include

Фільтр *include* дає змогу додати в один шаблон інший шаблон. Назву шаблону, що підключається, передають фільтру як параметр:

```
{% include підключаємий_файл %}
```

Припустимо, у застосунку в нас є два шаблони: *index.html* і *banner.html*.

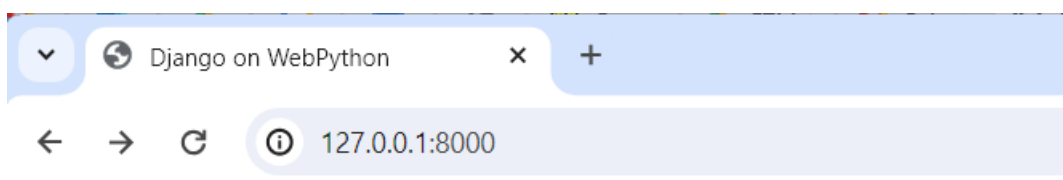
Нехай у файлі *banner.html* буде який-небудь найпростіший код:

```
<div>Лабораторна робота WebPython: setup, urls, views, templates</div>
```

В іншому файлі - *index.html* підключимо цей шаблон:

```
<!DOCTYPE html>
<html>
<head>
  <meta charset="utf-8" />
  <title>Django on WebPython</title>
</head>
<body>
<h2>Home page</h2>
{% include "banner.html" %}
</body>
</html>
```

У підсумку під час звернення до шаблону *index.html* на веб-сторінці з'явиться вміст із підключеного шаблону *banner.html*.



Home page

Лабораторна робота WebPython: setup, urls, views, templates

Рисунок 3.18 – Результат роботи вкладених шаблонів

Варто зазначити, що замість жорсткого встановлення шляху до вкладеного шаблону у вигляді рядка, можна використовувати змінні:

```
{% include template_name %}
```

У цьому випадку передбачається, що `template_name` - це змінна, яка зберігає шлях до вкладеного шаблону.

3.9.2 Передання даних

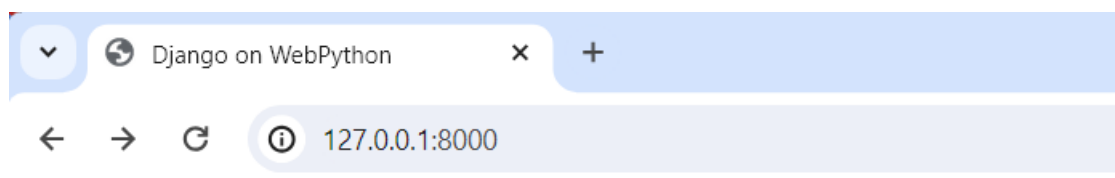
У вкладені шаблони також можна передавати деякі дані. Наприклад, змінимо код у *banner.html* таким чином:

```
<div>Лабораторна робота {{ course }}: {{ subject}}</div>
```

Тут використовуються змінні *course* і *subject*. Тепер змінимо код *index.html*, передавши для цих змінних деякі дані:

```
<!DOCTYPE html>
<html>
<head>
  <meta charset="utf-8" />
  <title>Django on WebPython</title>
</head>
<body>
<h2>Home page</h2>
{% include "banner.html" with course="WebPython" subject="setup, urls, views,
templates" %}
</body>
</html>
```

Для передачі даних для фільтра визначаються дві змінні: *course* і *subject*. Оголошення змінних іде після оператора *with*.



Home page

Лабораторна робота WebPython: setup, urls, views, templates

Рисунок 3.19 – Результат передачі даних

3.9.2 Передача даних із батьківського шаблону

Вкладені шаблони автоматично підхоплюють дані їхніх батьківських шаблонів. Наприклад, візьмемо вище визначений шаблон *banner.html*:

```
<div>Лабораторна робота {{ course }}: {{ subject}}</div>
```

У файлі *views.py* визначимо таку функцію-представлення:

```
from django.shortcuts import render

def index(request):
    return render(request, "index.html", context={"subject": "setup, urls, views, templates"})
```

Тут у шаблон *index.html* передається елемент *subject*. Тепер змінимо код *index.html*:

```
<!DOCTYPE html>
<html>
<head>
    <meta charset="utf-8" />
    <title>Django on WebPython</title>
</head>
<body>
<h2>Home page</h2>
{% include "banner.html" with course="WebPython" %}
</body>
</html>
```

Тепер для шаблону *banner.html* НЕ створюється змінна *subject*, тому що вкладений шаблон автоматично отримає дані за ключем "*subject*" з батьківського шаблону *index.html*. Однак ця поведінка не завжди зручна. Ми можемо заборонити автоматичну передачу даних із батьківського у вкладений шаблон, використовуючи оператор *only*:

```
{% include "banner.html" with course="WebPython" only %}
```

У цьому випадку шаблон *banner.html* не отримає дані за ключем "*subject*"

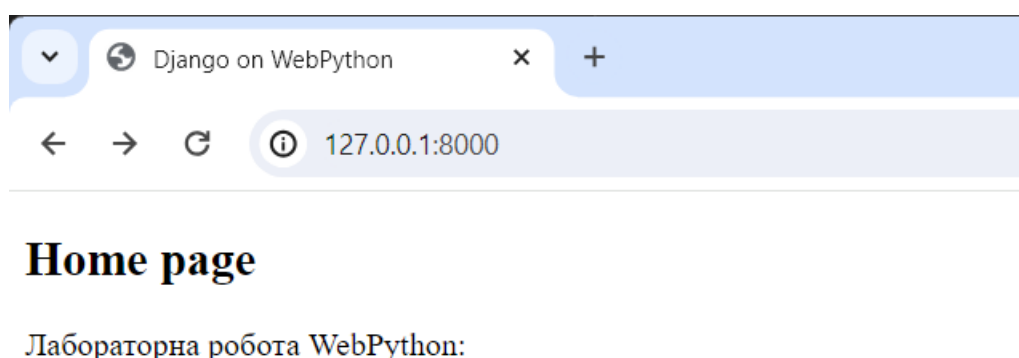


Рисунок 3.20 – Результат передачі даних із батьківського шаблону

